ROYAL COLLEGE OF ARTS, SCIENCE & COMMERCE

(AUTONOMOUS)

Mira Road (East), MiraBhayandar, Maharashtra 401107.

DEPARTMENT OF DATA SCIENCE

M.Sc. Data Science (Part II) – **Semester III**

Course Code: **RPDSMJP301**

Course Name: **Next Generation Database Technologies**

Practical Journal

(Academic Year: 2026-2027)

Seat No: _____

Since 1989

Devoted to Serve

Royal Higher Education Society's

ROYAL COLLEGE
OF ARTS, SCIENCE & COMMERCE

Empowerment through Value Education

(A Minority Autonomous Institution)

ROYAL COLLEGE OF ARTS, SCIENCE & COMMERCE (AUTONOMOUS)

Mira Road (East),MiraBhayandar,Maharashtra 401107.

Department of Data Science

CERTIFICATE

This is to certify that **Mr. Mohd Danish Ansari**, a student of **M.Sc. Data Science (Part II) - Semester III**, having **Roll No. 13** and **Seat No. _____**, has satisfactorily completed **08 Practicals** in the subject of **Next Generation Database Technologies** as part of the **M.Sc. Data Science Programme** during the academic year **2026 – 2027**.

Place: Mira Road(E)

Date:

Subject In-charge

Co-Ordinator,
M.Sc. Data Science

Signature of Examiner

INDEX

Sr. No	Aim	Date	Sign
1	To design a relational database schema using ER modeling concepts, implement it by creating tables and performing insert and update operations, and normalize the schema up to Third Normal Form (3NF).	28/07/26	
2	To perform complex data retrieval operations using SQL by executing queries involving joins, aggregation functions, grouping, ordering, and nested (sub) queries to analyze and extract meaningful information from the dataset.	11/08/26	
3	To implement advanced database programming concepts by creating and managing database objects such as views, sequences, triggers, and stored procedures to automate data handling and enforce business logic.	28/08/26	
4	To apply data mining techniques such as classification and clustering on a real-world dataset to discover hidden patterns, relationships, and insights for knowledge discovery.	12/09/26	
5	To perform file storage, retrieval, and management operations using Hadoop Distributed File System (HDFS), and to explore the Hadoop environment and architecture through command-line interface and web interface.	12/09/26	
6	To create and manage structured data using Apache Hive by defining managed and external tables, and to perform NoSQL data storage and retrieval operations using HBase including table creation and data manipulation.	19/09/26	

7	**To perform Create, Read, Update, and Delete (CRUD) operations and execute advanced queries using MongoDB, including the use of conditional operators, indexing, and sorting on document based data.**	**26/09/26**	
8	**To implement distributed data storage using Cassandra by creating key spaces and tables and performing data operations, and to model and analyze relationships using a graph database through Neo4j by creating nodes, relationships, and executing Cypher queries.**	**30/09/26**	

Practical No: 01
Name: Mohd Danish Ansari
Roll No: 13
Date: 28/07/26

AIM : To design a relational database schema using ER modeling concepts, implement it by creating tables and performing insert and update operations, and normalize the schema up to Third Normal Form (3NF). (College ERP scenario can be considered as an example.)

Code :

-- Create Database

-- Display Available Databases

-- Select Database

```
mysql> CREATE DATABASE bank;
Query OK, 1 row affected (0.01 sec)

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| bank               |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.00 sec)

mysql> USE bank;
Database changed
```

-- Create Customer Table

-- Display Customer Table Structure

```
mysql> CREATE TABLE customer(
    ->     cust_id INT PRIMARY KEY,
    ->     cust_name VARCHAR(100),
    ->     cust_city VARCHAR(100),
    ->     cust_age INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> DESC customer;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| cust_id   | int          | NO   | PRI | NULL    |       |
| cust_name | varchar(100) | YES  |     | NULL    |       |
| cust_city | varchar(100) | YES  |     | NULL    |       |
| cust_age  | int          | YES  |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
4 rows in set (0.01 sec)
```

-- Create Branch Table

-- Display Branch Table Structure

```
mysql> CREATE TABLE branch(
    ->     branch_name VARCHAR(100),
    ->     branch_city VARCHAR(100),
    ->     assets REAL
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> DESC branch;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| branch_name | varchar(100) | YES  |     | NULL    |       |
| branch_city | varchar(100) | YES  |     | NULL    |       |
| assets      | double       | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
3 rows in set (0.00 sec)
```

-- Add Primary Key Constraint on Branch Name

```
mysql> ALTER TABLE branch
    -> ADD PRIMARY KEY(branch_name);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

-- Create Account Table with Foreign Key

-- Display Account Table Structure

```
mysql> CREATE TABLE account(
    ->     account_id VARCHAR(50) PRIMARY KEY,
    ->     balance REAL,
    ->     branch_name VARCHAR(100) REFERENCES branch(branch_name)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> DESC account;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| account_id  | varchar(50)  | NO   | PRI | NULL    |       |
| balance     | double       | YES  |     | NULL    |       |
| branch_name | varchar(100) | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
3 rows in set (0.00 sec)
```

-- Create Depositor Table

-- Add Foreign Key Constraint for Customer

-- Add Foreign Key Constraint for Account

-- Display Depositor Table Structure

```
mysql> CREATE TABLE depositor(
    ->     cust_id INT,
    ->     account_id VARCHAR(50),
    ->     PRIMARY KEY(cust_id, account_id)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> ALTER TABLE depositor
    -> ADD CONSTRAINT fk1
    -> FOREIGN KEY(cust_id)
    -> REFERENCES customer(cust_id);
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE depositor
    -> ADD CONSTRAINT fk2
    -> FOREIGN KEY(account_id)
    -> REFERENCES account(account_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESC depositor;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| cust_id    | int         | NO   | PRI | NULL    |       |
| account_id | varchar(50) | NO   | PRI | NULL    |       |
+------------+-------------+------+-----+---------+-------+
2 rows in set (0.00 sec)
```

-- Create Loan Table

-- Display Loan Table Structure

```
mysql> CREATE TABLE loan(
    ->     loan_no INT PRIMARY KEY,
    ->     loan_amount FLOAT,
    ->     branch_name VARCHAR(200),
    ->     FOREIGN KEY(branch_name) REFERENCES branch(branch_name)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> DESC loan;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| loan_no     | int          | NO   | PRI | NULL    |       |
| loan_amount | float        | YES  |     | NULL    |       |
| branch_name | varchar(200) | YES  | MUL | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
3 rows in set (0.00 sec)
```

-- Create Borrower Table

-- Add Foreign Key Constraint for Customer

-- Add Foreign Key Constraint for Loan

-- Display Borrower Table Structure

```
mysql> CREATE TABLE borrower(
    ->     cust_id INT,
    ->     loan_no INT,
    ->     PRIMARY KEY(cust_id, loan_no)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> ALTER TABLE borrower
    -> ADD CONSTRAINT fk_borrower_customer
    -> FOREIGN KEY(cust_id)
    -> REFERENCES customer(cust_id);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE borrower
    -> ADD CONSTRAINT fk_borrower_loan
    -> FOREIGN KEY(loan_no)
    -> REFERENCES loan(loan_no);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESC borrower;
+---------+------+------+-----+---------+-------+
| Field   | Type | Null | Key | Default | Extra |
+---------+------+------+-----+---------+-------+
| cust_id | int  | NO   | PRI | NULL    |       |
| loan_no | int  | NO   | PRI | NULL    |       |
+---------+------+------+-----+---------+-------+
2 rows in set (0.00 sec)
```

-- INSERT OPERATIONS

-- Insert Customer Records

-- Display Customer Records

```
mysql> INSERT INTO customer VALUES
    -> (1, 'Rahul', 'Mumbai', 35),
    -> (2, 'Raj', 'Pune', 30),
    -> (3, 'Aman', 'Mumbai', 28),
    -> (4, 'Amit', 'Mumbai', 19),
    -> (5, 'Shifa', 'Nagpur', 20),
    -> (6, 'Riya', 'Pune', 32),
    -> (7, 'Rajesh', 'Mumbai', 25),
    -> (8, 'Sara', 'Nagpur', 26),
    -> (9, 'Nikhat', 'Pune', 20),
    -> (10, 'Arushi', 'Mumbai', 22);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM customer;
+---------+-----------+-----------+----------+
| cust_id | cust_name | cust_city | cust_age |
+---------+-----------+-----------+----------+
|       1 | Rahul     | Mumbai    |       35 |
|       2 | Raj       | Pune      |       30 |
|       3 | Aman      | Mumbai    |       28 |
|       4 | Amit      | Mumbai    |       19 |
|       5 | Shifa     | Nagpur    |       20 |
|       6 | Riya      | Pune      |       32 |
|       7 | Rajesh    | Mumbai    |       25 |
|       8 | Sara      | Nagpur    |       26 |
|       9 | Nikhat    | Pune      |       20 |
|      10 | Arushi    | Mumbai    |       22 |
+---------+-----------+-----------+----------+
10 rows in set (0.00 sec)
```

-- Insert Branch Records

-- Display Branch Records

```
mysql> INSERT INTO branch VALUES
    -> ('Malad', 'Mumbai', 5000000),
    -> ('Goregaon', 'Mumbai', 7500000),
    -> ('Mira Road', 'Mumbai', 8000000),
    -> ('Kothrud', 'Pune', 2000000),
    -> ('Dharamsagar', 'Nagpur', 1000000);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM branch;
+-------------+-------------+---------+
| branch_name | branch_city | assets  |
+-------------+-------------+---------+
| Dharamsagar | Nagpur      | 1000000 |
| Goregaon    | Mumbai      | 7500000 |
| Kothrud     | Pune        | 2000000 |
| Malad       | Mumbai      | 5000000 |
| Mira Road   | Mumbai      | 8000000 |
+-------------+-------------+---------+
5 rows in set (0.00 sec)
```

-- Insert Account Records

-- Display Account Records

```
mysql> INSERT INTO account VALUES
    -> ('a1', 50000, 'Malad'),
    -> ('a2', 60000, 'Goregaon'),
    -> ('a3', 10000, 'Mira Road'),
    -> ('a4', 20000, 'Malad'),
    -> ('a5', 100000, 'Goregaon'),
    -> ('a6', 35000, 'Kothrud'),
    -> ('a7', 5000, 'Kothrud'),
    -> ('a8', 15000, 'Dharamsagar');
Query OK, 8 rows affected (0.01 sec)
Records: 8  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM account;
+------------+---------+-------------+
| account_id | balance | branch_name |
+------------+---------+-------------+
| a1         |   50000 | Malad       |
| a2         |   60000 | Goregaon    |
| a3         |   10000 | Mira Road   |
| a4         |   20000 | Malad       |
| a5         |  100000 | Goregaon    |
| a6         |   35000 | Kothrud     |
| a7         |    5000 | Kothrud     |
| a8         |   15000 | Dharamsagar |
+------------+---------+-------------+
8 rows in set (0.00 sec)
```

-- Insert Loan Records

-- Display Loan Records

```
mysql> INSERT INTO loan VALUES
    -> (1, 25000, 'Malad'),
    -> (2, 10000, 'Goregaon'),
    -> (3, 50000, 'Mira Road'),
    -> (4, 60000, 'Kothrud'),
    -> (5, 80000, 'Dharamsagar');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM loan;
+---------+-------------+-------------+
| loan_no | loan_amount | branch_name |
+---------+-------------+-------------+
|       1 |       25000 | Malad       |
|       2 |       10000 | Goregaon    |
|       3 |       50000 | Mira Road   |
|       4 |       60000 | Kothrud     |
|       5 |       80000 | Dharamsagar |
+---------+-------------+-------------+
5 rows in set (0.00 sec)
```

-- Insert Depositor Records

-- Display Depositor Records

```
mysql> INSERT INTO depositor VALUES
    -> (1,'a1'),
    -> (2,'a2'),
    -> (3,'a3'),
    -> (4,'a4'),
    -> (5,'a5'),
    -> (6,'a6'),
    -> (7,'a7'),
    -> (8,'a8');
Query OK, 8 rows affected (0.01 sec)
Records: 8  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM depositor;
+---------+------------+
| cust_id | account_id |
+---------+------------+
|       1 | a1         |
|       2 | a2         |
|       3 | a3         |
|       4 | a4         |
|       5 | a5         |
|       6 | a6         |
|       7 | a7         |
|       8 | a8         |
+---------+------------+
8 rows in set (0.00 sec)
```

-- Insert Borrower Records

-- Display Borrower Records

```
mysql> INSERT INTO borrower VALUES
    -> (6,1),
    -> (7,2),
    -> (8,3),
    -> (9,4),
    -> (10,5);
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM borrower;
+---------+---------+
| cust_id | loan_no |
+---------+---------+
|       6 |       1 |
|       7 |       2 |
|       8 |       3 |
|       9 |       4 |
|      10 |       5 |
+---------+---------+
5 rows in set (0.00 sec)
```

-- UPDATE OPERATIONS

-- 1. Change the age of customer with ID 9 to 27

-- 2. Increase the assets value of Kothrud branch by 10%

-- 3. Change the balance of account no. a7 by adding Rs. 1000 -- 4. Increase loan amount by 10% for loans taken from Goregaon branch -- 5. For customers whose name begins with alphabet 'A', change city to Mumbai -- 6. For customers with customer ID 1 change name to Ramesh and age to 36

```
mysql> UPDATE customer
    -> SET cust_age = 27
    -> WHERE cust_id = 9;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE branch
    -> SET assets = assets * 1.10
    -> WHERE branch_name = 'Kothrud';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE account
    -> SET balance = balance + 1000
    -> WHERE account_id = 'a7';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE loan
    -> SET loan_amount = loan_amount * 1.10
    -> WHERE branch_name = 'Goregaon';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE customer
    -> SET cust_city = 'Mumbai'
    -> WHERE cust_name LIKE 'A%';
Query OK, 0 rows affected (0.00 sec)
Rows matched: 3  Changed: 0  Warnings: 0

mysql>
mysql> UPDATE customer
    -> SET cust_name = 'Ramesh', cust_age = 36
    -> WHERE cust_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

-- ALTER QUERIES

-- 1. Add a new column

-- 2. Drop a column

-- 3. Add multiple columns

-- 4. Modify the datatype/size of a column

-- 5. Rename a column

```
mysql> ALTER TABLE customer
    -> ADD email VARCHAR(255);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> ALTER TABLE customer
    -> DROP COLUMN email;
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> ALTER TABLE customer
    -> ADD (
    ->     email VARCHAR(255),
    ->     pincode INT
    -> );
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> ALTER TABLE customer
    -> ADD street VARCHAR(255);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> ALTER TABLE customer
    -> MODIFY street VARCHAR(50);
Query OK, 10 rows affected (0.06 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql>
mysql> ALTER TABLE customer
    -> CHANGE COLUMN street street_name VARCHAR(50);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

-- Q1. Change column name 'loan_amount' to 'loan_amt' in loan

table -- Q2. Change datatype of 'name' column to VARCHAR(200)

-- Q3. Add 'acc_type' column to account table

```
mysql>
mysql> ALTER TABLE loan
    -> CHANGE COLUMN loan_amount loan_amt FLOAT;
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> ALTER TABLE customer
    -> MODIFY cust_name VARCHAR(200);
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> ALTER TABLE account
    -> ADD acc_type VARCHAR(255);
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Practical No : 02

Name : Mohd Danish Ansari

Roll No : 13

Date : 11/08/26

AIM : To perform complex data retrieval operations using SQL by executing queries involving joins, aggregation functions, grouping, ordering, and nested (sub) queries to analyze and extract meaningful information from the dataset.

Code :

-- SINGLE TABLE QUERIES

-- Display All Loan Records

```
mysql> SELECT * FROM loan;
+---------+-------------+-------------+
| loan_no | loan_amount | branch_name |
+---------+-------------+-------------+
|       1 |       25000 | Malad       |
|       2 |       10000 | Goregaon    |
|       3 |       50000 | Mira Road   |
|       4 |       60000 | Kothrud     |
|       5 |       80000 | Dharamsagar |
+---------+-------------+-------------+
5 rows in set (0.00 sec)
```

-- Display Branch Names from Loan Table

```
mysql> SELECT branch_name
    -> FROM loan;
+-------------+
| branch_name |
+-------------+
| Dharamsagar |
| Goregaon    |
| Kothrud     |
| Malad       |
| Mira Road   |
+-------------+
5 rows in set (0.00 sec)
```

-- Display Unique Branch Names

```
mysql> SELECT DISTINCT branch_name
    -> FROM loan;
+-------------+
| branch_name |
+-------------+
| Dharamsagar |
| Goregaon    |
| Kothrud     |
| Malad       |
| Mira Road   |
+-------------+
5 rows in set (0.00 sec)
```

-- Calculate Loan Amount Multiplied by 100

```
mysql> SELECT loan_no,
    ->        loan_amount * 100 AS hundred_times_loan
    -> FROM loan;
+---------+--------------------+
| loan_no | hundred_times_loan |
+---------+--------------------+
|       1 |            2500000 |
|       2 |            1000000 |
|       3 |            5000000 |
|       4 |            6000000 |
|       5 |            8000000 |
+---------+--------------------+
5 rows in set (0.00 sec)
```

-- Display Loans from Mumbai Branch

```
mysql> SELECT loan_no
    -> FROM loan
    -> WHERE branch_name = 'Mumbai';
Empty set (0.00 sec)
```

-- Display Loans Greater Than 50000

```
mysql> SELECT loan_no
    -> FROM loan
    -> WHERE loan_amount > 50000;
+---------+
| loan_no |
+---------+
|       4 |
|       5 |
+---------+
2 rows in set (0.00 sec)
```

-- Display Loans Between 50000 and 100000

```
mysql> SELECT loan_no
    -> FROM loan
    -> WHERE loan_amount BETWEEN 50000 AND 100000;
+---------+
| loan_no |
+---------+
|       3 |
|       4 |
|       5 |
+---------+
3 rows in set (0.00 sec)
```

-- Display Customers from Mumbai

```
mysql> SELECT *
    -> FROM customer
    -> WHERE cust_city = 'Mumbai';
+---------+-----------+-----------+----------+
| cust_id | cust_name | cust_city | cust_age |
+---------+-----------+-----------+----------+
|       1 | Rahul     | Mumbai    |       35 |
|       3 | Aman      | Mumbai    |       28 |
|       4 | Amit      | Mumbai    |       19 |
|       7 | Rajesh    | Mumbai    |       25 |
|      10 | Arushi    | Mumbai    |       22 |
+---------+-----------+-----------+----------+
5 rows in set (0.00 sec)
```

-- Display Customers from Mumbai or Pune

```
mysql> SELECT *
    -> FROM customer
    -> WHERE cust_city = 'Mumbai'
    -> OR cust_city = 'Pune';
+---------+-----------+-----------+----------+
| cust_id | cust_name | cust_city | cust_age |
+---------+-----------+-----------+----------+
|       1 | Rahul     | Mumbai    |       35 |
|       2 | Raj       | Pune      |       30 |
|       3 | Aman      | Mumbai    |       28 |
|       4 | Amit      | Mumbai    |       19 |
|       6 | Riya      | Pune      |       32 |
|       7 | Rajesh    | Mumbai    |       25 |
|       9 | Nikhat    | Pune      |       20 |
|      10 | Arushi    | Mumbai    |       22 |
+---------+-----------+-----------+----------+
8 rows in set (0.00 sec)
```

-- Display Customers from Multiple Cities

```
mysql> SELECT *
    -> FROM customer
    -> WHERE cust_city IN ('Mumbai', 'Delhi', 'Pune', 'Nagpur');
+---------+-----------+-----------+----------+
| cust_id | cust_name | cust_city | cust_age |
+---------+-----------+-----------+----------+
|       1 | Rahul     | Mumbai    |       35 |
|       2 | Raj       | Pune      |       30 |
|       3 | Aman      | Mumbai    |       28 |
|       4 | Amit      | Mumbai    |       19 |
|       5 | Shifa     | Nagpur    |       20 |
|       6 | Riya      | Pune      |       32 |
|       7 | Rajesh    | Mumbai    |       25 |
|       8 | Sara      | Nagpur    |       26 |
|       9 | Nikhat    | Pune      |       20 |
|      10 | Arushi    | Mumbai    |       22 |
+---------+-----------+-----------+----------+
10 rows in set (0.00 sec)
```

-- STRING FUNCTIONS

-- Display Customers Whose Name Starts With A

```
mysql> SELECT cust_name
    -> FROM customer
    -> WHERE cust_name LIKE 'A%';
+-----------+
| cust_name |
+-----------+
| Aman      |
| Amit      |
| Arushi    |
+-----------+
3 rows in set (0.00 sec)
```

-- Display Customers Whose Name Ends With a

```
mysql> SELECT cust_name
    -> FROM customer
    -> WHERE cust_name LIKE '%a';
+-----------+
| cust_name |
+-----------+
| Shifa     |
| Riya      |
| Sara      |
+-----------+
3 rows in set (0.00 sec)
```

-- Display Four Letter Names Starting With A

```
mysql> SELECT cust_name
    -> FROM customer
    -> WHERE cust_name LIKE 'A___';
+-----------+
| cust_name |
+-----------+
| Aman      |
| Amit      |
+-----------+
2 rows in set (0.00 sec)
```

-- Display Names Ending With a and Having Five Characters

```
mysql> SELECT cust_name
    -> FROM customer
    -> WHERE cust_name LIKE '____a';
+-----------+
| cust_name |
+-----------+
| Shifa     |
+-----------+
1 row in set (0.00 sec)
```

-- Display Branch Names Containing Letter e

```
mysql> SELECT *
    -> FROM loan
    -> WHERE branch_name LIKE '%e%';
+---------+-------------+-------------+
| loan_no | loan_amount | branch_name |
+---------+-------------+-------------+
|       2 |       10000 | Goregaon    |
+---------+-------------+-------------+
1 row in set (0.00 sec)
```

-- JOIN QUERIES

-- Display Borrower Details with Loan Information

```
mysql> SELECT borrower.cust_id,
    ->        loan.loan_no,
    ->        loan.loan_amount
    -> FROM borrower, loan
    -> WHERE borrower.loan_no = loan.loan_no;
+---------+---------+-------------+
| cust_id | loan_no | loan_amount |
+---------+---------+-------------+
|       6 |       1 |       25000 |
|       7 |       2 |       10000 |
|       8 |       3 |       50000 |
|       9 |       4 |       60000 |
|      10 |       5 |       80000 |
+---------+---------+-------------+
5 rows in set (0.00 sec)
```

-- Display Borrower and Loan Details Using NATURAL JOIN

```
mysql> SELECT cust_id,
    ->        loan_no,
    ->        loan_amount
    -> FROM borrower
    -> NATURAL JOIN loan;
+---------+---------+-------------+
| cust_id | loan_no | loan_amount |
+---------+---------+-------------+
|       6 |       1 |       25000 |
|       7 |       2 |       10000 |
|       8 |       3 |       50000 |
|       9 |       4 |       60000 |
|      10 |       5 |       80000 |
+---------+---------+-------------+
5 rows in set (0.00 sec)
```

-- Display Customer Names and Their Loan Amounts

```
mysql> SELECT customer.cust_name,
    ->        loan.loan_amount
    -> FROM customer, borrower, loan
    -> WHERE customer.cust_id = borrower.cust_id
    -> AND borrower.loan_no = loan.loan_no;
+-----------+-------------+
| cust_name | loan_amount |
+-----------+-------------+
| Riya      |       25000 |
| Rajesh    |       10000 |
| Sara      |       50000 |
| Nikhat    |       60000 |
| Arushi    |       80000 |
+-----------+-------------+
5 rows in set (0.00 sec)
```

-- Display Customer Names Starting With A and Their Account Balance

```
mysql> SELECT customer.cust_name,
    ->        account.balance
    -> FROM customer, depositor, account
    -> WHERE customer.cust_id = depositor.cust_id
    -> AND depositor.account_id = account.account_id
    -> AND customer.cust_name LIKE 'A%';
+-----------+---------+
| cust_name | balance |
+-----------+---------+
| Aman      |   10000 |
| Amit      |   20000 |
+-----------+---------+
2 rows in set (0.00 sec)
```

-- AGGREGATE QUERIES

-- Count the Total Number of Customers in the Bank

```
mysql> SELECT COUNT(*) AS total_customers
    -> FROM customer;
+-----------------+
| total_customers |
+-----------------+
|              10 |
+-----------------+
1 row in set (0.00 sec)
```

-- Count the Total Number of Accounts

```
mysql> SELECT COUNT(*) AS total_accounts
    -> FROM account;
+----------------+
| total_accounts |
+----------------+
|              8 |
+----------------+
1 row in set (0.00 sec)
```

-- Count the Total Number of Customers in Each City

```
mysql> SELECT cust_city,
    ->        COUNT(*) AS total_customers
    -> FROM customer
    -> GROUP BY cust_city;
+-----------+-----------------+
| cust_city | total_customers |
+-----------+-----------------+
| Mumbai    |               5 |
| Pune      |               3 |
| Nagpur    |               2 |
+-----------+-----------------+
3 rows in set (0.00 sec)
```

-- Count the Number of Accounts in Each Branch

```
mysql> SELECT branch_name,
    ->        COUNT(*) AS total_accounts
    -> FROM account
    -> GROUP BY branch_name;
+-------------+----------------+
| branch_name | total_accounts |
+-------------+----------------+
| Malad       |              2 |
| Goregaon    |              2 |
| Mira Road   |              1 |
| Kothrud     |              2 |
| Dharamsagar |              1 |
+-------------+----------------+
5 rows in set (0.00 sec)
```

-- Find the Total Balance in Each Branch

```
mysql> SELECT branch_name,
    ->        SUM(balance) AS total_balance
    -> FROM account
    -> GROUP BY branch_name;
+-------------+---------------+
| branch_name | total_balance |
+-------------+---------------+
| Malad       |         70000 |
| Goregaon    |        160000 |
| Mira Road   |         10000 |
| Kothrud     |         40000 |
| Dharamsagar |         15000 |
+-------------+---------------+
5 rows in set (0.00 sec)
```

-- Find the Total Loan Amount in Each Branch

```
mysql> SELECT branch_name,
    ->        SUM(loan_amount) AS total_loan
    -> FROM loan
    -> GROUP BY branch_name;
+-------------+------------+
| branch_name | total_loan |
+-------------+------------+
| Dharamsagar |      80000 |
| Goregaon    |      10000 |
| Kothrud     |      60000 |
| Malad       |      25000 |
| Mira Road   |      50000 |
+-------------+------------+
5 rows in set (0.00 sec)
```

-- Find the Total Assets of All Branches

```
mysql> SELECT SUM(assets) AS total_assets
    -> FROM branch;
+--------------+
| total_assets |
+--------------+
|     23500000 |
+--------------+
1 row in set (0.00 sec)
```

-- Find the Average Account Balance

```
mysql> SELECT AVG(balance) AS average_balance
    -> FROM account;
+-----------------+
| average_balance |
+-----------------+
|           36875 |
+-----------------+
1 row in set (0.00 sec)
```

-- Find the Average Age of Customers in Each City

```
mysql> SELECT cust_city,
    ->        AVG(cust_age) AS average_age
    -> FROM customer
    -> GROUP BY cust_city;
+-----------+-------------+
| cust_city | average_age |
+-----------+-------------+
| Mumbai    |     25.8000 |
| Pune      |     27.3333 |
| Nagpur    |     23.0000 |
+-----------+-------------+
3 rows in set (0.00 sec)
```

-- Find the Average Loan Amount for Each Branch

```
mysql> SELECT branch_name,
    ->        AVG(loan_amount) AS average_loan
    -> FROM loan
    -> GROUP BY branch_name;
+-------------+--------------+
| branch_name | average_loan |
+-------------+--------------+
| Dharamsagar |        80000 |
| Goregaon    |        10000 |
| Kothrud     |        60000 |
| Malad       |        25000 |
| Mira Road   |        50000 |
+-------------+--------------+
5 rows in set (0.00 sec)
```

-- Find the Highest Account Balance

```
mysql> SELECT MAX(balance) AS highest_balance
    -> FROM account;
+-----------------+
| highest_balance |
+-----------------+
|          100000 |
+-----------------+
1 row in set (0.00 sec)
```

-- Find the Minimum Loan Amount

```
mysql> SELECT MIN(loan_amount) AS minimum_loan
    -> FROM loan;
+--------------+
| minimum_loan |
+--------------+
|        10000 |
+--------------+
1 row in set (0.00 sec)
```

-- Find the Maximum Assets Among All Branches

```
mysql> SELECT MAX(assets) AS maximum_assets
    -> FROM branch;
+----------------+
| maximum_assets |
+----------------+
|        8000000 |
+----------------+
1 row in set (0.00 sec)
```

-- Show Branches Having More Than Two Accounts

```
mysql> SELECT branch_name,
    ->        COUNT(*) AS total_accounts
    -> FROM account
    -> GROUP BY branch_name
    -> HAVING COUNT(*) > 2;
Empty set (0.00 sec)
```

-- Show Branches Whose Total Balance is Greater Than 100000

```
mysql> SELECT branch_name,
    ->        SUM(balance) AS total_balance
    -> FROM account
    -> GROUP BY branch_name
    -> HAVING SUM(balance) > 100000;
+-------------+---------------+
| branch_name | total_balance |
+-------------+---------------+
| Goregaon    |        160000 |
+-------------+---------------+
1 row in set (0.00 sec)
```

-- Show Customers Having More Than One Account

```
mysql> SELECT cust_id,
    ->        COUNT(account_id) AS total_accounts
    -> FROM depositor
    -> GROUP BY cust_id
    -> HAVING COUNT(account_id) > 1;
Empty set (0.00 sec)
```

-- Show Branches Whose Average Loan Amount is Greater Than 50000

```
mysql> SELECT branch_name,
    ->        AVG(loan_amount) AS average_loan
    -> FROM loan
    -> GROUP BY branch_name
    -> HAVING AVG(loan_amount) > 50000;
+-------------+--------------+
| branch_name | average_loan |
+-------------+--------------+
| Dharamsagar |        80000 |
| Kothrud     |        60000 |
+-------------+--------------+
2 rows in set (0.00 sec)
```

-- Find Cities That Have More Than Two Customers Whose Age is Greater Than 25

```
mysql> SELECT cust_city,
    ->        COUNT(cust_id) AS total_customers
    -> FROM customer
    -> WHERE cust_age > 25
    -> GROUP BY cust_city
    -> HAVING COUNT(cust_id) > 2;
Empty set (0.00 sec)
```

-- SUBQUERIES

-- Get all account details for customers living in Mumbai city (Using Joins)

```
mysql> SELECT a.account_id,
    ->        a.balance,
    ->        a.branch_name
    -> FROM account a, customer c, depositor d
    -> WHERE a.account_id = d.account_id
    ->   AND d.cust_id = c.cust_id
    ->   AND c.cust_city = 'Mumbai';
+------------+---------+-------------+
| account_id | balance | branch_name |
+------------+---------+-------------+
| a1         |   50000 | Malad       |
| a3         |   10000 | Mira Road   |
| a4         |   20000 | Malad       |
| a7         |    5000 | Kothrud     |
+------------+---------+-------------+
4 rows in set (0.00 sec)
```

-- Get all account details for customers living in Mumbai city (Using Subquery)

```
mysql> SELECT *
    -> FROM account
    -> WHERE account_id IN (
    ->     SELECT d.account_id
    ->     FROM depositor d, customer c
    ->     WHERE d.cust_id = c.cust_id
    ->       AND c.cust_city = 'Mumbai'
    -> );
+------------+---------+-------------+
| account_id | balance | branch_name |
+------------+---------+-------------+
| a1         |   50000 | Malad       |
| a3         |   10000 | Mira Road   |
| a4         |   20000 | Malad       |
| a7         |    5000 | Kothrud     |
+------------+---------+-------------+
4 rows in set (0.00 sec)
```

-- Find the names of customers who have not taken any loan

```
mysql> SELECT cust_name
    -> FROM customer
    -> WHERE cust_id NOT IN (
    ->     SELECT cust_id
    ->     FROM borrower
    -> );
+-----------+
| cust_name |
+-----------+
| Rahul     |
| Raj       |
| Aman      |
| Amit      |
| Shifa     |
+-----------+
5 rows in set (0.00 sec)
```

-- List names of customers who have taken a loan using EXISTS

```
mysql> SELECT cust_name
    -> FROM customer c
    -> WHERE EXISTS (
    ->     SELECT *
    ->     FROM borrower b
    ->     WHERE c.cust_id = b.cust_id
    -> );
+-----------+
| cust_name |
+-----------+
| Riya      |
| Rajesh    |
| Sara      |
| Nikhat    |
| Arushi    |
+-----------+
5 rows in set (0.00 sec)
```

-- List customers who do not have a loan using NOT EXISTS

```
mysql> SELECT cust_name
    -> FROM customer c
    -> WHERE NOT EXISTS (
    ->     SELECT *
    ->     FROM borrower b
    ->     WHERE c.cust_id = b.cust_id
    -> );
+-----------+
| cust_name |
+-----------+
| Rahul     |
| Raj       |
| Aman      |
| Amit      |
| Shifa     |
+-----------+
5 rows in set (0.00 sec)
```

-- Find customers who are older than ALL customers in Pune

```
mysql> SELECT *
    -> FROM customer
    -> WHERE cust_age > ALL (
    ->     SELECT cust_age
    ->     FROM customer
    ->     WHERE cust_city = 'Pune'
    -> );
+---------+-----------+-----------+----------+
| cust_id | cust_name | cust_city | cust_age |
+---------+-----------+-----------+----------+
|       1 | Rahul     | Mumbai    |       35 |
+---------+-----------+-----------+----------+
1 row in set (0.00 sec)
```

-- Find customers who are older than ANY customer in Pune

```
mysql> SELECT *
    -> FROM customer
    -> WHERE cust_age > ANY (
    ->     SELECT cust_age
    ->     FROM customer
    ->     WHERE cust_city = 'Pune'
    -> );
+---------+-----------+-----------+----------+
| cust_id | cust_name | cust_city | cust_age |
+---------+-----------+-----------+----------+
|       1 | Rahul     | Mumbai    |       35 |
|       2 | Raj       | Pune      |       30 |
|       3 | Aman      | Mumbai    |       28 |
|       6 | Riya      | Pune      |       32 |
|       7 | Rajesh    | Mumbai    |       25 |
|       8 | Sara      | Nagpur    |       26 |
|      10 | Arushi    | Mumbai    |       22 |
+---------+-----------+-----------+----------+
7 rows in set (0.00 sec)
```

-- VIEWS QUERIES

-- Q1. Create a view that stores all data of Mumbai customers

```
mysql> CREATE VIEW mumbai_customers AS
    -> SELECT *
    -> FROM customer
    -> WHERE cust_city = 'Mumbai';
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> SELECT * FROM mumbai_customers;
+---------+-----------+-----------+----------+-------+---------+-------------+
| cust_id | cust_name | cust_city | cust_age | email | pincode | street_name |
+---------+-----------+-----------+----------+-------+---------+-------------+
|       1 | Ramesh    | Mumbai    |       36 | NULL  |    NULL | NULL        |
|       3 | Aman      | Mumbai    |       28 | NULL  |    NULL | NULL        |
|       4 | Amit      | Mumbai    |       19 | NULL  |    NULL | NULL        |
|       7 | Rajesh    | Mumbai    |       25 | NULL  |    NULL | NULL        |
|      10 | Arushi    | Mumbai    |       22 | NULL  |    NULL | NULL        |
+---------+-----------+-----------+----------+-------+---------+-------------+
5 rows in set (0.00 sec)
```

-- Q2. Create a view that stores city of branches and number of branches in each city

```
mysql> CREATE VIEW branch_city_count AS
    -> SELECT branch_city, COUNT(*) AS number_of_branches
    -> FROM branch
    -> GROUP BY branch_city;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> SELECT * FROM branch_city_count;
+-------------+--------------------+
| branch_city | number_of_branches |
+-------------+--------------------+
| Nagpur      |                  1 |
| Mumbai      |                  3 |
| Pune        |                  1 |
+-------------+--------------------+
3 rows in set (0.00 sec)
```

-- Q3. Create a view that stores customer name along with their account ID

```
mysql> CREATE VIEW customer_account_details AS
    -> SELECT cust_name, account_id
    -> FROM customer, depositor
    -> WHERE customer.cust_id = depositor.cust_id;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> SELECT * FROM customer_account_details;
+-----------+------------+
| cust_name | account_id |
+-----------+------------+
| Ramesh    | a1         |
| Raj       | a2         |
| Aman      | a3         |
| Amit      | a4         |
| Shifa     | a5         |
| Riya      | a6         |
| Rajesh    | a7         |
| Sara      | a8         |
+-----------+------------+
8 rows in set (0.00 sec)
```

-- Q4. Create a view that stores customer name along with the loan amount

```
mysql> CREATE VIEW customer_loan_details AS
    -> SELECT cust_name, loan_no
    -> FROM customer, borrower
    -> WHERE customer.cust_id = borrower.loan_no;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> SELECT * FROM customer_loan_details;
+-----------+---------+
| cust_name | loan_no |
+-----------+---------+
| Ramesh    |       1 |
| Raj       |       2 |
| Aman      |       3 |
| Amit      |       4 |
| Shifa     |       5 |
+-----------+---------+
5 rows in set (0.00 sec)
```

Practical No: 03
Name: Mohd Danish Ansari
Roll No: 13
Date: 28/08/26

Aim: To implement advanced database programming concepts by creating and managing database objects such as views, sequences, triggers, and stored procedures to automate data handling and enforce business logic.

Theory

1. **PL/SQL:** PL/SQL (Procedural Language/Structured Query Language) is an extension of SQL developed by Oracle. It combines SQL statements with programming features like variables, conditions, loops, and functions. It is used to store, manage, and manipulate data in an Oracle database.
2. **PL/SQL Block:** A PL/SQL block is the basic structure of a PL/SQL program. It consists of three sections: DECLARE (for declaring variables), BEGIN (for writing executable statements), and EXCEPTION (for handling errors). The DECLARE and EXCEPTION sections are optional, but BEGIN and END are compulsory.
3. **Variables:** Variables are used to store temporary values during program execution. They are declared in the DECLARE section and can store different types of data, such as numbers, characters, and dates. Examples include NUMBER, VARCHAR2, and DATE.
4. **SELECT INTO Statement:** SELECT INTO is used to retrieve data from a database table and store the result in a PL/SQL variable. It is generally used when the query returns exactly one row. It helps access table records and use them in PL/SQL programs.
5. **DBMS_OUTPUT.PUT_LINE:** DBMS_OUTPUT.PUT_LINE is used to display messages and results on the screen during PL/SQL program execution. It is commonly used to print variable values and output messages.
6. **DML Statements:** DML (Data Manipulation Language) statements are used to insert, update, delete, and retrieve data from database tables. Examples include INSERT, UPDATE, DELETE, and SELECT.
7. **Exception Handling:** Exception handling is used to manage errors that occur during PL/SQL program execution. It prevents unexpected termination and displays appropriate error messages. Examples include NO_DATA_FOUND and TOO_MANY_ROWS.

8. **Advantages of PL/SQL:** PL/SQL improves code reusability, supports error handling, reduces network traffic, provides better security, and allows multiple SQL statements to execute together in a single block.

queries.sql

```
1   -- EXAMPLE 1: Printing a message
2   BEGIN
3       DBMS_OUTPUT.PUT_LINE('Hello World!!');
4   END;
5   /
```

Output:

```
Hello World!!
```

```
1   -- EXAMPLE 2: Addition of two numbers
2   DECLARE
3       num1 NUMBER := 10;
4       num2 NUMBER := 20;
5       total NUMBER;
6   BEGIN
7       total := num1 + num2;
8       DBMS_OUTPUT.PUT_LINE('Sum = ' || total);
9   END;
10  /
```

Output:

```
Sum = 30
```

```sql
-- EXAMPLE 3: To take input from user
DECLARE
    v_num NUMBER;
BEGIN
    v_num := &Enter_Number;
    DBMS_OUTPUT.PUT_LINE('You entered: ' || v_num);
END;
/
```

```
You entered: 69


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.005
```

```
-- EXAMPLE 4: IF statement
DECLARE
    marks NUMBER := 85;
BEGIN
    IF marks >= 50 THEN
        DBMS_OUTPUT.PUT_LINE('Pass');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Fail');
    END IF;
END;
/
```

```
Output:
Pass
```

```
-- EXAMPLE 5: FOR loop
BEGIN
  FOR i IN 1..5 LOOP
      DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
/
```

```
Output:
1
2
3
4
5
```

Procedure

```
1   CREATE OR REPLACE PROCEDURE greet IS
2   BEGIN
3       DBMS_OUTPUT.PUT_LINE('Welcome!');
4   END;
5   /
6
7   BEGIN
8       greet;
9   END;
10  /
11
```

```
Output:
Welcome!
```

Function

```
1   CREATE OR REPLACE FUNCTION square(n NUMBER)
2   RETURN NUMBER
3   IS
4   BEGIN
5       RETURN n * n;
6   END;
7   /
8
9   SELECT square(5) FROM dual;
```

```
Output:

  SQUARE(5)
----------
        25

  SQUARE(5)
----------
        25
```

Cursor

```sql
CREATE TABLE Employee (
    Empno NUMBER,
    Emp_Name VARCHAR2(50),
    Dept_No NUMBER,
    Salary NUMBER
);

INSERT INTO Employee VALUES (101, 'Raj', 10, 10000);
INSERT INTO Employee VALUES (102, 'Eva', 20, 15000);
INSERT INTO Employee VALUES (103, 'John', 30, 20000);
INSERT INTO Employee VALUES (104, 'Rahul', 10, 25000);
INSERT INTO Employee VALUES (105, 'Bob', 20, 30000);

desc Employee;

SELECT * FROM Employee;
```

```
Output:
 Name                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 EMPNO                                    NUMBER
 EMP_NAME                                 VARCHAR2(50)
 DEPT_NO                                  NUMBER
 SALARY                                   NUMBER


     EMPNO EMP_NAME                                           DEPT_NO
---------- -------------------------------------------------- ----------
    SALARY
----------
       101 Raj                                                      10
     10000

       102 Eva                                                      20
     15000

       103 John                                                     30
     20000
```

Implicit Cursor

```
18    DECLARE
19        emp_name VARCHAR2(50);
20    BEGIN
21        SELECT Emp_Name
22        INTO emp_name
23        FROM Employee
24        WHERE Empno = 101;
25        DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_name);
26    END;
27    /
```

```
Employee Name: Raj
```

```
BEGIN
  UPDATE Employee
  SET Salary = Salary + 1000
  WHERE Dept_No = 10;
  DBMS_OUTPUT.PUT_LINE('Rows Updated: ' || SQL%ROWCOUNT);
END;
/
```

```
Rows Updated: 2
```

Explicit Cursor

```
DECLARE
    CURSOR c_emp IS
        SELECT Empno, Emp_Name
        FROM Employee;
    v_empno  Employee.Empno%TYPE;
    v_ename  Employee.Emp_Name%TYPE;
BEGIN
    OPEN c_emp;
    LOOP
        FETCH c_emp INTO v_empno, v_ename;
        EXIT WHEN c_emp%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(
            'Empno: ' || v_empno || '  Emp Name: ' || v_ename
        );
    END LOOP;
    CLOSE c_emp;
END;
/
```

```
Empno: 101  Emp Name: Raj
Empno: 102  Emp Name: Eva
Empno: 103  Emp Name: John
Empno: 104  Emp Name: Rahul
Empno: 105  Emp Name: Bob
```

Explicit Cursor

```
72    DECLARE
73        CURSOR c_emp(p_dept NUMBER) IS
74            SELECT Empno, Emp_Name
75            FROM Employee
76            WHERE Dept_No = p_dept;
77    BEGIN
78        FOR rec IN c_emp(10) LOOP
79            DBMS_OUTPUT.PUT_LINE(
80                'Empno: ' || rec.Empno ||
81                '   Emp Name: ' || rec.Emp_Name
82            );
83        END LOOP;
84    END;
85    /
```

```
Empno: 101   Emp Name: Raj
Empno: 104   Emp Name: Rahul
```

Triggers

BEFORE INSERT

```
CREATE OR REPLACE TRIGGER before_employee_insert
BEFORE INSERT
ON Employee
FOR EACH ROW
BEGIN
    IF :NEW.Salary < 10000 THEN
        :NEW.Salary := 10000;
    END IF;
END;
/

INSERT INTO Employee VALUES (106, 'Sara', 30, 5000);

SELECT * FROM Employee;
```

```
     EMPNO EMP_NAME                                         DEPT_NO
---------- -------------------------------------------------- ----------
    SALARY
----------
       101 Raj                                                    10
     11000

       102 Eva                                                    20
     15000

       103 John                                                   30
     20000


     EMPNO EMP_NAME                                         DEPT_NO
---------- -------------------------------------------------- ----------
    SALARY
----------
       104 Rahul                                                  10
     26000

       105 Bob                                                    20
```

```
       105 Bob                                                    20
     30000

       106 Sara                                                   30
     10000


     EMPNO EMP_NAME                                         DEPT_NO
---------- -------------------------------------------------- ----------
    SALARY
----------
       101 Raj                                                    10
     11000

       102 Eva                                                    20
     15000

       103 John                                                   30
     20000
```

```
     EMPNO EMP_NAME                                         DEPT_NO
---------- -------------------------------------------------- ----------
    SALARY
----------
       104 Rahul                                                  10
     26000

       105 Bob                                                    20
     30000

       106 Sara                                                   30
     10000
```

Practical No: 04

Name: Mohd Danish Ansari

Roll No: 13

Date: 12/09/26

Aim: To apply data mining techniques such as classification and clustering on a real-world dataset to discover hidden patterns, relationships, and insights for knowledge discovery.

Theory

1. Data Mining: Data mining is the process of extracting useful information, hidden patterns, and meaningful knowledge from large datasets. It helps in analyzing data and making better decisions.

2. Classification: Classification is a supervised data mining technique used to classify data into predefined categories or classes. It uses training data to predict the class of new data. Examples include Decision Tree, Naive Bayes, and Random Forest.

3. Clustering: Clustering is an unsupervised data mining technique used to group similar data points based on their characteristics. It helps identify natural groups and hidden patterns in datasets. Examples include K-Means and Hierarchical Clustering.

4. Pattern Discovery: Pattern discovery helps identify relationships, similarities, trends, and useful information within a dataset. It allows us to understand the behaviour of data and discover meaningful insights.

5. Knowledge Discovery: Knowledge discovery is the process of converting raw data into useful knowledge. By applying classification and clustering techniques, we can identify important patterns, understand relationships, and support better decision-making.

In [1]:
```
from sklearn.datasets import load_iris
```

In [2]:
```
data1 = load_iris()
data1
```

Out[2]:
```
{'data': array([[5.1, 3.5, 1.4, 0.2],
                [4.9, 3. , 1.4, 0.2],
                [4.7, 3.2, 1.3, 0.2],
                [4.6, 3.1, 1.5, 0.2],
                [5. , 3.6, 1.4, 0.2],
                [5.4, 3.9, 1.7, 0.4],
                [4.6, 3.4, 1.4, 0.3],
                [5. , 3.4, 1.5, 0.2],
                [4.4, 2.9, 1.4, 0.2],
                [4.9, 3.1, 1.5, 0.1],
                [5.4, 3.7, 1.5, 0.2],
                [4.8, 3.4, 1.6, 0.2],
                [4.8, 3. , 1.4, 0.1],
                [4.3, 3. , 1.1, 0.1],
                [5.8, 4. , 1.2, 0.2],
                [5.7, 4.4, 1.5, 0.4],
                [5.4, 3.9, 1.3, 0.4],
                [5.1, 3.5, 1.4, 0.3],
                [5.7, 3.8, 1.7, 0.3],
                [5.1, 3.8, 1.5, 0.3],
                [5.4, 3.4, 1.7, 0.2],
                [5.1, 3.7, 1.5, 0.4],
                [4.6, 3.6, 1. , 0.2],
                [5.1, 3.3, 1.7, 0.5],
                [4.8, 3.4, 1.9, 0.2],
                [5. , 3. , 1.6, 0.2],
                [5. , 3.4, 1.6, 0.4],
                [5.2, 3.5, 1.5, 0.2],
                [5.2, 3.4, 1.4, 0.2],
                [4.7, 3.2, 1.6, 0.2],
                [4.8, 3.1, 1.6, 0.2],
                [5.4, 3.4, 1.5, 0.4],
                [5.2, 4.1, 1.5, 0.1],
                [5.5, 4.2, 1.4, 0.2],
                [4.9, 3.1, 1.5, 0.2],
                [5. , 3.2, 1.2, 0.2],
                [5.5, 3.5, 1.3, 0.2],
                [4.9, 3.6, 1.4, 0.1],
                [4.4, 3. , 1.3, 0.2],
                [5.1, 3.4, 1.5, 0.2],
                [5. , 3.5, 1.3, 0.3],
                [4.5, 2.3, 1.3, 0.3],
                [4.4, 3.2, 1.3, 0.2],
                [5. , 3.5, 1.6, 0.6],
                [5.1, 3.8, 1.9, 0.4],
                [4.8, 3. , 1.4, 0.3],
                [5.1, 3.8, 1.6, 0.2],
                [4.6, 3.2, 1.4, 0.2],
                [5.3, 3.7, 1.5, 0.2],
                [5. , 3.3, 1.4, 0.2],
                [7. , 3.2, 4.7, 1.4],
                [6.4, 3.2, 4.5, 1.5],
                [6.9, 3.1, 4.9, 1.5],
                [5.5, 2.3, 4. , 1.3],
                [6.5, 2.8, 4.6, 1.5],
                [5.7, 2.8, 4.5, 1.3],
                [6.3, 3.3, 4.7, 1.6],
                [4.9, 2.4, 3.3, 1. ],
                [6.6, 2.9, 4.6, 1.3],
                [5.2, 2.7, 3.9, 1.4],
```

```
[5. , 2. , 3.5, 1. ],
[5.9, 3. , 4.2, 1.5],
[6. , 2.2, 4. , 1. ],
[6.1, 2.9, 4.7, 1.4],
[5.6, 2.9, 3.6, 1.3],
[6.7, 3.1, 4.4, 1.4],
[5.6, 3. , 4.5, 1.5],
[5.8, 2.7, 4.1, 1. ],
[6.2, 2.2, 4.5, 1.5],
[5.6, 2.5, 3.9, 1.1],
[5.9, 3.2, 4.8, 1.8],
[6.1, 2.8, 4. , 1.3],
[6.3, 2.5, 4.9, 1.5],
[6.1, 2.8, 4.7, 1.2],
[6.4, 2.9, 4.3, 1.3],
[6.6, 3. , 4.4, 1.4],
[6.8, 2.8, 4.8, 1.4],
[6.7, 3. , 5. , 1.7],
[6. , 2.9, 4.5, 1.5],
[5.7, 2.6, 3.5, 1. ],
[5.5, 2.4, 3.8, 1.1],
[5.5, 2.4, 3.7, 1. ],
[5.8, 2.7, 3.9, 1.2],
[6. , 2.7, 5.1, 1.6],
[5.4, 3. , 4.5, 1.5],
[6. , 3.4, 4.5, 1.6],
[6.7, 3.1, 4.7, 1.5],
[6.3, 2.3, 4.4, 1.3],
[5.6, 3. , 4.1, 1.3],
[5.5, 2.5, 4. , 1.3],
[5.5, 2.6, 4.4, 1.2],
[6.1, 3. , 4.6, 1.4],
[5.8, 2.6, 4. , 1.2],
[5. , 2.3, 3.3, 1. ],
[5.6, 2.7, 4.2, 1.3],
[5.7, 3. , 4.2, 1.2],
[5.7, 2.9, 4.2, 1.3],
[6.2, 2.9, 4.3, 1.3],
[5.1, 2.5, 3. , 1.1],
[5.7, 2.8, 4.1, 1.3],
[6.3, 3.3, 6. , 2.5],
[5.8, 2.7, 5.1, 1.9],
[7.1, 3. , 5.9, 2.1],
[6.3, 2.9, 5.6, 1.8],
[6.5, 3. , 5.8, 2.2],
[7.6, 3. , 6.6, 2.1],
[4.9, 2.5, 4.5, 1.7],
[7.3, 2.9, 6.3, 1.8],
[6.7, 2.5, 5.8, 1.8],
[7.2, 3.6, 6.1, 2.5],
[6.5, 3.2, 5.1, 2. ],
[6.4, 2.7, 5.3, 1.9],
[6.8, 3. , 5.5, 2.1],
[5.7, 2.5, 5. , 2. ],
[5.8, 2.8, 5.1, 2.4],
[6.4, 3.2, 5.3, 2.3],
[6.5, 3. , 5.5, 1.8],
[7.7, 3.8, 6.7, 2.2],
[7.7, 2.6, 6.9, 2.3],
[6. , 2.2, 5. , 1.5],
```

```
        [6.9, 3.2, 5.7, 2.3],
        [5.6, 2.8, 4.9, 2. ],
        [7.7, 2.8, 6.7, 2. ],
        [6.3, 2.7, 4.9, 1.8],
        [6.7, 3.3, 5.7, 2.1],
        [7.2, 3.2, 6. , 1.8],
        [6.2, 2.8, 4.8, 1.8],
        [6.1, 3. , 4.9, 1.8],
        [6.4, 2.8, 5.6, 2.1],
        [7.2, 3. , 5.8, 1.6],
        [7.4, 2.8, 6.1, 1.9],
        [7.9, 3.8, 6.4, 2. ],
        [6.4, 2.8, 5.6, 2.2],
        [6.3, 2.8, 5.1, 1.5],
        [6.1, 2.6, 5.6, 1.4],
        [7.7, 3. , 6.1, 2.3],
        [6.3, 3.4, 5.6, 2.4],
        [6.4, 3.1, 5.5, 1.8],
        [6. , 3. , 4.8, 1.8],
        [6.9, 3.1, 5.4, 2.1],
        [6.7, 3.1, 5.6, 2.4],
        [6.9, 3.1, 5.1, 2.3],
        [5.8, 2.7, 5.1, 1.9],
        [6.8, 3.2, 5.9, 2.3],
        [6.7, 3.3, 5.7, 2.5],
        [6.7, 3. , 5.2, 2.3],
        [6.3, 2.5, 5. , 1.9],
        [6.5, 3. , 5.2, 2. ],
        [6.2, 3.4, 5.4, 2.3],
        [5.9, 3. , 5.1, 1.8]]),
 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]),
 'frame': None,
 'target_names': array(['setosa', 'versicolor', 'virginica'], dtype='<U10'),
 'DESCR': '.. _iris_dataset:\n\nIris plants dataset\n--------------------\n\n**
Data Set Characteristics:**\n\n:Number of Instances: 150 (50 in each of three c
lasses)\n:Number of Attributes: 4 numeric, predictive attributes and the class
\n:Attribute Information:\n    - sepal length in cm\n    - sepal width in cm\n
- petal length in cm\n    - petal width in cm\n    - class:\n            - Iris
-Setosa\n            - Iris-Versicolour\n            - Iris-Virginica\n\n:Summa
ry Statistics:\n\n============== ==== ==== ======= ===== ====================\n
Min  Max   Mean    SD   Class Correlation\n============== ==== ==== ======= ===
== ====================\nsepal length:   4.3  7.9   5.84   0.83    0.7826\nsepa
l width:    2.0  4.4   3.05   0.43   -0.4194\npetal length:   1.0  6.9   3.76
1.76    0.9490  (high!)\npetal width:    0.1  2.5   1.20   0.76    0.9565  (hig
h!)\n============== ==== ==== ======= ===== ====================\n\n:Missing At
tribute Values: None\n:Class Distribution: 33.3% for each of 3 classes.\n:Creat
or: R.A. Fisher\n:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)\n:Dat
e: July, 1988\n\nThe famous Iris database, first used by Sir R.A. Fisher. The d
ataset is taken\nfrom Fisher\'s paper. Note that it\'s the same as in R, but no
t as in the UCI\nMachine Learning Repository, which has two wrong data point
s.\n\nThis is perhaps the best known database to be found in the\npattern recog
nition literature.  Fisher\'s paper is a classic in the field and\nis reference
d frequently to this day.  (See Duda & Hart, for example.)  The\ndata set conta
```

```
ins 3 classes of 50 instances each, where each class refers to a\ntype of iris
plant.  One class is linearly separable from the other 2; the\nlatter are NOT l
inearly separable from each other.\n\n|details-start|\n**References**\n|details
-split|\n\n- Fisher, R.A. "The use of multiple measurements in taxonomic proble
ms"\n  Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to\n
Mathematical Statistics" (John Wiley, NY, 1950).\n- Duda, R.O., & Hart, P.E. (1
973) Pattern Classification and Scene Analysis.\n  (Q327.D83) John Wiley & Son
s.  ISBN 0-471-22361-1.  See page 218.\n- Dasarathy, B.V. (1980) "Nosing Around
the Neighborhood: A New System\n  Structure and Classification Rule for Recogni
tion in Partially Exposed\n  Environments".  IEEE Transactions on Pattern Analy
sis and Machine\n  Intelligence, Vol. PAMI-2, No. 1, 67-71.\n- Gates, G.W. (197
2) "The Reduced Nearest Neighbor Rule".  IEEE Transactions\n  on Information Th
eory, May 1972, 431-433.\n- See also: 1988 MLC Proceedings, 54-64.  Cheeseman e
t al"s AUTOCLASS II\n  conceptual clustering system finds 3 classes in the dat
a.\n- Many, many more ...\n\n|details-end|\n',
 'feature_names': ['sepal length (cm)',
  'sepal width (cm)',
  'petal length (cm)',
  'petal width (cm)'],
 'filename': 'iris.csv',
 'data_module': 'sklearn.datasets.data'}
```

In [3]:
```
x, y = data1.data, data1.target
```

In [4]:
```
x
```

Out[4]:
```
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2],
       [5.4, 3.9, 1.7, 0.4],
       [4.6, 3.4, 1.4, 0.3],
       [5. , 3.4, 1.5, 0.2],
       [4.4, 2.9, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.1],
       [5.4, 3.7, 1.5, 0.2],
       [4.8, 3.4, 1.6, 0.2],
       [4.8, 3. , 1.4, 0.1],
       [4.3, 3. , 1.1, 0.1],
       [5.8, 4. , 1.2, 0.2],
       [5.7, 4.4, 1.5, 0.4],
       [5.4, 3.9, 1.3, 0.4],
       [5.1, 3.5, 1.4, 0.3],
       [5.7, 3.8, 1.7, 0.3],
       [5.1, 3.8, 1.5, 0.3],
       [5.4, 3.4, 1.7, 0.2],
       [5.1, 3.7, 1.5, 0.4],
       [4.6, 3.6, 1. , 0.2],
       [5.1, 3.3, 1.7, 0.5],
       [4.8, 3.4, 1.9, 0.2],
       [5. , 3. , 1.6, 0.2],
       [5. , 3.4, 1.6, 0.4],
       [5.2, 3.5, 1.5, 0.2],
       [5.2, 3.4, 1.4, 0.2],
       [4.7, 3.2, 1.6, 0.2],
       [4.8, 3.1, 1.6, 0.2],
       [5.4, 3.4, 1.5, 0.4],
       [5.2, 4.1, 1.5, 0.1],
       [5.5, 4.2, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.2],
       [5. , 3.2, 1.2, 0.2],
       [5.5, 3.5, 1.3, 0.2],
       [4.9, 3.6, 1.4, 0.1],
       [4.4, 3. , 1.3, 0.2],
       [5.1, 3.4, 1.5, 0.2],
       [5. , 3.5, 1.3, 0.3],
       [4.5, 2.3, 1.3, 0.3],
       [4.4, 3.2, 1.3, 0.2],
       [5. , 3.5, 1.6, 0.6],
       [5.1, 3.8, 1.9, 0.4],
       [4.8, 3. , 1.4, 0.3],
       [5.1, 3.8, 1.6, 0.2],
       [4.6, 3.2, 1.4, 0.2],
       [5.3, 3.7, 1.5, 0.2],
       [5. , 3.3, 1.4, 0.2],
       [7. , 3.2, 4.7, 1.4],
       [6.4, 3.2, 4.5, 1.5],
       [6.9, 3.1, 4.9, 1.5],
       [5.5, 2.3, 4. , 1.3],
       [6.5, 2.8, 4.6, 1.5],
       [5.7, 2.8, 4.5, 1.3],
       [6.3, 3.3, 4.7, 1.6],
       [4.9, 2.4, 3.3, 1. ],
       [6.6, 2.9, 4.6, 1.3],
       [5.2, 2.7, 3.9, 1.4],
```

```
[5. , 2. , 3.5, 1. ],
[5.9, 3. , 4.2, 1.5],
[6. , 2.2, 4. , 1. ],
[6.1, 2.9, 4.7, 1.4],
[5.6, 2.9, 3.6, 1.3],
[6.7, 3.1, 4.4, 1.4],
[5.6, 3. , 4.5, 1.5],
[5.8, 2.7, 4.1, 1. ],
[6.2, 2.2, 4.5, 1.5],
[5.6, 2.5, 3.9, 1.1],
[5.9, 3.2, 4.8, 1.8],
[6.1, 2.8, 4. , 1.3],
[6.3, 2.5, 4.9, 1.5],
[6.1, 2.8, 4.7, 1.2],
[6.4, 2.9, 4.3, 1.3],
[6.6, 3. , 4.4, 1.4],
[6.8, 2.8, 4.8, 1.4],
[6.7, 3. , 5. , 1.7],
[6. , 2.9, 4.5, 1.5],
[5.7, 2.6, 3.5, 1. ],
[5.5, 2.4, 3.8, 1.1],
[5.5, 2.4, 3.7, 1. ],
[5.8, 2.7, 3.9, 1.2],
[6. , 2.7, 5.1, 1.6],
[5.4, 3. , 4.5, 1.5],
[6. , 3.4, 4.5, 1.6],
[6.7, 3.1, 4.7, 1.5],
[6.3, 2.3, 4.4, 1.3],
[5.6, 3. , 4.1, 1.3],
[5.5, 2.5, 4. , 1.3],
[5.5, 2.6, 4.4, 1.2],
[6.1, 3. , 4.6, 1.4],
[5.8, 2.6, 4. , 1.2],
[5. , 2.3, 3.3, 1. ],
[5.6, 2.7, 4.2, 1.3],
[5.7, 3. , 4.2, 1.2],
[5.7, 2.9, 4.2, 1.3],
[6.2, 2.9, 4.3, 1.3],
[5.1, 2.5, 3. , 1.1],
[5.7, 2.8, 4.1, 1.3],
[6.3, 3.3, 6. , 2.5],
[5.8, 2.7, 5.1, 1.9],
[7.1, 3. , 5.9, 2.1],
[6.3, 2.9, 5.6, 1.8],
[6.5, 3. , 5.8, 2.2],
[7.6, 3. , 6.6, 2.1],
[4.9, 2.5, 4.5, 1.7],
[7.3, 2.9, 6.3, 1.8],
[6.7, 2.5, 5.8, 1.8],
[7.2, 3.6, 6.1, 2.5],
[6.5, 3.2, 5.1, 2. ],
[6.4, 2.7, 5.3, 1.9],
[6.8, 3. , 5.5, 2.1],
[5.7, 2.5, 5. , 2. ],
[5.8, 2.8, 5.1, 2.4],
[6.4, 3.2, 5.3, 2.3],
[6.5, 3. , 5.5, 1.8],
[7.7, 3.8, 6.7, 2.2],
[7.7, 2.6, 6.9, 2.3],
[6. , 2.2, 5. , 1.5],
```

```
       [6.9, 3.2, 5.7, 2.3],
       [5.6, 2.8, 4.9, 2. ],
       [7.7, 2.8, 6.7, 2. ],
       [6.3, 2.7, 4.9, 1.8],
       [6.7, 3.3, 5.7, 2.1],
       [7.2, 3.2, 6. , 1.8],
       [6.2, 2.8, 4.8, 1.8],
       [6.1, 3. , 4.9, 1.8],
       [6.4, 2.8, 5.6, 2.1],
       [7.2, 3. , 5.8, 1.6],
       [7.4, 2.8, 6.1, 1.9],
       [7.9, 3.8, 6.4, 2. ],
       [6.4, 2.8, 5.6, 2.2],
       [6.3, 2.8, 5.1, 1.5],
       [6.1, 2.6, 5.6, 1.4],
       [7.7, 3. , 6.1, 2.3],
       [6.3, 3.4, 5.6, 2.4],
       [6.4, 3.1, 5.5, 1.8],
       [6. , 3. , 4.8, 1.8],
       [6.9, 3.1, 5.4, 2.1],
       [6.7, 3.1, 5.6, 2.4],
       [6.9, 3.1, 5.1, 2.3],
       [5.8, 2.7, 5.1, 1.9],
       [6.8, 3.2, 5.9, 2.3],
       [6.7, 3.3, 5.7, 2.5],
       [6.7, 3. , 5.2, 2.3],
       [6.3, 2.5, 5. , 1.9],
       [6.5, 3. , 5.2, 2. ],
       [6.2, 3.4, 5.4, 2.3],
       [5.9, 3. , 5.1, 1.8]])
```

In [5]:
```
y
```

Out[5]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

In [6]:
```
from sklearn.model_selection import train_test_split
```

In [7]:
```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random
```

In [8]:
```
x_train.shape, x_test.shape, y_train.shape, y_test.shape
```

Out[8]:
```
((112, 4), (38, 4), (112,), (38,))
```

In [9]:
```
from sklearn.tree import DecisionTreeClassifier
```

In [10]:
```
dt = DecisionTreeClassifier()
```

In [11]:
```
dt.fit(x_train, y_train)
```

Out[11]:

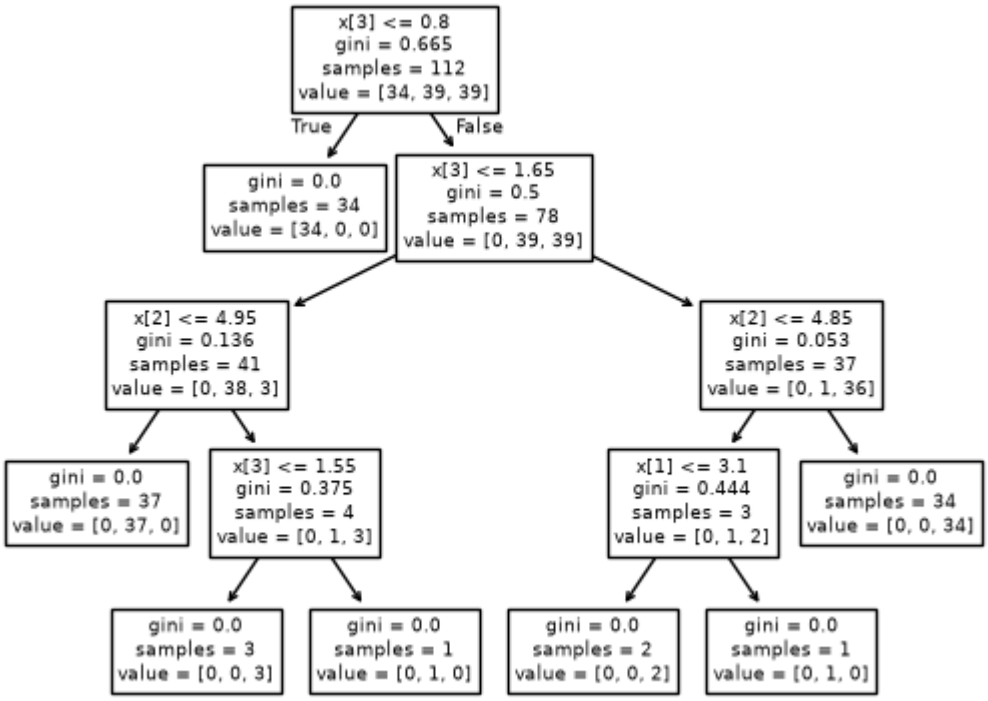
DecisionTreeClassifier

DecisionTreeClassifier()

In [12]:
```
from sklearn.tree import plot_tree
```

In [13]:
```
plot_tree(dt)
```

Out[13]:
```
[Text(0.4, 0.9, 'x[3] <= 0.8\ngini = 0.665\nsamples = 112\nvalue = [34, 39, 3
9]'),
 Text(0.3, 0.7, 'gini = 0.0\nsamples = 34\nvalue = [34, 0, 0]'),
 Text(0.35, 0.8, 'True  '),
 Text(0.5, 0.7, 'x[3] <= 1.65\ngini = 0.5\nsamples = 78\nvalue = [0, 39, 39]'),
 Text(0.45, 0.8, '  False'),
 Text(0.2, 0.5, 'x[2] <= 4.95\ngini = 0.136\nsamples = 41\nvalue = [0, 38,
3]'),
 Text(0.1, 0.3, 'gini = 0.0\nsamples = 37\nvalue = [0, 37, 0]'),
 Text(0.3, 0.3, 'x[3] <= 1.55\ngini = 0.375\nsamples = 4\nvalue = [0, 1, 3]'),
 Text(0.2, 0.1, 'gini = 0.0\nsamples = 3\nvalue = [0, 0, 3]'),
 Text(0.4, 0.1, 'gini = 0.0\nsamples = 1\nvalue = [0, 1, 0]'),
 Text(0.8, 0.5, 'x[2] <= 4.85\ngini = 0.053\nsamples = 37\nvalue = [0, 1, 3
6]'),
 Text(0.7, 0.3, 'x[1] <= 3.1\ngini = 0.444\nsamples = 3\nvalue = [0, 1, 2]'),
 Text(0.6, 0.1, 'gini = 0.0\nsamples = 2\nvalue = [0, 0, 2]'),
 Text(0.8, 0.1, 'gini = 0.0\nsamples = 1\nvalue = [0, 1, 0]'),
 Text(0.9, 0.3, 'gini = 0.0\nsamples = 34\nvalue = [0, 0, 34]')]
```



In [14]:
```
prediction = dt.predict(x_test)
```

In [15]:
```
from sklearn.metrics import accuracy_score, confusion_matrix
```

In [16]:
```
accuracy_score(y_test, prediction)
```

Out[16]:
```
0.9473684210526315
```

In [17]:
```
confusion_matrix(y_test, prediction)
```

Out[17]:
```
array([[16,  0,  0],
       [ 0, 10,  1],
       [ 0,  1, 10]])
```

In [1]:
```
import pandas as pd
```

In [2]:
```
data = pd.read_csv(r"F:\MSC DS 02\diabetes.csv")
data
```

Out[2]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigre
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	
...	...	...	...	...	...	...	
763	10	101	76	48	180	32.9	
764	2	122	70	27	0	36.8	
765	5	121	72	23	112	26.2	
766	1	126	60	0	0	30.1	
767	1	93	70	31	0	30.4	

768 rows × 9 columns

In [3]:
```
X = data.iloc[:,:-1]
X
```

Out[3]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigre
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	
...	...	...	...	...	...	...	
763	10	101	76	48	180	32.9	
764	2	122	70	27	0	36.8	
765	5	121	72	23	112	26.2	
766	1	126	60	0	0	30.1	
767	1	93	70	31	0	30.4	

768 rows × 8 columns

In [4]:
```
Y = data.iloc[:,-1]
Y
```

Out[4]:
```
0      1
1      0
2      1
3      0
4      1
      ..
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64
```

In [5]:
```
from sklearn.model_selection import train_test_split
```

In [6]:
```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.25, random
```

In [7]:
```
X_train.shape, X_test.shape, Y_train.shape, Y_test.shape
```

Out[7]:
```
((576, 8), (192, 8), (576,), (192,))
```

In [8]:
```
from sklearn.linear_model import LogisticRegression
```

In [9]:
```
LR = LogisticRegression()
```

In [10]:
```
LR.fit(X_train, Y_train)
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:451:
OptimizeWarning: Unknown solver options: iprint
  opt_res = optimize.minimize(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:465:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

Out[10]:

▾ LogisticRegression

LogisticRegression()

In [11]:
```
Prediction = LR.predict(X_test)
```

In [12]:
```
from sklearn.metrics import accuracy_score, confusion_matrix
```

In [13]:
```
accuracy_score(Y_test, Prediction)
```

Out[13]:
```
0.7604166666666666
```

In [14]:
```
confusion_matrix(Y_test, Prediction)
```

Out[14]:
```
array([[120,  14],
       [ 32,  26]])
```

In [1]:
```
from sklearn.datasets import load_breast_cancer
```

In [2]:
```
ds = load_breast_cancer()
```

In [3]:
```
ds
```

Out[3]:

```
{'data': array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
         1.189e-01],
        [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
         8.902e-02],
        [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
         8.758e-02],
        ...,
        [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
         7.820e-02],
        [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
         1.240e-01],
        [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
         7.039e-02]], shape=(569, 30)),
 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
1, 1,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
        1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
        1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
        1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
        0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
        1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
        0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
        1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
        1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,
        0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
        1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1,
        1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
        1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
        1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
        1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
        1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1]),
 'frame': None,
 'target_names': array(['malignant', 'benign'], dtype='<U9'),
 'DESCR': '.. _breast_cancer_dataset:\n\nBreast cancer wisconsin (diagnostic) d
ataset\n--------------------------------------------\n\n**Data Set Characterist
ics:**\n\n:Number of Instances: 569\n\n:Number of Attributes: 30 numeric, predi
ctive attributes and the class\n\n:Attribute Information:\n    - radius (mean o
f distances from center to points on the perimeter)\n    - texture (standard de
viation of gray-scale values)\n    - perimeter\n    - area\n    - smoothness (l
ocal variation in radius lengths)\n    - compactness (perimeter^2 / area - 1.0)
\n    - concavity (severity of concave portions of the contour)\n    - concave
points (number of concave portions of the contour)\n    - symmetry\n    - fract
al dimension ("coastline approximation" - 1)\n\n    The mean, standard error, a
nd "worst" or largest (mean of the three\n    worst/largest values) of these fe
atures were computed for each image,\n    resulting in 30 features.  For instan
ce, field 0 is Mean Radius, field\n    10 is Radius SE, field 20 is Worst Radiu
s.\n\n    - class:\n            - WDBC-Malignant\n            - WDBC-Benign\n
\n:Summary Statistics:\n\n===================================== ====== ======\n
Min    Max\n===================================== ====== ======\nradius (mean):
6.981  28.11\ntexture (mean):                        9.71   39.28\nperimeter (me
an):                     43.79  188.5\narea (mean):                           14
```

```
3.5  2501.0\nsmoothness (mean):                    0.053  0.163\ncompactness (m
ean):                   0.019  0.345\nconcavity (mean):                     0.0
0.427\nconcave points (mean):                0.0    0.201\nsymmetry (mean):
0.106  0.304\nfractal dimension (mean):             0.05   0.097\nradius (stand
ard error):              0.112  2.873\ntexture (standard error):             0.
36   4.885\nperimeter (standard error):           0.757  21.98\narea (standard
error):                6.802  542.2\nsmoothness (standard error):          0.00
2  0.031\ncompactness (standard error):         0.002  0.135\nconcavity (standa
rd error):           0.0    0.396\nconcave points (standard error):      0.0
0.053\nsymmetry (standard error):            0.008  0.079\nfractal dimension (s
tandard error):   0.001  0.03\nradius (worst):                       7.93   36.
04\ntexture (worst):                       12.02  49.54\nperimeter (worst):
50.41  251.2\narea (worst):                           185.2  4254.0\nsmoothness
(worst):                   0.071  0.223\ncompactness (worst):
0.027  1.058\nconcavity (worst):                     0.0    1.252\nconcave point
s (worst):               0.0    0.291\nsymmetry (worst):                     0.
156  0.664\nfractal dimension (worst):            0.055  0.208\n==============
====================== ====== ======\n\n:Missing Attribute Values: None\n\n:Cla
ss Distribution: 212 - Malignant, 357 - Benign\n\n:Creator:  Dr. William H. Wol
berg, W. Nick Street, Olvi L. Mangasarian\n\n:Donor: Nick Street\n\n:Date: Nove
mber, 1995\n\nThis is a copy of UCI ML Breast Cancer Wisconsin (Diagnostic) dat
asets.\nhttps://goo.gl/U2Uwz2\n\nFeatures are computed from a digitized image o
f a fine needle\naspirate (FNA) of a breast mass.  They describe\ncharacteristi
cs of the cell nuclei present in the image.\n\nSeparating plane described above
was obtained using\nMultisurface Method-Tree (MSM-T) [K. P. Bennett, "Decision
Tree\nConstruction Via Linear Programming." Proceedings of the 4th\nMidwest Art
ificial Intelligence and Cognitive Science Society,\npp. 97-101, 1992], a class
ification method which uses linear\nprogramming to construct a decision tree.
Relevant features\nwere selected using an exhaustive search in the space of 1-4
\nfeatures and 1-3 separating planes.\n\nThe actual linear program used to obta
in the separating plane\nin the 3-dimensional space is that described in:\n[K.
P. Bennett and O. L. Mangasarian: "Robust Linear\nProgramming Discrimination of
Two Linearly Inseparable Sets",\nOptimization Methods and Software 1, 1992, 23-
34].\n\nThis database is also available through the UW CS ftp server:\n\nftp ft
p.cs.wisc.edu\ncd math-prog/cpo-dataset/machine-learn/WDBC/\n\n.. dropdown:: Re
ferences\n\n  - W.N. Street, W.H. Wolberg and O.L. Mangasarian. Nuclear feature
extraction\n    for breast tumor diagnosis. IS&T/SPIE 1993 International Sympos
ium on\n    Electronic Imaging: Science and Technology, volume 1905, pages 861-
870,\n    San Jose, CA, 1993.\n  - O.L. Mangasarian, W.N. Street and W.H. Wolbe
rg. Breast cancer diagnosis and\n    prognosis via linear programming. Operatio
ns Research, 43(4), pages 570-577,\n    July-August 1995.\n  - W.H. Wolberg, W.
N. Street, and O.L. Mangasarian. Machine learning techniques\n    to diagnose b
reast cancer from fine-needle aspirates. Cancer Letters 77 (1994)\n    163-17
1.\n',
 'feature_names': array(['mean radius', 'mean texture', 'mean perimeter', 'mean
area',
        'mean smoothness', 'mean compactness', 'mean concavity',
        'mean concave points', 'mean symmetry', 'mean fractal dimension',
        'radius error', 'texture error', 'perimeter error', 'area error',
        'smoothness error', 'compactness error', 'concavity error',
        'concave points error', 'symmetry error',
        'fractal dimension error', 'worst radius', 'worst texture',
        'worst perimeter', 'worst area', 'worst smoothness',
        'worst compactness', 'worst concavity', 'worst concave points',
        'worst symmetry', 'worst fractal dimension'], dtype='<U23'),
 'filename': 'breast_cancer.csv',
 'data_module': 'sklearn.datasets.data'}
```

In [4]:
```
x, y = ds.data, ds.target
```

In [5]:
```
x
```

Out[5]:
```
array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
        1.189e-01],
       [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
        8.902e-02],
       [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
        8.758e-02],
       ...,
       [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
        7.820e-02],
       [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
        1.240e-01],
       [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
        7.039e-02]], shape=(569, 30))
```

In [6]:
```
y
```

Out[6]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
       1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
       1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
       1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
       1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,
       0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
       1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
       1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
       1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
       1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1])
```

In [7]:
```
from sklearn.model_selection import train_test_split
```

In [8]:
```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random
```

In [9]:
```
x_train.shape, x_test.shape, y_train.shape, y_test.shape
```

Out[9]:
```
((426, 30), (143, 30), (426,), (143,))
```

In [10]:
```
from sklearn.linear_model import LogisticRegression
```

In [11]:
```
lr = LogisticRegression()
```

In [12]:
```
lr.fit(x_train, y_train)
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:451:
OptimizeWarning: Unknown solver options: iprint
  opt_res = optimize.minimize(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:465:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

Out[12]:

▾ LogisticRegression

LogisticRegression()

In [13]:
```
prediction = lr.predict(x_test)
```

In [14]:
```
from sklearn.metrics import accuracy_score, confusion_matrix
```

In [15]:
```
accuracy_score(y_test, prediction)
```

Out[15]:
```
0.9370629370629371
```

In [16]:
```
confusion_matrix(y_test, prediction)
```

Out[16]:
```
array([[52,  4],
       [ 5, 82]])
```

In [1]:
```
from sklearn.datasets import load_iris
```

In [2]:
```
import pandas as pd
```

In [3]:
```
dataset = load_iris()
```

In [4]:
```
df = pd.DataFrame(dataset.data, columns = ['sepal_length', 'sepal_width', 'petal
```

In [5]:
```
df.sample(2)
```

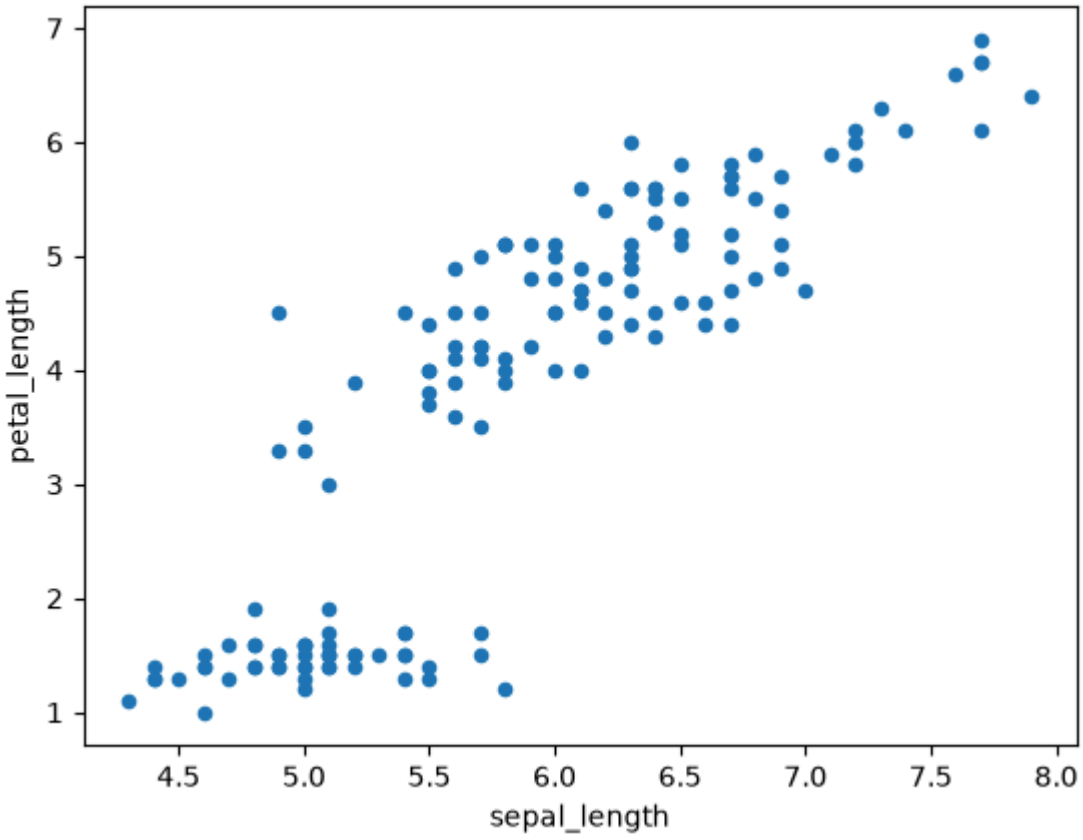
Out[5]:

	sepal_length	sepal_width	petal_length	petal_width
11	4.8	3.4	1.6	0.2
105	7.6	3.0	6.6	2.1

In [6]:
```
from matplotlib import pyplot as plt
```

In [7]:
```
df.plot(kind = 'scatter', x = 'sepal_length', y = 'petal_length')
```

Out[7]:
```
<Axes: xlabel='sepal_length', ylabel='petal_length'>
```



In [8]:
```
from sklearn.cluster import KMeans
```

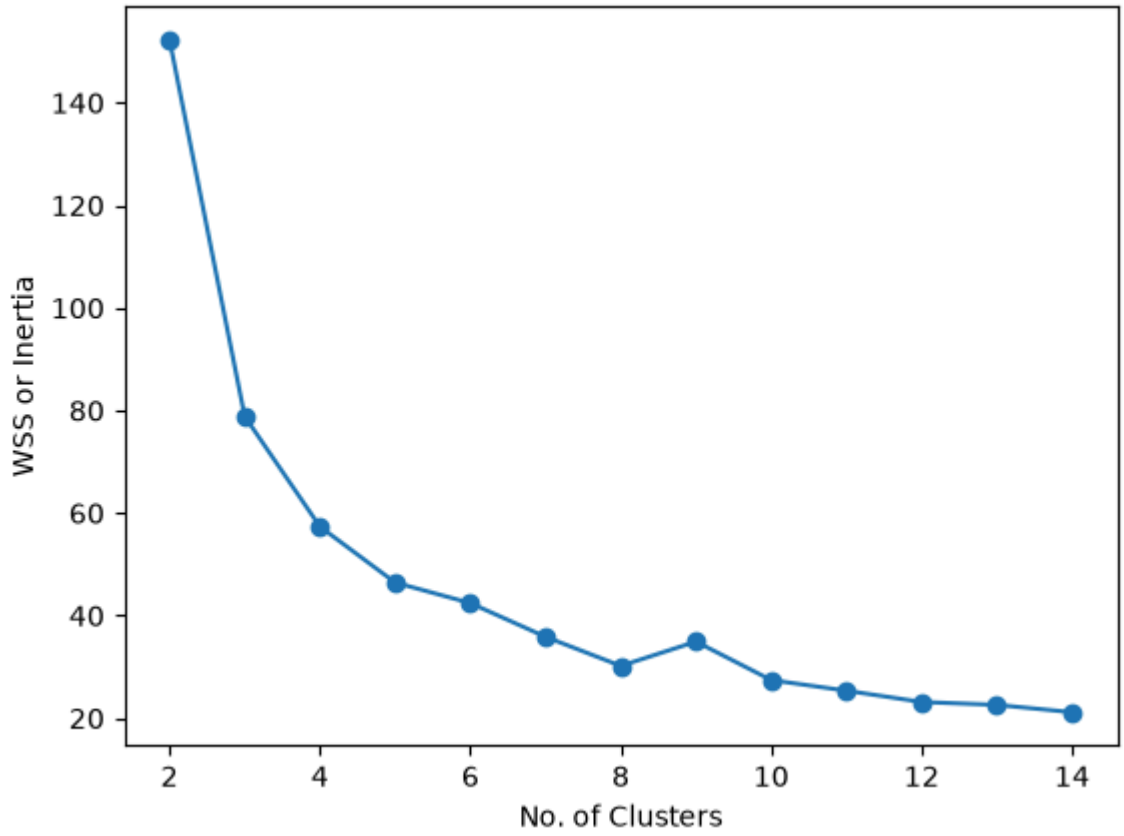
In [9]:
```
distortion = []
k = range(2,15)
```

In [10]:
```
for i in k:
    kmeans_model = KMeans(n_clusters=i, max_iter=25)
```

```
    kmeans_model.fit(df)
    distortion.append(kmeans_model.inertia_)
```

```
C:\ProgramData\anaconda3\Lib\site-packages\joblib\externals\loky\backend\context.
py:136: UserWarning: Could not find the number of physical cores for the followin
g reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by se
tting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
  warnings.warn(
  File "C:\ProgramData\anaconda3\Lib\site-packages\joblib\externals\loky\backend
\context.py", line 257, in _count_physical_cores
    cpu_info = subprocess.run(
        "wmic CPU Get NumberOfCores /Format:csv".split(),
        capture_output=True,
        text=True,
    )
  File "C:\ProgramData\anaconda3\Lib\subprocess.py", line 554, in run
    with Popen(*popenargs, **kwargs) as process:
         ~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "C:\ProgramData\anaconda3\Lib\subprocess.py", line 1039, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    ~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                        pass_fds, cwd, env,
                        ^^^^^^^^^^^^^^^^^^^
    ...<5 lines>...
                        gid, gids, uid, umask,
                        ^^^^^^^^^^^^^^^^^^^^^^
                        start_new_session, process_group)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\ProgramData\anaconda3\Lib\subprocess.py", line 1554, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
                       ~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^
                            # no special security
                            ^^^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
                            cwd,
                            ^^^^
                            startupinfo)
                            ^^^^^^^^^^^^
```

In [11]:
```
plt.plot(k, distortion, 'o-')
plt.xlabel('No. of Clusters')
plt.ylabel('WSS or Inertia')
plt.show()
```



In [12]:
```
cluster_model = KMeans(n_clusters=3)
cluster_model.fit(df)
```

Out[12]:
```
KMeans(n_clusters=3)
```

In [13]:
```
cluster_model.cluster_centers_
```

Out[13]:
```
array([[5.006     , 3.428     , 1.462     , 0.246     ],
       [6.85      , 3.07368421, 5.74210526, 2.07105263],
       [5.9016129 , 2.7483871 , 4.39354839, 1.43387097]])
```

In [14]:
```
cluster_model.labels_
```

Out[14]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 1, 1, 1, 1, 2, 1, 1, 1,
       1, 1, 1, 2, 2, 1, 1, 1, 1, 2, 1, 2, 1, 2, 1, 1, 2, 2, 1, 1, 1, 1,
       1, 2, 1, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 2], dtype=int32)
```

In [15]:
```
df['cluster_no'] = cluster_model.predict(df)
```

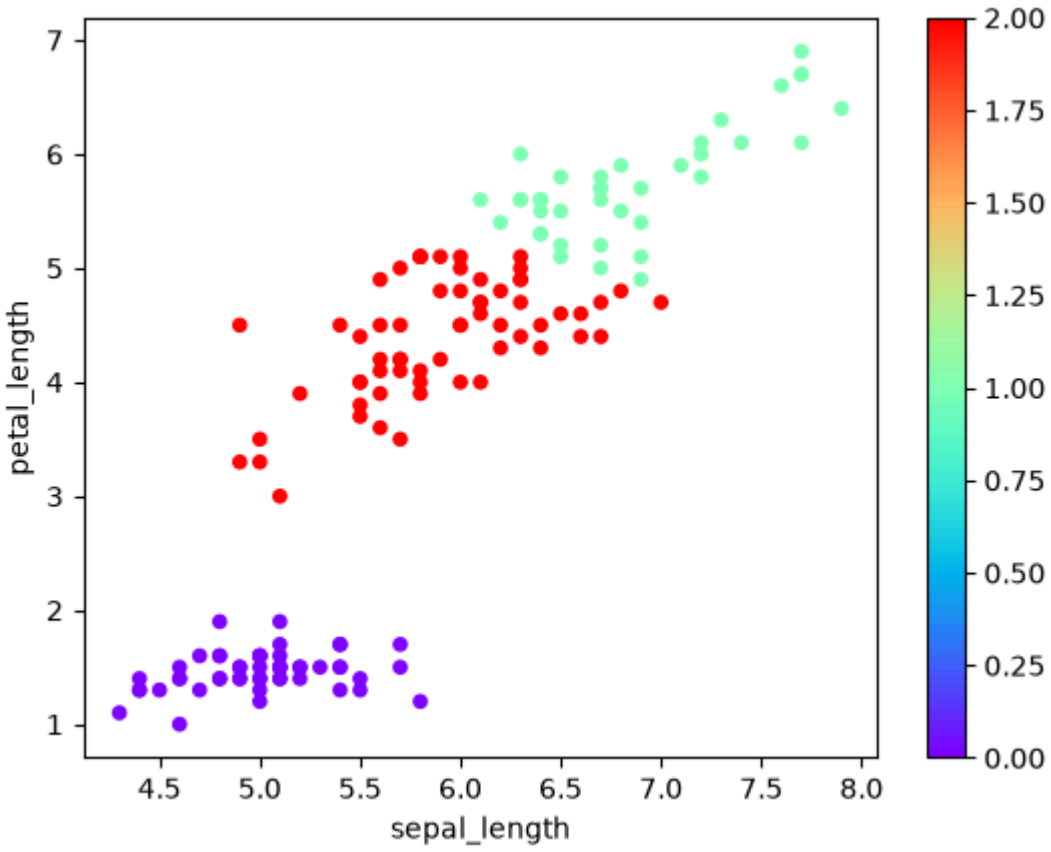
In [16]:
```
df.head()
```

Out[16]:

	sepal_length	sepal_width	petal_length	petal_width	cluster_no
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

In [17]:
```
df.plot(kind = 'scatter', x = 'sepal_length', y = 'petal_length', c= df['cluster
```

Out[17]:
```
<Axes: xlabel='sepal_length', ylabel='petal_length'>
```



Practical No: 05
Name: Mohd Danish Ansari
Roll No: 13
Date: 12/09/26

Aim : To perform file storage, retrieval, and management operations using Hadoop Distributed File System (HDFS), and to explore the Hadoop environment and architecture through command-line interface and web interface.

Commands for Linux

1. sudo : Used to perform tasks that requires root permissions.
2. pwd : It shows the present working directory.

```
[cloudera@quickstart ~]$ pwd
/home/cloudera
```

3. cd : It is use to come back in the root directory.

```
[cloudera@quickstart ~]$ cd Desktop
[cloudera@quickstart Desktop]$
```

4. ls : It list down all the directories and folders present.

```
[cloudera@quickstart Desktop]$ ls
BDTDATA  Eclipse.desktop     Express.desktop  matix.pig~  wordcount.pig
data     Enterprise.desktop  matix.pig        Matrix      wordcount.pig~
```

5. cat : Displays the content of the file.

```
[cloudera@quickstart data]$ cat sid.txt
Hii Siddhesh here,
studying in DSAI Part-2
```

6. cp : Copy one file from current directory to another directory.

```
[cloudera@quickstart data]$ cp sid.txt /home/cloudera/Desktop/data/d1
[cloudera@quickstart data]$ cd d1
[cloudera@quickstart d1]$ ls
sid.txt
```

7. mv : Used to move and rename files and directories.

```
[cloudera@quickstart d1]$ mv sid.txt move.txt
[cloudera@quickstart d1]$ ls
move.txt
[cloudera@quickstart d1]$ cat move.txt
Hii Siddhesh here,
studying in DSAI Part-2
```

8. mkdir : Creates a new directory.

```
[cloudera@quickstart Desktop]$ mkdir siddhesh
[cloudera@quickstart Desktop]$ ls
BDTDATA  Eclipse.desktop     Express.desktop  matix.pig~  siddhesh       wordcount.pig~
data     Enterprise.desktop  matix.pig        Matrix      wordcount.pig
[cloudera@quickstart Desktop]$
```

9. rmdir : Deletes the directory.

```
[cloudera@quickstart Desktop]$ rmdir siddhesh
[cloudera@quickstart Desktop]$ ls
BDTDATA  Eclipse.desktop     Express.desktop  matix.pig~  wordcount.pig
data     Enterprise.desktop  matix.pig        Matrix      wordcount.pig~
[cloudera@quickstart Desktop]$
```

10. rm : Removes/ Deletes the file from a directory.

```
[cloudera@quickstart data]$ rm siddhesh.txt
rm: remove regular empty file `siddhesh.txt'? Y
[cloudera@quickstart data]$
```

11. touch : Creates an empty file.

```
[cloudera@quickstart data]$ touch siddhesh.txt
[cloudera@quickstart data]$ ls
details.txt  info.txt  siddhesh.txt  sid.txt
[cloudera@quickstart data]$
```

12. stat : Detail view of the file.

```
[cloudera@quickstart data]$ stat siddhesh.txt
  File: `siddhesh.txt'
  Size: 0               Blocks: 0          IO Block: 4096   regular empty file
Device: fd00h/64768d    Inode: 3278872     Links: 1
Access: (0664/-rw-rw-r--)  Uid: (  501/cloudera)   Gid: (  501/cloudera)
Access: 2023-04-02 21:02:23.116475076 -0700
Modify: 2023-04-02 21:02:23.116475076 -0700
Change: 2023-04-02 21:02:23.116475076 -0700
[cloudera@quickstart data]$
```

13. vi : Creates and open the file.

```
[cloudera@quickstart data]$ vi siddhesh.txt
```

```
Hii
This is Siddhesh
Studying in DSAi
Part-2
RJ Clg
Mumbai
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
-- INSERT --                                                    7,1           All
```

14. head : Displays the first 10 lines of a text file.

```
[cloudera@quickstart data]$ head siddhesh.txt
Hii
This is Siddhesh
Studying in DSAi
Part-2
RJ Clg
Mumbai
```

15. tail : Displays the last 10 lines of a text file.

```
[cloudera@quickstart data]$ tail siddhesh.txt
Hii
This is Siddhesh
Studying in DSAi
Part-2
RJ Clg
Mumbai
```

Commands for Hadoop

1. hadoop version : Gives the version of the hadoop.

```
[cloudera@quickstart ~]$ hadoop version
Hadoop 2.6.0-cdh5.4.2
Subversion http://github.com/cloudera/hadoop -r 15b703c8725733b7b2813d2325659eb7d57e7a3f
Compiled by jenkins on 2015-05-20T00:03Z
Compiled with protoc 2.5.0
From source with checksum de74f1adb3744f8ee85d9a5b98f90d
This command was run using /usr/jars/hadoop-common-2.6.0-cdh5.4.2.jar
```

2. ls : List down all the files in hadoop.

```
[cloudera@quickstart ~]$ hadoop fs -ls
Found 4 items
drwxr-xr-x   - cloudera cloudera          0 2023-03-19 23:10 .Trash
drwxr-xr-x   - cloudera cloudera          0 2023-01-16 20:18 myHDFSdir
drwxr-xr-x   - cloudera cloudera          0 2023-03-27 23:07 oozie-oozi
drwxr-xr-x   - cloudera cloudera          0 2023-03-01 21:56 training
```

```
[cloudera@quickstart ~]$ hdfs dfs -ls /
Found 9 items
drwxr-xr-x   - hbase    supergroup          0 2023-03-27 22:21 /hbase
drwxr-xr-x   - cloudera supergroup          0 2023-01-22 23:14 /inputdir
drwxr-xr-x   - cloudera supergroup          0 2023-01-29 23:21 /matrix
drwxr-xr-x   - cloudera supergroup          0 2023-01-16 20:25 /myHDFSdir
drwxr-xr-x   - cloudera supergroup          0 2023-03-28 00:07 /myimpaladata
drwxr-xr-x   - solr     solr                0 2023-03-27 22:26 /solr
drwxrwxrwx   - hdfs     supergroup          0 2023-03-27 23:52 /tmp
drwxr-xr-x   - hdfs     supergroup          0 2022-11-11 20:02 /user
drwxr-xr-x   - hdfs     supergroup          0 2015-06-09 03:36 /var
```

3. mkdir : Create a directory.

```
[cloudera@quickstart ~]$ hdfs dfs -mkdir /siddhesh
[cloudera@quickstart ~]$ hdfs dfs -ls /
Found 11 items
drwxrwxrwx   - hdfs     supergroup          0 2017-10-23 09:15 /benchmarks
drwxr-xr-x   - cloudera supergroup          0 2023-02-18 00:53 /char_inputdir
drwxr-xr-x   - cloudera supergroup          0 2023-02-18 01:01 /char_outputdir
drwxr-xr-x   - hbase    supergroup          0 2023-04-03 06:12 /hbase
drwxr-xr-x   - cloudera supergroup          0 2023-02-25 06:51 /inputdir
drwxr-xr-x   - cloudera supergroup          0 2023-02-25 06:55 /outputsdir
drwxr-xr-x   - cloudera supergroup          0 2023-04-03 06:46 /siddhesh
```

4. touchz : Creates a new file.

```
[cloudera@quickstart ~]$ hdfs dfs -touchz /siddhesh/file.txt
```

5. put : Copy files/folders from local file system to HDFS.

```
[cloudera@quickstart ~]$ hdfs dfs -put Desktop/weather.txt /siddhesh
```

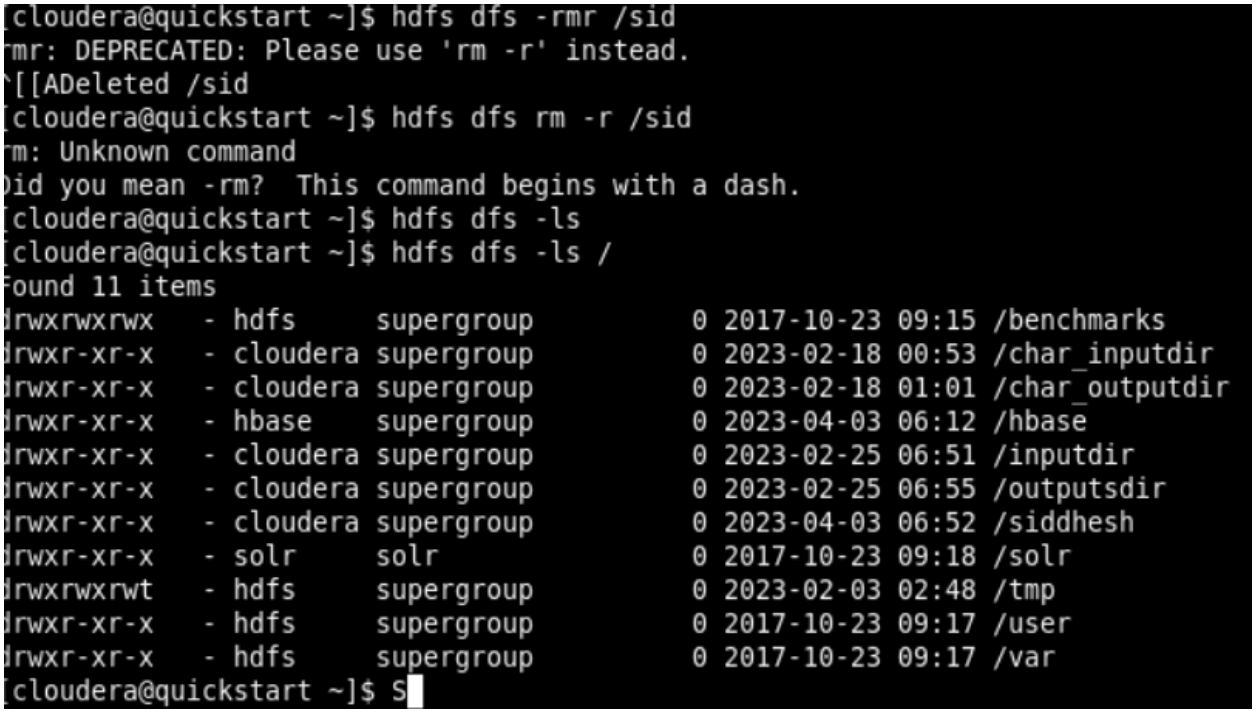
6. cat : Prints the file contents.

```
[cloudera@quickstart ~]$ hdfs dfs -cat /siddhesh/char_count.txt
Hii, myself Siddhesh.
I am studying in RJ College
I am a Data Science Student in RJ College
```

7. cp : Use to copy files within HDFS

```
[cloudera@quickstart ~]$ hdfs dfs -mkdir /sid
[cloudera@quickstart ~]$ hdfs -cp /siddhesh /sid
Error: Could not find or load main class .sid
[cloudera@quickstart ~]$ hdfs dfs -cp /siddhesh /sid
[cloudera@quickstart ~]$ hdfs dfs -ls /
Found 12 items
drwxrwxrwx   - hdfs     supergroup          0 2017-10-23 09:15 /benchmarks
drwxr-xr-x   - cloudera supergroup          0 2023-02-18 00:53 /char_inputdir
drwxr-xr-x   - cloudera supergroup          0 2023-02-18 01:01 /char_outputdir
drwxr-xr-x   - hbase    supergroup          0 2023-04-03 06:12 /hbase
drwxr-xr-x   - cloudera supergroup          0 2023-02-25 06:51 /inputdir
drwxr-xr-x   - cloudera supergroup          0 2023-02-25 06:55 /outputsdir
drwxr-xr-x   - cloudera supergroup          0 2023-04-03 06:56 /sid
drwxr-xr-x   - cloudera supergroup          0 2023-04-03 06:52 /siddhesh
drwxr-xr-x   - solr     solr                0 2017-10-23 09:18 /solr
drwxrwxrwt   - hdfs     supergroup          0 2023-02-03 02:48 /tmp
drwxr-xr-x   - hdfs     supergroup          0 2017-10-23 09:17 /user
drwxr-xr-x   - hdfs     supergroup          0 2017-10-23 09:17 /var
[cloudera@quickstart ~]$ hdfs dfs -cp /siddhesh /sid
cp: `/sid/siddhesh/char_count.txt': File exists
cp: `/sid/siddhesh/file.txt': File exists
cp: `/sid/siddhesh/weather.txt': File exists
[cloudera@quickstart ~]$ hdfs dfs -ls /sid
Found 1 items
drwxr-xr-x   - cloudera supergroup          0 2023-04-03 06:56 /sid/siddhesh
```

8. rm : Deletes a file from the HDFS.



Practical No: 06

Name : Mohd Danish Ansari
Roll No. : 13

Aim: To create and manage structured data using Apache Hive by defining managed and external tables, and to perform NoSQL data storage and retrieval operations using HBase including table creation and data manipulation.

Date:19-09-2026

Theory:

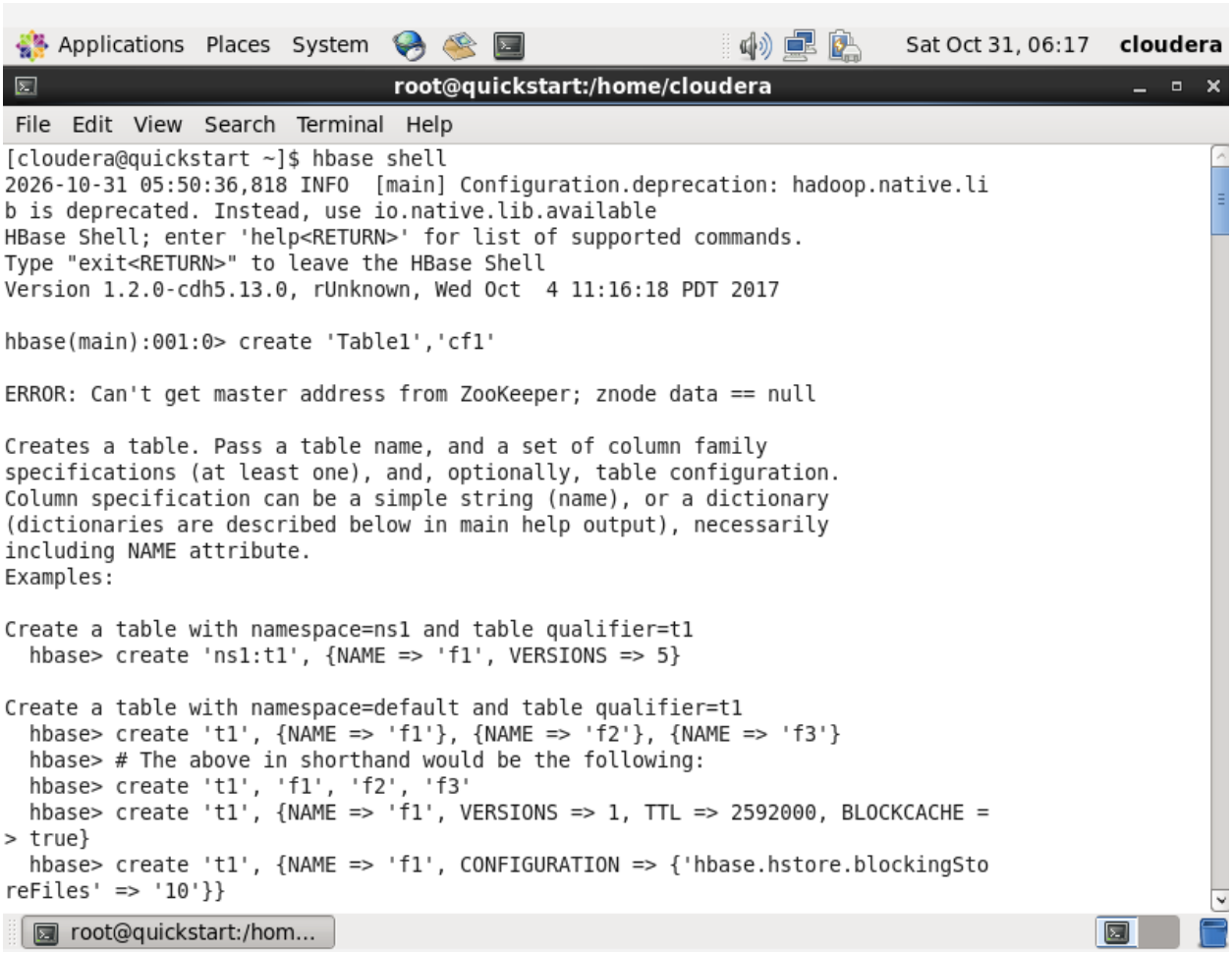
Apache HBase

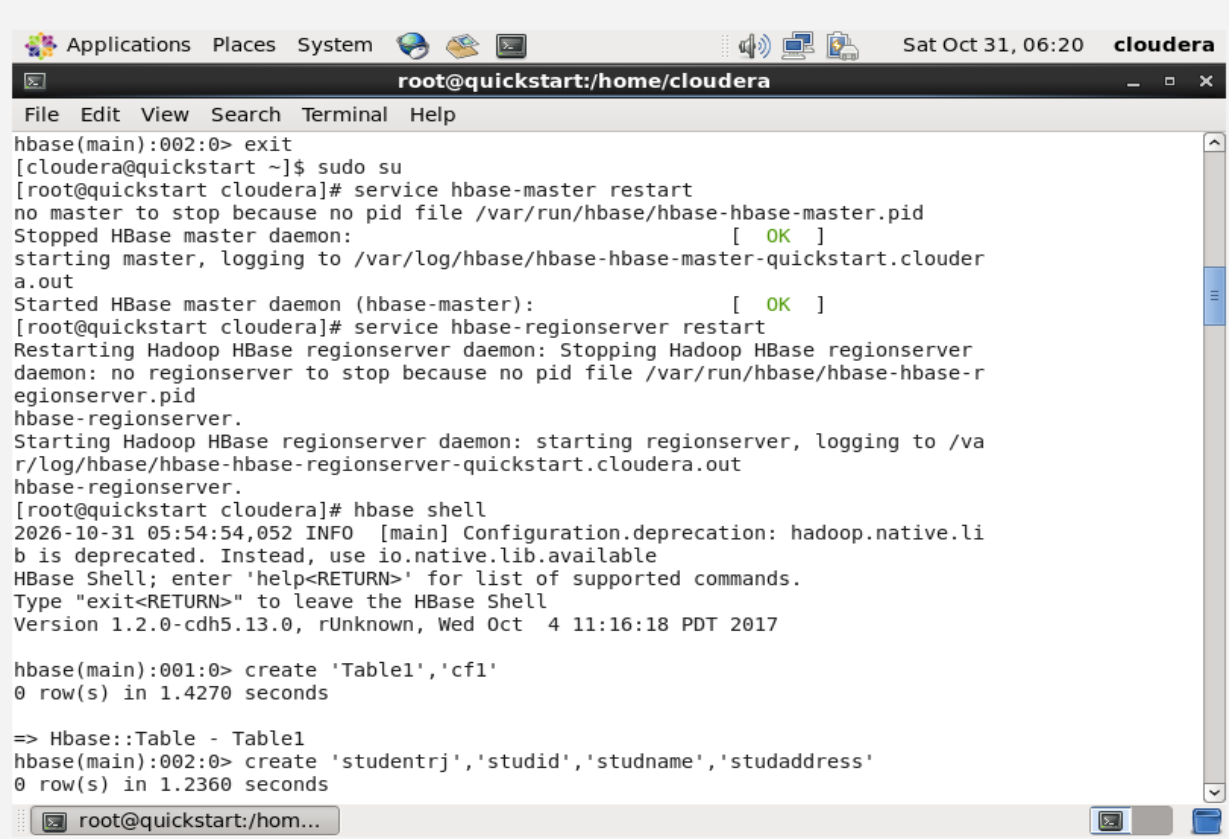
1. Apache HBase is an open-source, distributed NoSQL database built on Hadoop HDFS.
2. It is used to store and manage large amounts of structured and semi-structured data.
3. It follows a column-family-oriented data model and supports fast reading and writing.
4. It provides operations such as creating tables, inserting, retrieving, updating and deleting records.
5. It is widely used in big data applications because of its scalability, flexibility and high performance.

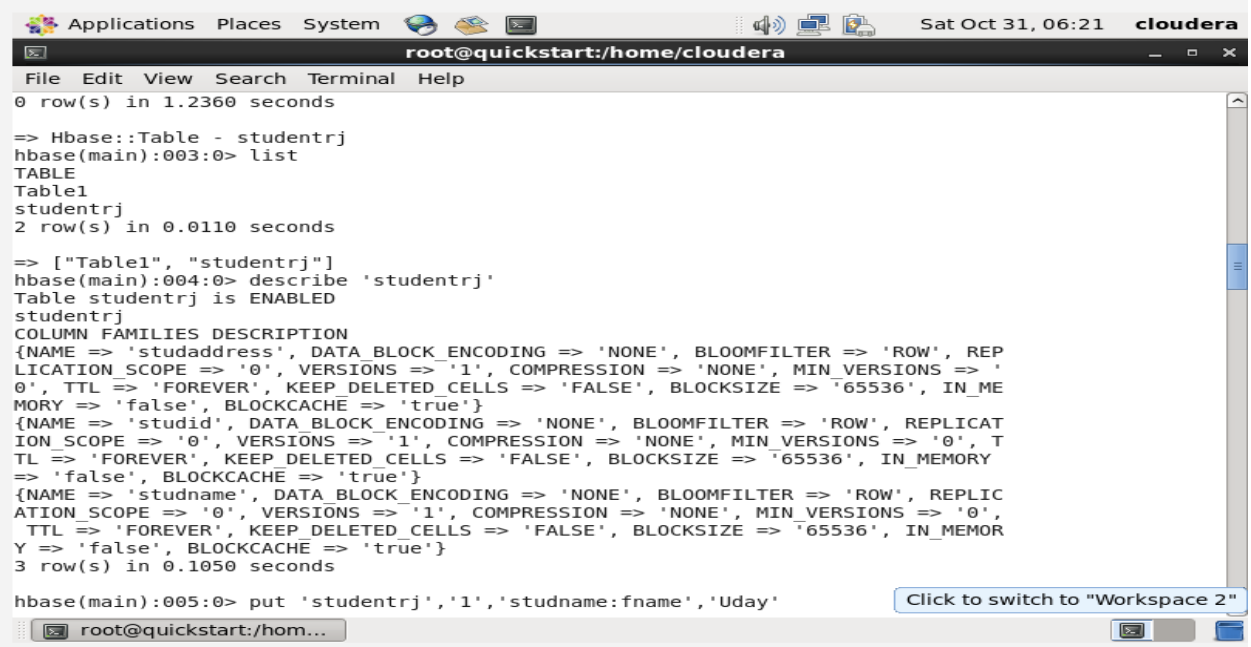
Apache Hive

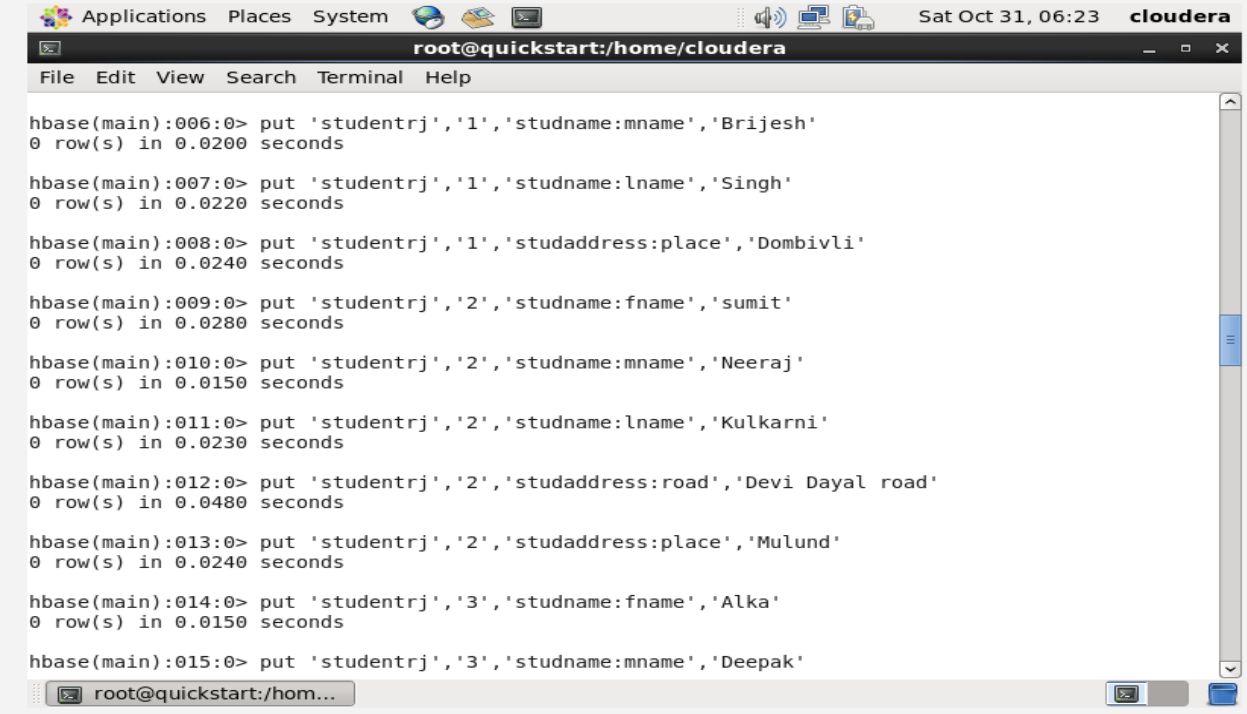
1. Apache Hive is a data warehouse tool built on top of Hadoop.
2. It is used to store, manage and analyze large datasets.
3. It provides an SQL-like query language called HiveQL.
4. It supports different types of tables, such as Internal, External and Temporary tables, along with partitioning and bucketing.
5. It is widely used for big data analysis, data summarization and data warehousing.

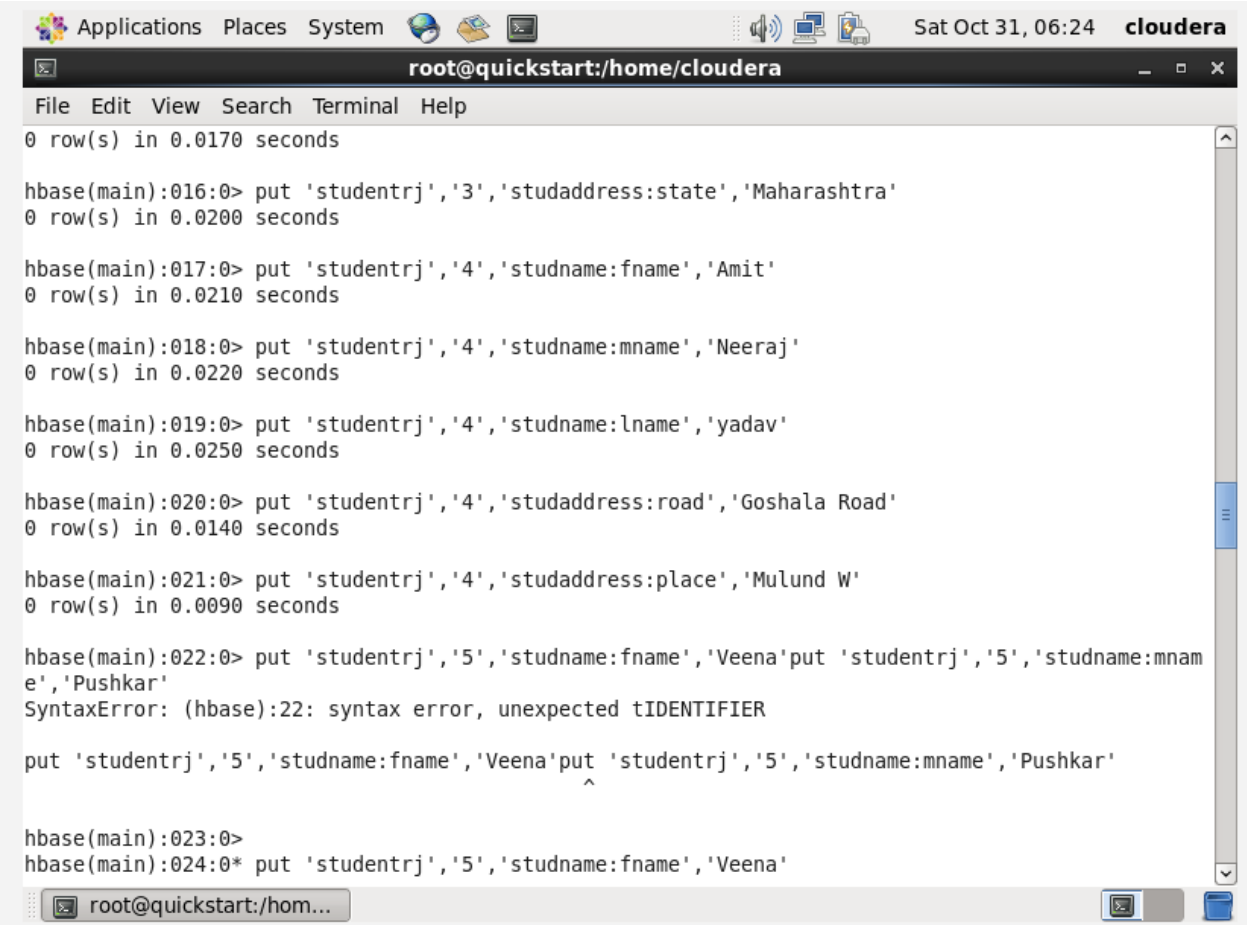
Hbase Code:

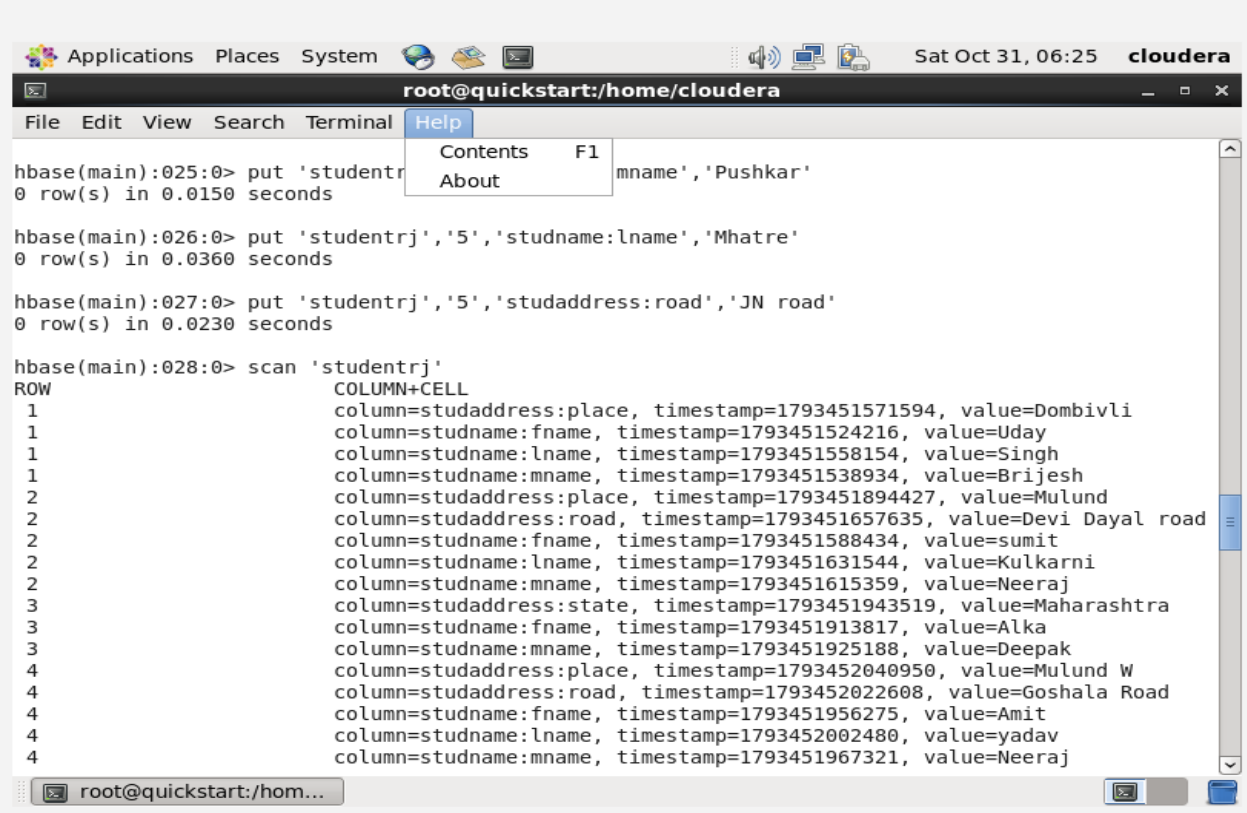

Applications Places System
Sat Oct 31, 06:17 cloudera
root@quickstart:/home/cloudera
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ hbase shell
2026-10-31 05:50:36,818 INFO [main] Configuration.deprecation: hadoop.native.li
b is deprecated. Instead, use io.native.lib.available
HBase Shell; enter 'help<RETURN>' for list of supported commands.
Type "exit<RETURN>" to leave the HBase Shell
Version 1.2.0-cdh5.13.0, rUnknown, Wed Oct 4 11:16:18 PDT 2017
hbase(main):001:0> create 'Table1','cf1'
ERROR: Can't get master address from ZooKeeper; znode data == null
Creates a table. Pass a table name, and a set of column family
specifications (at least one), and, optionally, table configuration.
Column specification can be a simple string (name), or a dictionary
(dictionaries are described below in main help output), necessarily
including NAME attribute.
Examples:
Create a table with namespace=ns1 and table qualifier=t1
hbase> create 'ns1:t1', {NAME => 'f1', VERSIONS => 5}
Create a table with namespace=default and table qualifier=t1
hbase> create 't1', {NAME => 'f1'}, {NAME => 'f2'}, {NAME => 'f3'}
hbase> # The above in shorthand would be the following:
hbase> create 't1', 'f1', 'f2', 'f3'
hbase> create 't1', {NAME => 'f1', VERSIONS => 1, TTL => 2592000, BLOCKCACHE =
> true}
hbase> create 't1', {NAME => 'f1', CONFIGURATION => {'hbase.hstore.blockingSto
reFiles' => '10'}}
root@quickstart:/hom...

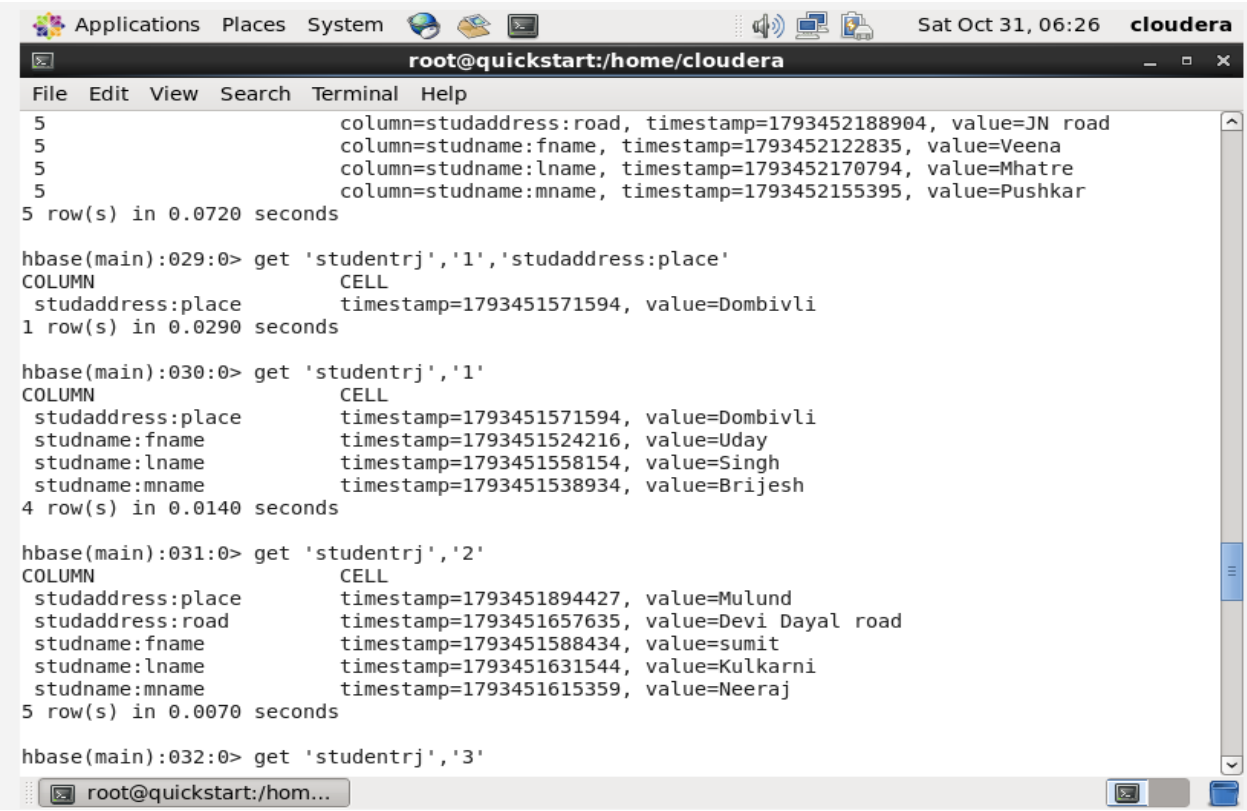

Applications Places System
Sat Oct 31, 06:20 cloudera
root@quickstart:/home/cloudera
File Edit View Search Terminal Help
hbase(main):002:0> exit
[cloudera@quickstart ~]$ sudo su
[root@quickstart cloudera]# service hbase-master restart
no master to stop because no pid file /var/run/hbase/hbase-hbase-master.pid
Stopped HBase master daemon: [OK]
starting master, logging to /var/log/hbase/hbase-hbase-master-quickstart.clouder
a.out
Started HBase master daemon (hbase-master): [OK]
[root@quickstart cloudera]# service hbase-regionserver restart
Restarting Hadoop HBase regionserver daemon: Stopping Hadoop HBase regionserver
daemon: no regionserver to stop because no pid file /var/run/hbase/hbase-hbase-r
egionserver.pid
hbase-regionserver.
Starting Hadoop HBase regionserver daemon: starting regionserver, logging to /va
r/log/hbase/hbase-hbase-regionserver-quickstart.cloudera.out
hbase-regionserver.
[root@quickstart cloudera]# hbase shell
2026-10-31 05:54:54,052 INFO [main] Configuration.deprecation: hadoop.native.li
b is deprecated. Instead, use io.native.lib.available
HBase Shell; enter 'help<RETURN>' for list of supported commands.
Type "exit<RETURN>" to leave the HBase Shell
Version 1.2.0-cdh5.13.0, rUnknown, Wed Oct 4 11:16:18 PDT 2017
hbase(main):001:0> create 'Table1','cf1'
0 row(s) in 1.4270 seconds
=> Hbase::Table - Table1
hbase(main):002:0> create 'studentrj','studid','studname','studaddress'
0 row(s) in 1.2360 seconds
root@quickstart:/hom...

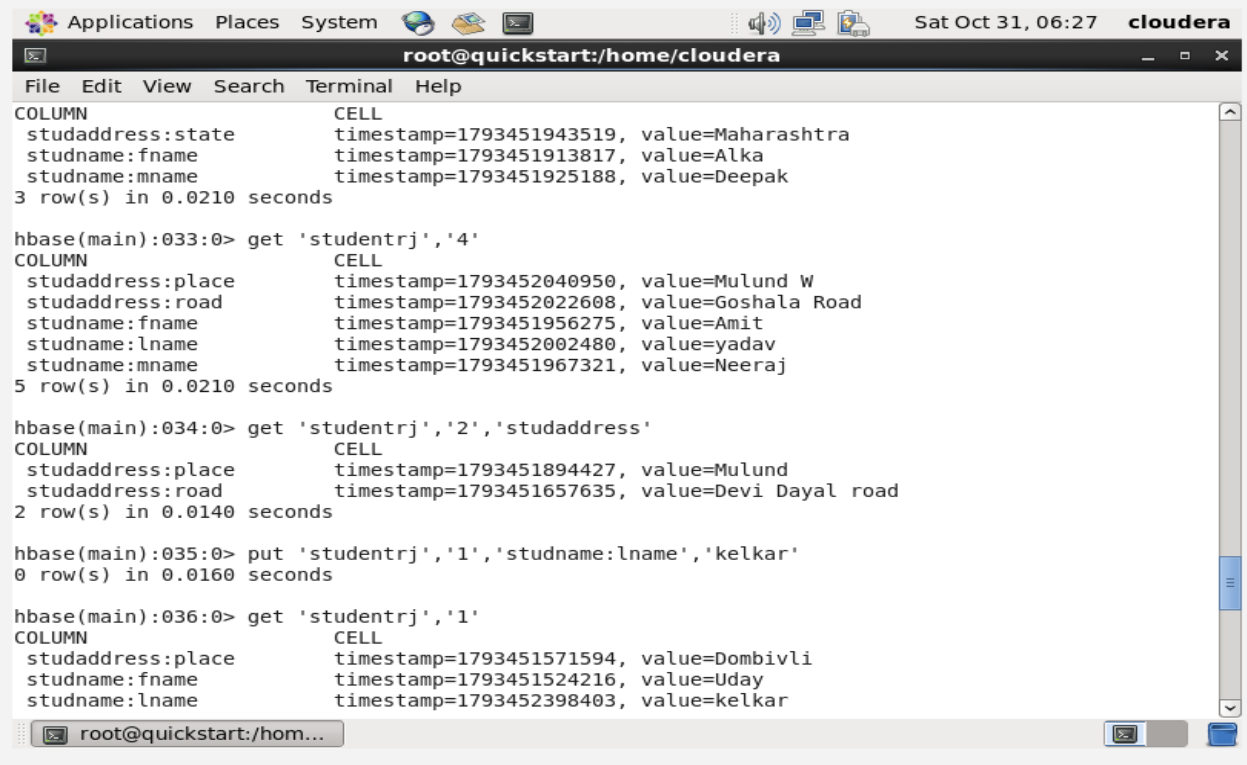

Applications Places System
Sat Oct 31, 06:21 cloudera
root@quickstart:/home/cloudera
File Edit View Search Terminal Help
0 row(s) in 1.2360 seconds
=> Hbase::Table - studentrj
hbase(main):003:0> list
TABLE
Table1
studentrj
2 row(s) in 0.0110 seconds
=> ["Table1", "studentrj"]
hbase(main):004:0> describe 'studentrj'
Table studentrj is ENABLED
studentrj
COLUMN FAMILIES DESCRIPTION
{NAME => 'studaddress', DATA_BLOCK_ENCODING => 'NONE', BLOOMFILTER => 'ROW', REP
LICATION_SCOPE => '0', VERSIONS => '1', COMPRESSION => 'NONE', MIN_VERSIONS => '
0', TTL => 'FOREVER', KEEP_DELETED_CELLS => 'FALSE', BLOCKSIZE => '65536', IN_ME
MORY => 'false', BLOCKCACHE => 'true'}
{NAME => 'studid', DATA_BLOCK_ENCODING => 'NONE', BLOOMFILTER => 'ROW', REPLICAT
ION_SCOPE => '0', VERSIONS => '1', COMPRESSION => 'NONE', MIN_VERSIONS => '0', T
TL => 'FOREVER', KEEP_DELETED_CELLS => 'FALSE', BLOCKSIZE => '65536', IN_MEMORY
=> 'false', BLOCKCACHE => 'true'}
{NAME => 'studname', DATA_BLOCK_ENCODING => 'NONE', BLOOMFILTER => 'ROW', REPLIC
ATION_SCOPE => '0', VERSIONS => '1', COMPRESSION => 'NONE', MIN_VERSIONS => '0',
TTL => 'FOREVER', KEEP_DELETED_CELLS => 'FALSE', BLOCKSIZE => '65536', IN_MEMOR
Y => 'false', BLOCKCACHE => 'true'}
3 row(s) in 0.1050 seconds
hbase(main):005:0> put 'studentrj','1','studname:fname','Uday'
Click to switch to "Workspace 2"
root@quickstart:/hom...

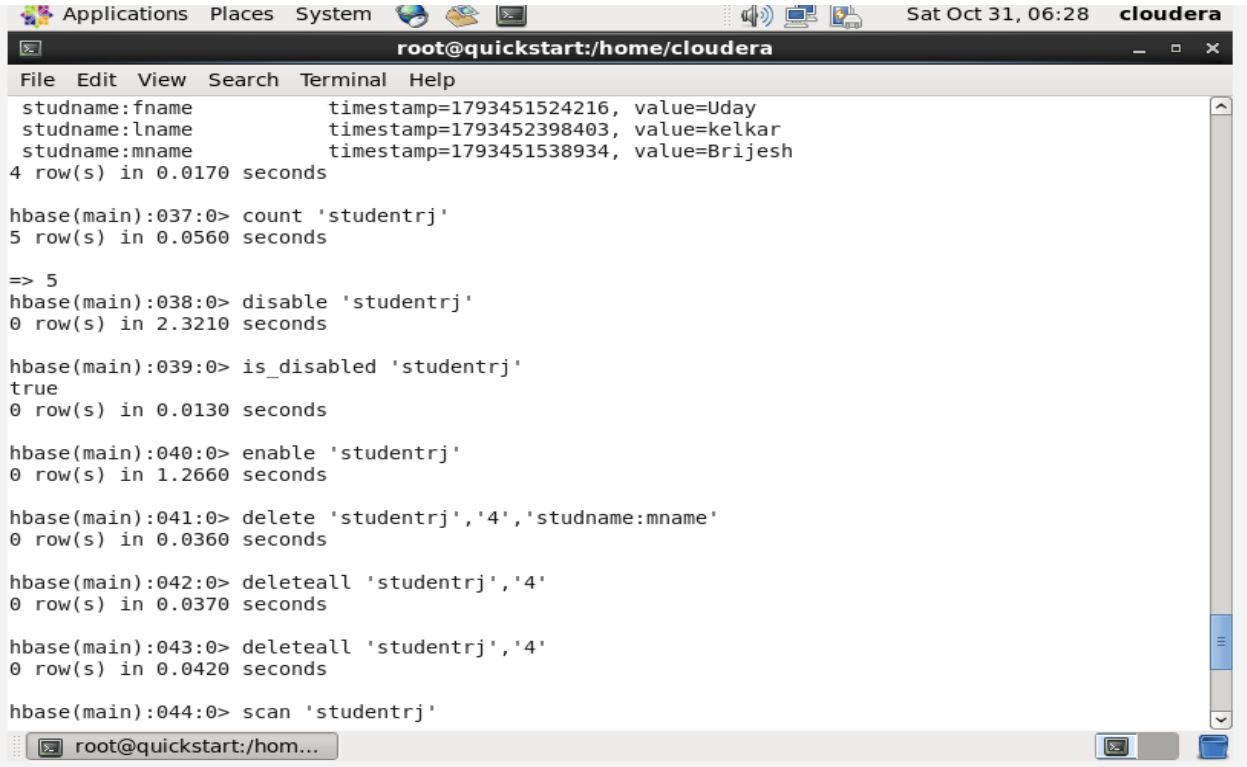

Applications Places System
Sat Oct 31, 06:23 cloudera
root@quickstart:/home/cloudera
File Edit View Search Terminal Help
hbase(main):006:0> put 'studentrj','1','studname:mname','Brijesh'
0 row(s) in 0.0200 seconds
hbase(main):007:0> put 'studentrj','1','studname:lname','Singh'
0 row(s) in 0.0220 seconds
hbase(main):008:0> put 'studentrj','1','studaddress:place','Dombivli'
0 row(s) in 0.0240 seconds
hbase(main):009:0> put 'studentrj','2','studname:fname','sumit'
0 row(s) in 0.0280 seconds
hbase(main):010:0> put 'studentrj','2','studname:mname','Neeraj'
0 row(s) in 0.0150 seconds
hbase(main):011:0> put 'studentrj','2','studname:lname','Kulkarni'
0 row(s) in 0.0230 seconds
hbase(main):012:0> put 'studentrj','2','studaddress:road','Devi Dayal road'
0 row(s) in 0.0480 seconds
hbase(main):013:0> put 'studentrj','2','studaddress:place','Mulund'
0 row(s) in 0.0240 seconds
hbase(main):014:0> put 'studentrj','3','studname:fname','Alka'
0 row(s) in 0.0150 seconds
hbase(main):015:0> put 'studentrj','3','studname:mname','Deepak'
root@quickstart:/hom...

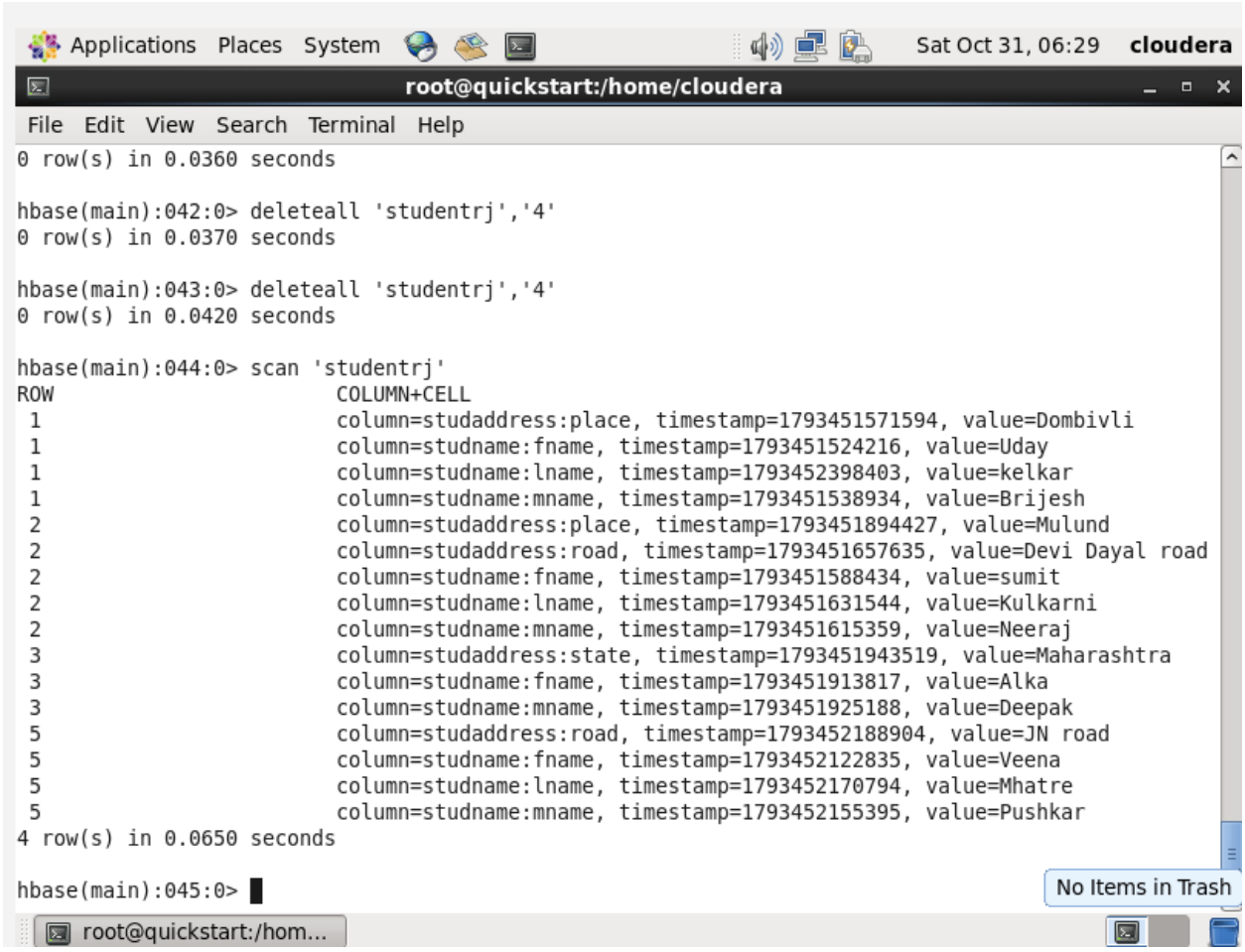

Applications Places System
Sat Oct 31, 06:24 cloudera
root@quickstart:/home/cloudera
File Edit View Search Terminal Help
0 row(s) in 0.0170 seconds
hbase(main):016:0> put 'studentrj','3','studaddress:state','Maharashtra'
0 row(s) in 0.0200 seconds
hbase(main):017:0> put 'studentrj','4','studname:fname','Amit'
0 row(s) in 0.0210 seconds
hbase(main):018:0> put 'studentrj','4','studname:mname','Neeraj'
0 row(s) in 0.0220 seconds
hbase(main):019:0> put 'studentrj','4','studname:lname','yadav'
0 row(s) in 0.0250 seconds
hbase(main):020:0> put 'studentrj','4','studaddress:road','Goshala Road'
0 row(s) in 0.0140 seconds
hbase(main):021:0> put 'studentrj','4','studaddress:place','Mulund W'
0 row(s) in 0.0090 seconds
hbase(main):022:0> put 'studentrj','5','studname:fname','Veena'put 'studentrj','5','studname:mnam
e','Pushkar'
SyntaxError: (hbase):22: syntax error, unexpected tIDENTIFIER
put 'studentrj','5','studname:fname','Veena'put 'studentrj','5','studname:mname','Pushkar'
^
hbase(main):023:0>
hbase(main):024:0* put 'studentrj','5','studname:fname','Veena'
root@quickstart:/hom...


Applications Places System
Sat Oct 31, 06:25 cloudera
root@quickstart:/home/cloudera
File Edit View Search Terminal Help
Contents F1
About
hbase(main):025:0> put 'studentr mname','Pushkar'
0 row(s) in 0.0150 seconds
hbase(main):026:0> put 'studentrj','5','studname:lname','Mhatre'
0 row(s) in 0.0360 seconds
hbase(main):027:0> put 'studentrj','5','studaddress:road','JN road'
0 row(s) in 0.0230 seconds
hbase(main):028:0> scan 'studentrj'
ROW COLUMN+CELL
1 column=studaddress:place, timestamp=1793451571594, value=Dombivli
1 column=studname:fname, timestamp=1793451524216, value=Uday
1 column=studname:lname, timestamp=1793451558154, value=Singh
1 column=studname:mname, timestamp=1793451538934, value=Brijesh
2 column=studaddress:place, timestamp=1793451894427, value=Mulund
2 column=studaddress:road, timestamp=1793451657635, value=Devi Dayal road
2 column=studname:fname, timestamp=1793451588434, value=sumit
2 column=studname:lname, timestamp=1793451631544, value=Kulkarni
2 column=studname:mname, timestamp=1793451615359, value=Neeraj
3 column=studaddress:state, timestamp=1793451943519, value=Maharashtra
3 column=studname:fname, timestamp=1793451913817, value=Alka
3 column=studname:mname, timestamp=1793451925188, value=Deepak
4 column=studaddress:place, timestamp=1793452040950, value=Mulund W
4 column=studaddress:road, timestamp=1793452022608, value=Goshala Road
4 column=studname:fname, timestamp=1793451956275, value=Amit
4 column=studname:lname, timestamp=1793452002480, value=yadav
4 column=studname:mname, timestamp=1793451967321, value=Neeraj
root@quickstart:/hom...


Applications Places System
Sat Oct 31, 06:26 cloudera
root@quickstart:/home/cloudera
File Edit View Search Terminal Help
5 column=studaddress:road, timestamp=1793452188904, value=JN road
5 column=studname:fname, timestamp=1793452122835, value=Veena
5 column=studname:lname, timestamp=1793452170794, value=Mhatre
5 column=studname:mname, timestamp=1793452155395, value=Pushkar
5 row(s) in 0.0720 seconds
hbase(main):029:0> get 'studentrj','1','studaddress:place'
COLUMN CELL
studaddress:place timestamp=1793451571594, value=Dombivli
1 row(s) in 0.0290 seconds
hbase(main):030:0> get 'studentrj','1'
COLUMN CELL
studaddress:place timestamp=1793451571594, value=Dombivli
studname:fname timestamp=1793451524216, value=Uday
studname:lname timestamp=1793451558154, value=Singh
studname:mname timestamp=1793451538934, value=Brijesh
4 row(s) in 0.0140 seconds
hbase(main):031:0> get 'studentrj','2'
COLUMN CELL
studaddress:place timestamp=1793451894427, value=Mulund
studaddress:road timestamp=1793451657635, value=Devi Dayal road
studname:fname timestamp=1793451588434, value=sumit
studname:lname timestamp=1793451631544, value=Kulkarni
studname:mname timestamp=1793451615359, value=Neeraj
5 row(s) in 0.0070 seconds
hbase(main):032:0> get 'studentrj','3'
root@quickstart:/hom...

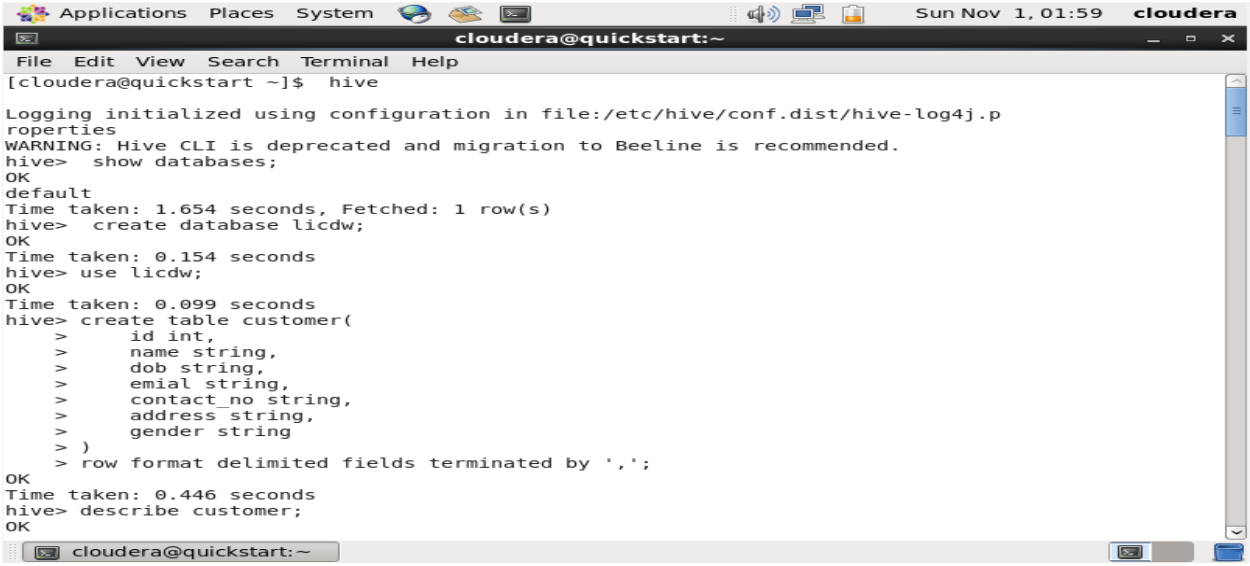

Applications Places System
Sat Oct 31, 06:27 cloudera
root@quickstart:/home/cloudera
File Edit View Search Terminal Help
COLUMN CELL
studaddress:state timestamp=1793451943519, value=Maharashtra
studname:fname timestamp=1793451913817, value=Alka
studname:mname timestamp=1793451925188, value=Deepak
3 row(s) in 0.0210 seconds
hbase(main):033:0> get 'studentrj','4'
COLUMN CELL
studaddress:place timestamp=1793452040950, value=Mulund W
studaddress:road timestamp=1793452022608, value=Goshala Road
studname:fname timestamp=1793451956275, value=Amit
studname:lname timestamp=1793452002480, value=yadav
studname:mname timestamp=1793451967321, value=Neeraj
5 row(s) in 0.0210 seconds
hbase(main):034:0> get 'studentrj','2','studaddress'
COLUMN CELL
studaddress:place timestamp=1793451894427, value=Mulund
studaddress:road timestamp=1793451657635, value=Devi Dayal road
2 row(s) in 0.0140 seconds
hbase(main):035:0> put 'studentrj','1','studname:lname','kelkar'
0 row(s) in 0.0160 seconds
hbase(main):036:0> get 'studentrj','1'
COLUMN CELL
studaddress:place timestamp=1793451571594, value=Dombivli
studname:fname timestamp=1793451524216, value=Uday
studname:lname timestamp=1793452398403, value=kelkar
root@quickstart:/hom...

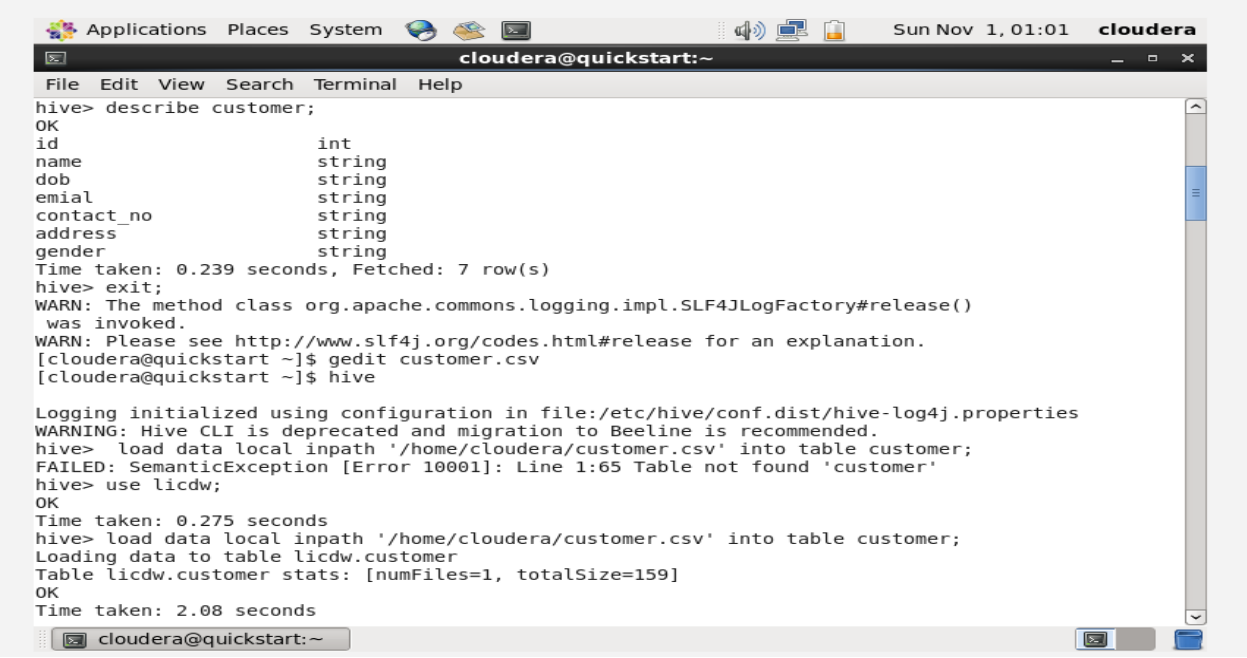

Applications Places System
Sat Oct 31, 06:28 cloudera
root@quickstart:/home/cloudera
File Edit View Search Terminal Help
studname:fname timestamp=1793451524216, value=Uday
studname:lname timestamp=1793452398403, value=kelkar
studname:mname timestamp=1793451538934, value=Brijesh
4 row(s) in 0.0170 seconds
hbase(main):037:0> count 'studentrj'
5 row(s) in 0.0560 seconds
=> 5
hbase(main):038:0> disable 'studentrj'
0 row(s) in 2.3210 seconds
hbase(main):039:0> is_disabled 'studentrj'
true
0 row(s) in 0.0130 seconds
hbase(main):040:0> enable 'studentrj'
0 row(s) in 1.2660 seconds
hbase(main):041:0> delete 'studentrj','4','studname:mname'
0 row(s) in 0.0360 seconds
hbase(main):042:0> deleteall 'studentrj','4'
0 row(s) in 0.0370 seconds
hbase(main):043:0> deleteall 'studentrj','4'
0 row(s) in 0.0420 seconds
hbase(main):044:0> scan 'studentrj'
root@quickstart:/hom...

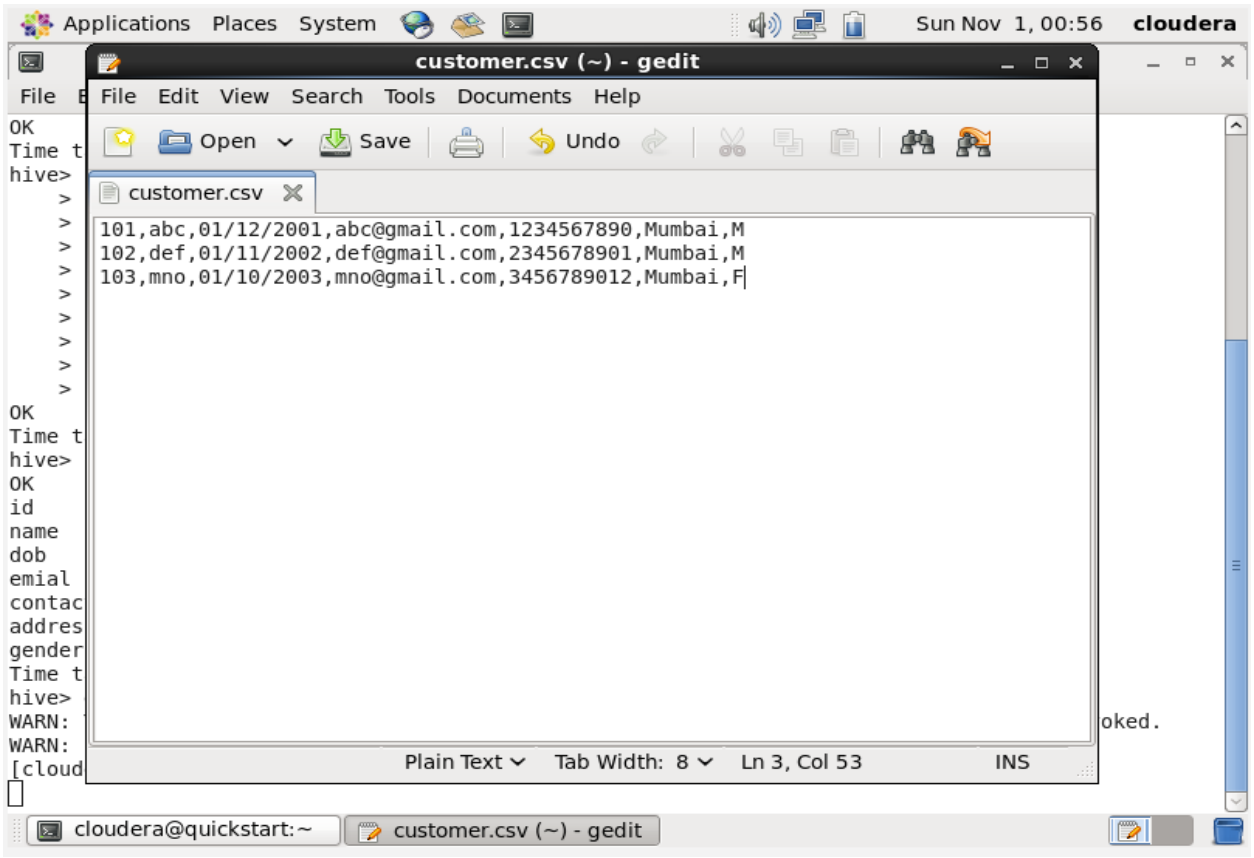

Applications Places System
Sat Oct 31, 06:29 cloudera
root@quickstart:/home/cloudera
File Edit View Search Terminal Help
0 row(s) in 0.0360 seconds
hbase(main):042:0> deleteall 'studentrj','4'
0 row(s) in 0.0370 seconds
hbase(main):043:0> deleteall 'studentrj','4'
0 row(s) in 0.0420 seconds
hbase(main):044:0> scan 'studentrj'
ROW COLUMN+CELL
1 column=studaddress:place, timestamp=1793451571594, value=Dombivli
1 column=studname:fname, timestamp=1793451524216, value=Uday
1 column=studname:lname, timestamp=1793452398403, value=kelkar
1 column=studname:mname, timestamp=1793451538934, value=Brijesh
2 column=studaddress:place, timestamp=1793451894427, value=Mulund
2 column=studaddress:road, timestamp=1793451657635, value=Devi Dayal road
2 column=studname:fname, timestamp=1793451588434, value=sumit
2 column=studname:lname, timestamp=1793451631544, value=Kulkarni
2 column=studname:mname, timestamp=1793451615359, value=Neeraj
3 column=studaddress:state, timestamp=1793451943519, value=Maharashtra
3 column=studname:fname, timestamp=1793451913817, value=Alka
3 column=studname:mname, timestamp=1793451925188, value=Deepak
5 column=studaddress:road, timestamp=1793452188904, value=JN road
5 column=studname:fname, timestamp=1793452122835, value=Veena
5 column=studname:lname, timestamp=1793452170794, value=Mhatre
5 column=studname:mname, timestamp=1793452155395, value=Pushkar
4 row(s) in 0.0650 seconds
hbase(main):045:0>
No Items in Trash
root@quickstart:/hom...

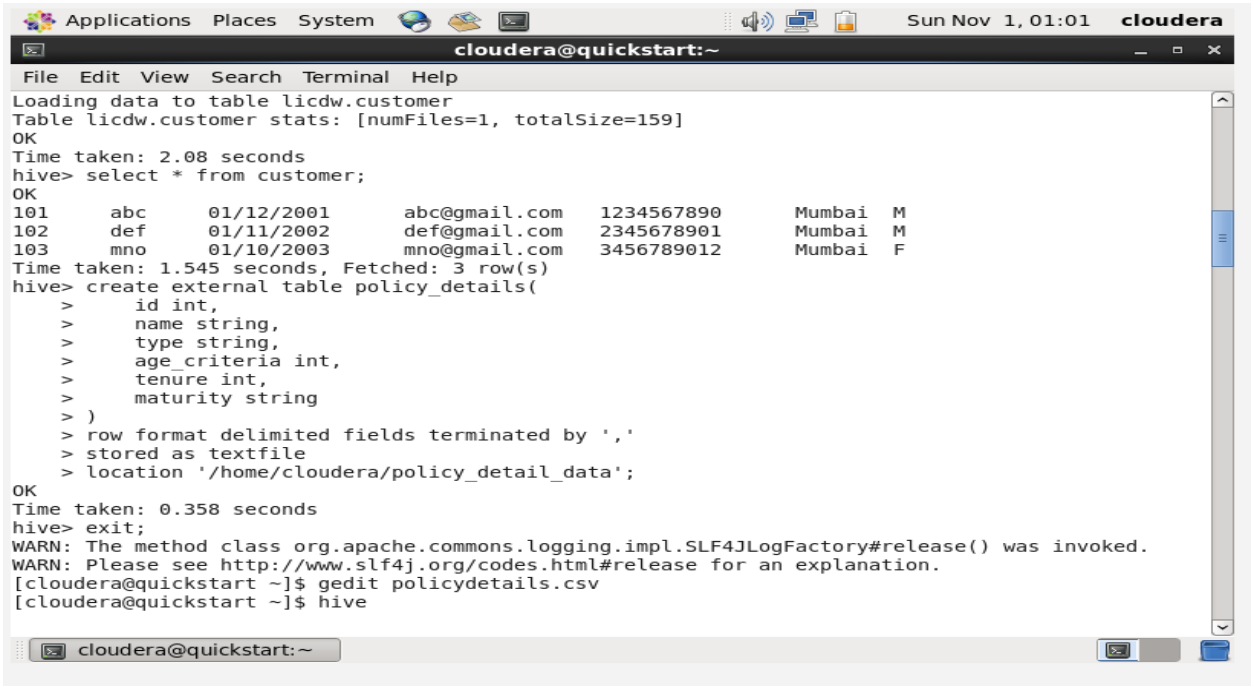
Hive Code:

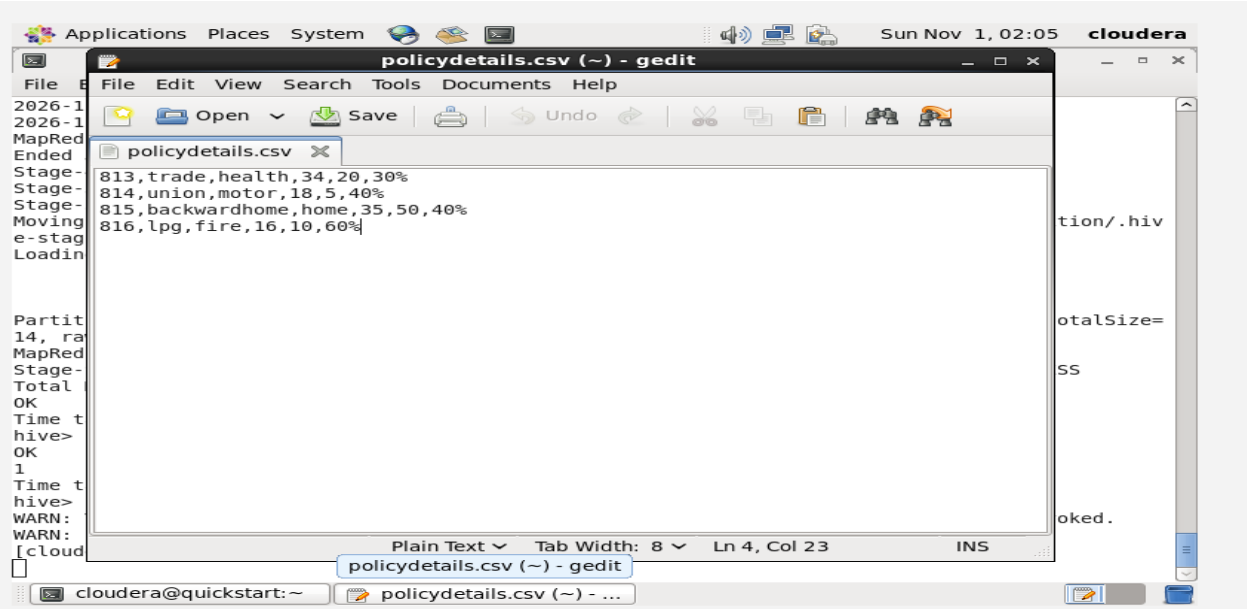
A) Aim: Working with different types of tables in Hive.

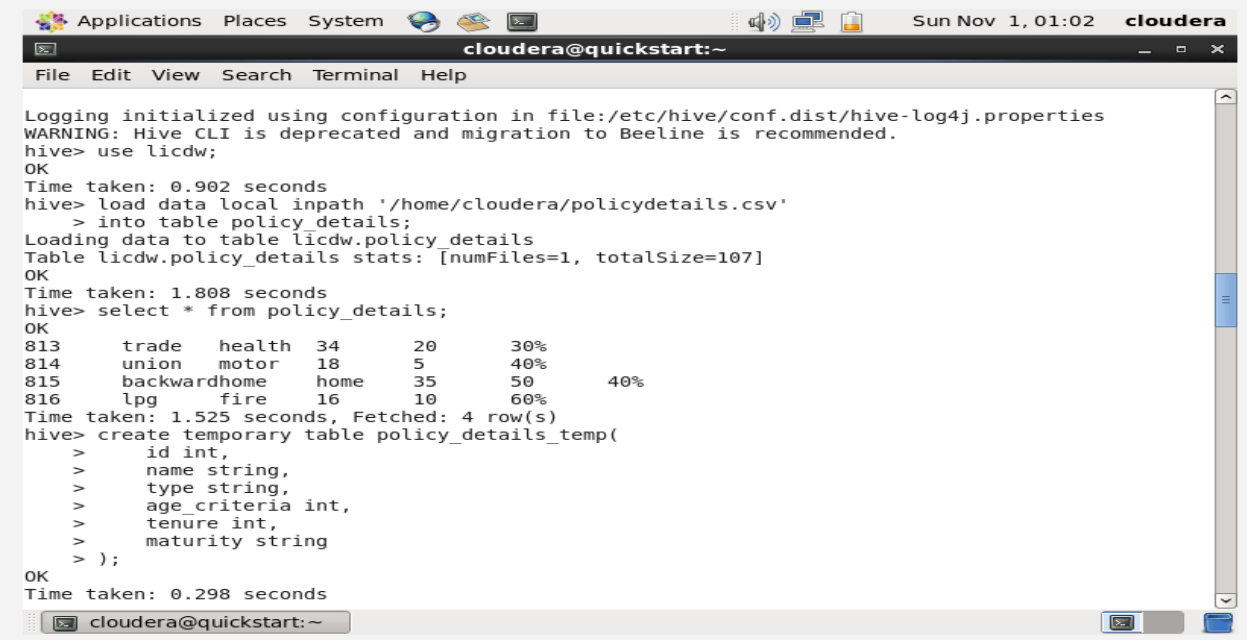


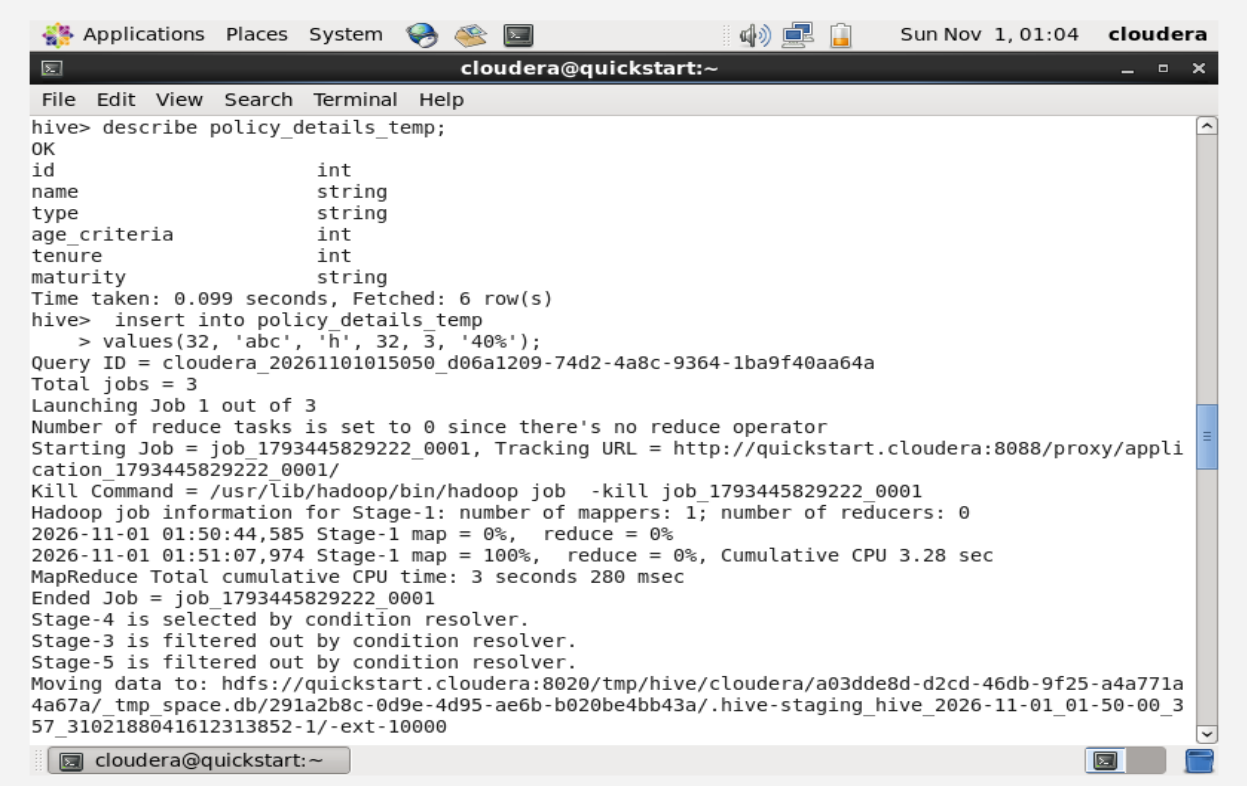


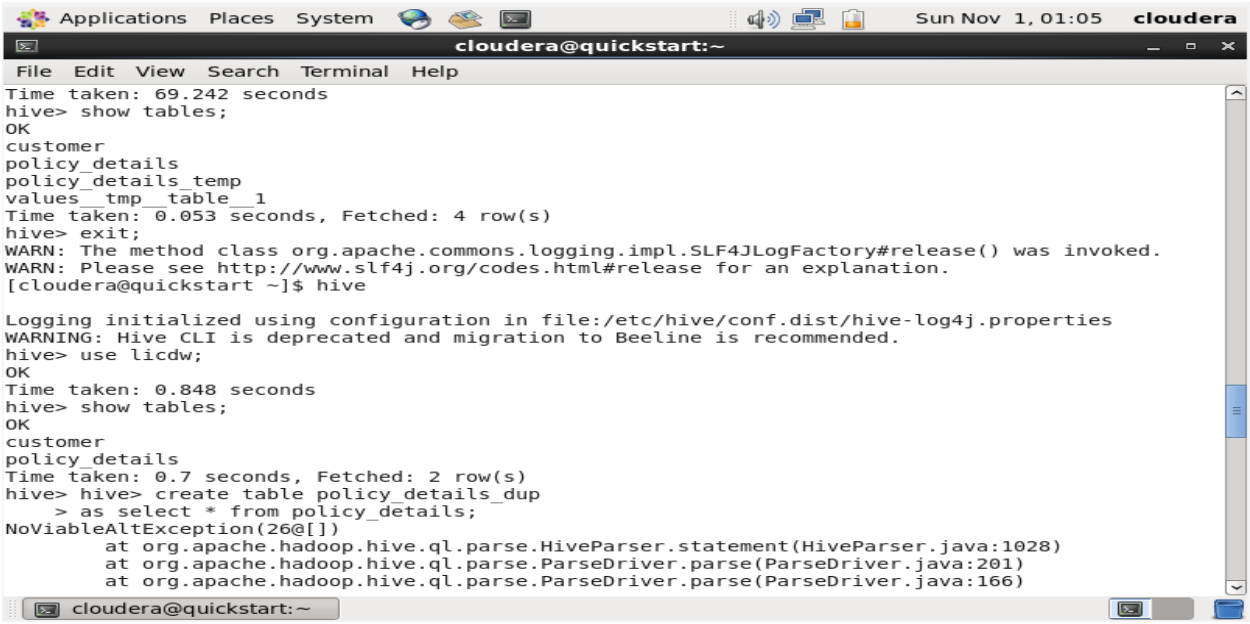

Applications Places System
Sun Nov 1, 00:56 cloudera
customer.csv (~) - gedit
File Edit View Search Tools Documents Help
Open Save Undo
customer.csv
101,abc,01/12/2001,abc@gmail.com,1234567890,Mumbai,M
102,def,01/11/2002,def@gmail.com,2345678901,Mumbai,M
103,mno,01/10/2003,mno@gmail.com,3456789012,Mumbai,F
Plain Text Tab Width: 8 Ln 3, Col 53 INS
cloudera@quickstart:~
customer.csv (~) - gedit

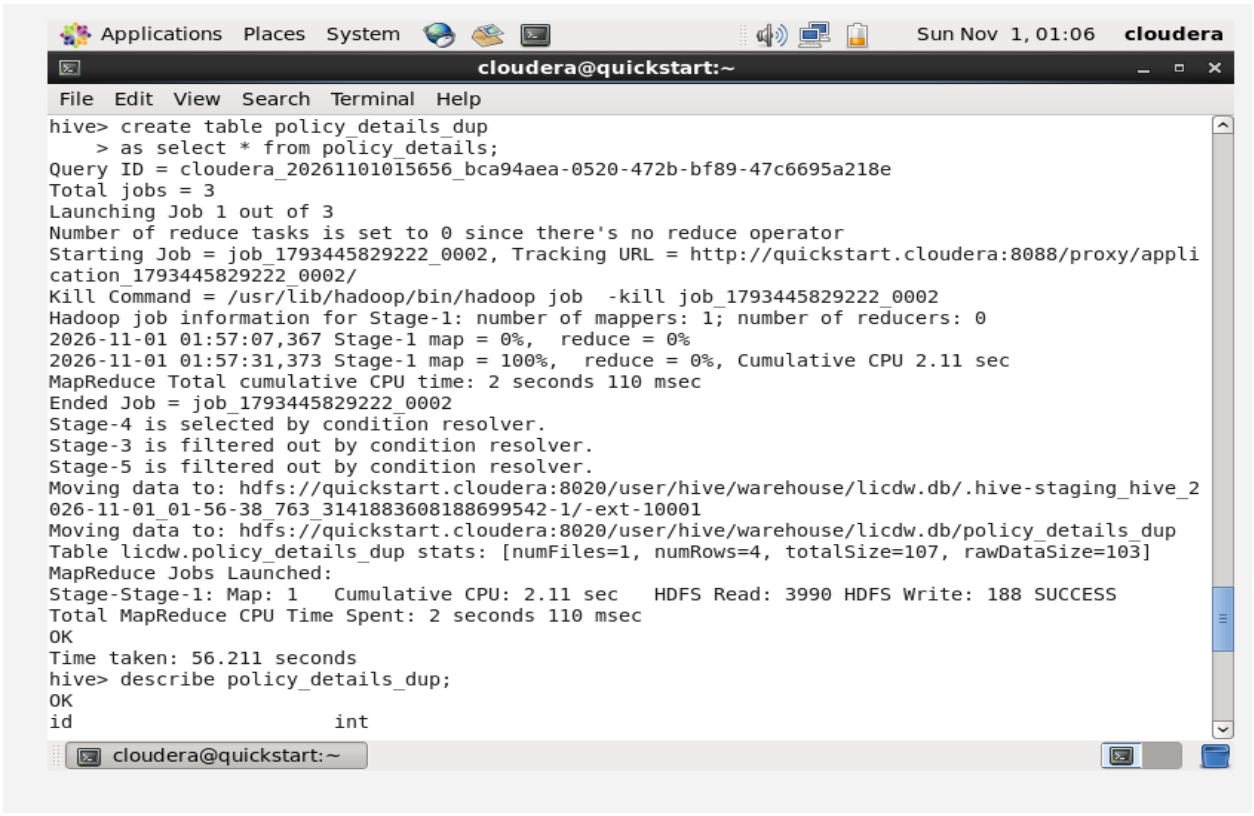

Applications Places System
Sun Nov 1, 01:01 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
Loading data to table licdw.customer
Table licdw.customer stats: [numFiles=1, totalSize=159]
OK
Time taken: 2.08 seconds
hive> select * from customer;
OK
101 abc 01/12/2001 abc@gmail.com 1234567890 Mumbai M
102 def 01/11/2002 def@gmail.com 2345678901 Mumbai M
103 mno 01/10/2003 mno@gmail.com 3456789012 Mumbai F
Time taken: 1.545 seconds, Fetched: 3 row(s)
hive> create external table policy_details(
> id int,
> name string,
> type string,
> age_criteria int,
> tenure int,
> maturity string
>)
> row format delimited fields terminated by ','
> stored as textfile
> location '/home/cloudera/policy_detail_data';
OK
Time taken: 0.358 seconds
hive> exit;
WARN: The method class org.apache.commons.logging.impl.SLF4JLogFactory#release() was invoked.
WARN: Please see http://www.slf4j.org/codes.html#release for an explanation.
[cloudera@quickstart ~]$ gedit policydetails.csv
[cloudera@quickstart ~]$ hive
cloudera@quickstart:~

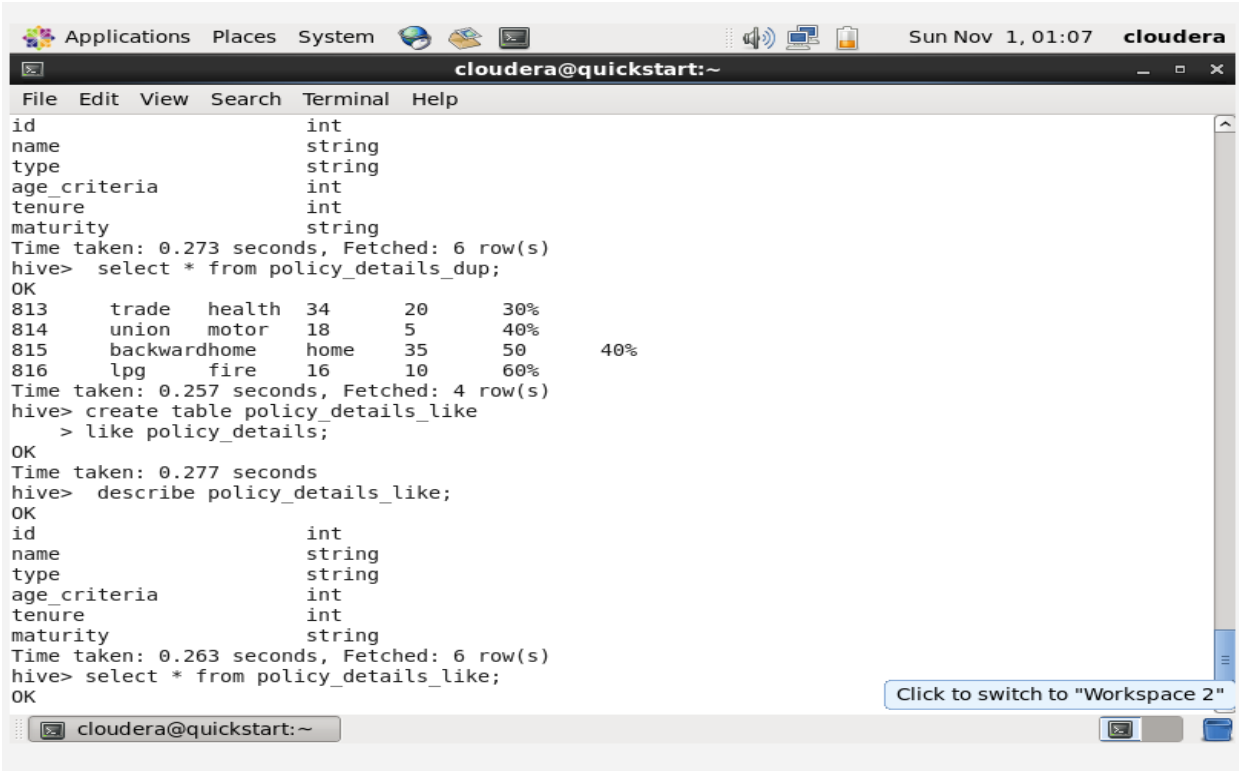

Applications Places System
Sun Nov 1, 02:05 cloudera
policydetails.csv (~) - gedit
File Edit View Search Tools Documents Help
Open Save Undo
policydetails.csv
813,trade,health,34,20,30%
814,union,motor,18,5,40%
815,backwardhome,home,35,50,40%
816,lpg,fire,16,10,60%
Plain Text Tab Width: 8 Ln 4, Col 23 INS
policydetails.csv (~) - gedit
cloudera@quickstart:~
policydetails.csv (~) - ...


Applications Places System
Sun Nov 1, 01:02 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> use licdw;
OK
Time taken: 0.902 seconds
hive> load data local inpath '/home/cloudera/policydetails.csv'
> into table policy_details;
Loading data to table licdw.policy_details
Table licdw.policy_details stats: [numFiles=1, totalSize=107]
OK
Time taken: 1.808 seconds
hive> select * from policy_details;
OK
813 trade health 34 20 30%
814 union motor 18 5 40%
815 backwardhome home 35 50 40%
816 lpg fire 16 10 60%
Time taken: 1.525 seconds, Fetched: 4 row(s)
hive> create temporary table policy_details_temp(
> id int,
> name string,
> type string,
> age_criteria int,
> tenure int,
> maturity string
>);
OK
Time taken: 0.298 seconds
cloudera@quickstart:~

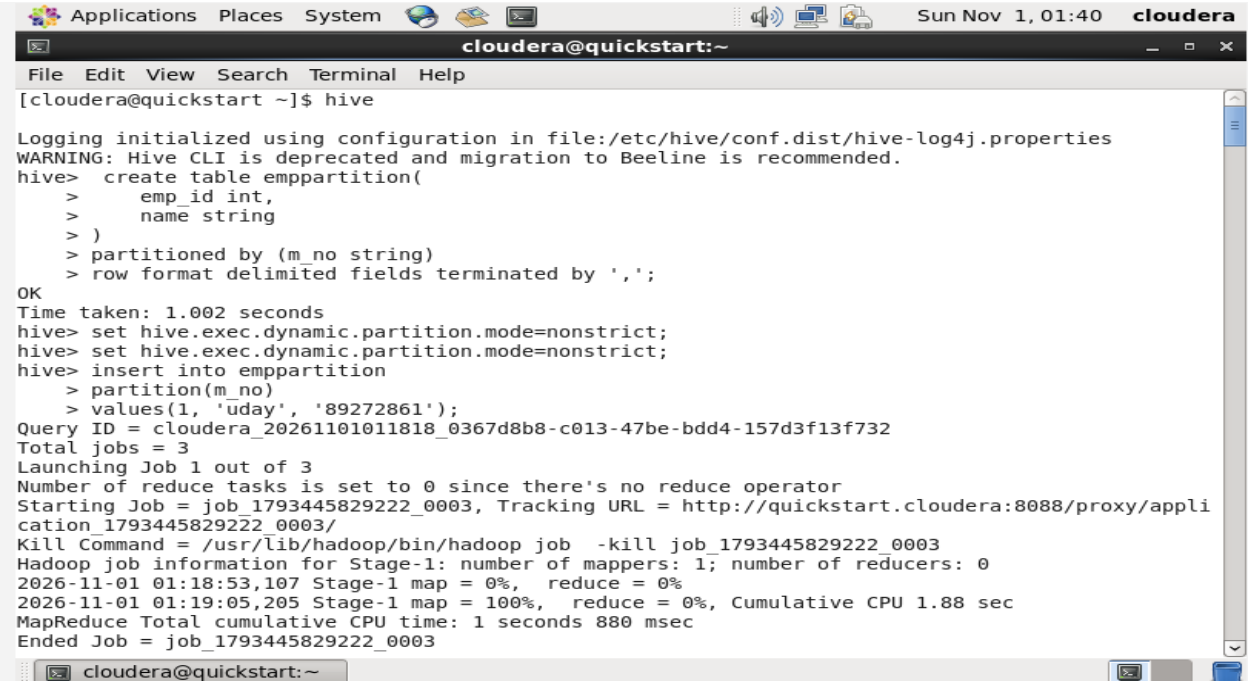

Applications Places System
Sun Nov 1, 01:04 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
hive> describe policy_details_temp;
OK
id int
name string
type string
age_criteria int
tenure int
maturity string
Time taken: 0.099 seconds, Fetched: 6 row(s)
hive> insert into policy_details_temp
> values(32, 'abc', 'h', 32, 3, '40%');
Query ID = cloudera_20261101015050_d06a1209-74d2-4a8c-9364-1ba9f40aa64a
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1793445829222_0001, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1793445829222_0001/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1793445829222_0001
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-11-01 01:50:44,585 Stage-1 map = 0%, reduce = 0%
2026-11-01 01:51:07,974 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 3.28 sec
MapReduce Total cumulative CPU time: 3 seconds 280 msec
Ended Job = job_1793445829222_0001
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/tmp/hive/cloudera/a03dde8d-d2cd-46db-9f25-a4a771a4a67a/_tmp_space.db/291a2b8c-0d9e-4d95-ae6b-b020be4bb43a/.hive-staging_hive_2026-11-01_01-50-00_357_3102188041612313852-1/-ext-10000
cloudera@quickstart:~

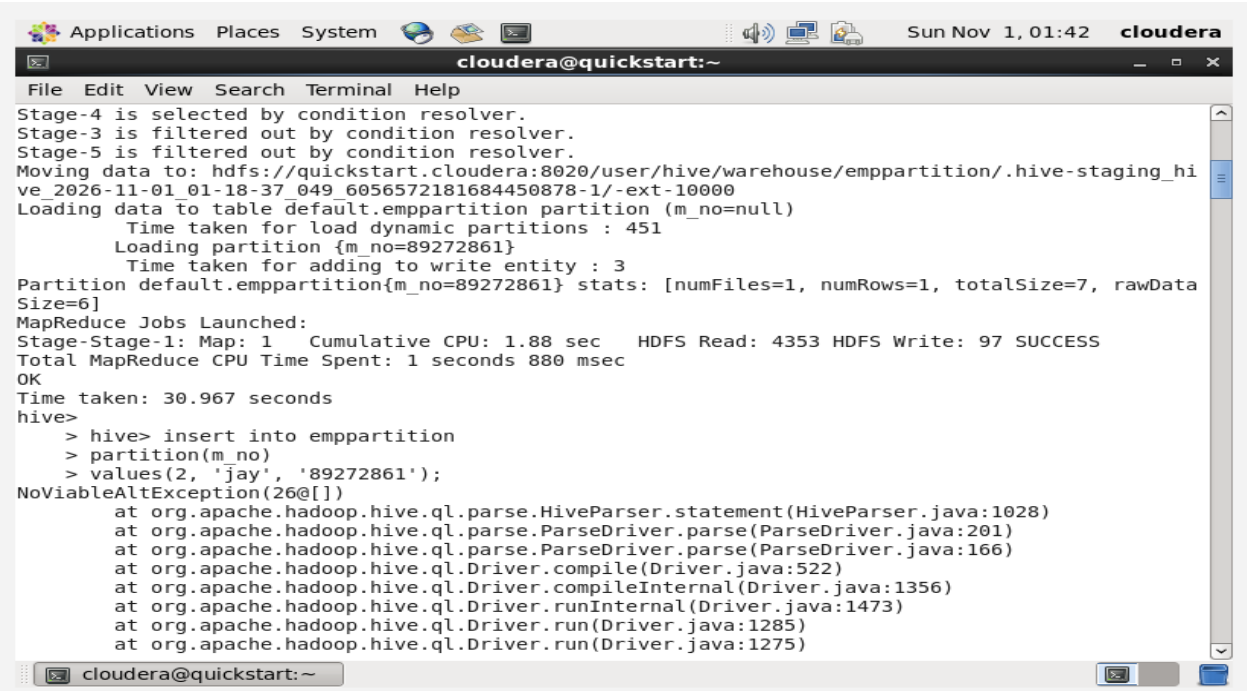

Applications Places System
Sun Nov 1, 01:05 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
Time taken: 69.242 seconds
hive> show tables;
OK
customer
policy_details
policy_details_temp
values__tmp__table__1
Time taken: 0.053 seconds, Fetched: 4 row(s)
hive> exit;
WARN: The method class org.apache.commons.logging.impl.SLF4JLogFactory#release() was invoked.
WARN: Please see http://www.slf4j.org/codes.html#release for an explanation.
[cloudera@quickstart ~]$ hive
Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> use licdw;
OK
Time taken: 0.848 seconds
hive> show tables;
OK
customer
policy_details
Time taken: 0.7 seconds, Fetched: 2 row(s)
hive> hive> create table policy_details_dup
> as select * from policy_details;
NoViableAltException(26@[])
at org.apache.hadoop.hive.ql.parse.HiveParser.statement(HiveParser.java:1028)
at org.apache.hadoop.hive.ql.parse.ParseDriver.parse(ParseDriver.java:201)
at org.apache.hadoop.hive.ql.parse.ParseDriver.parse(ParseDriver.java:166)
cloudera@quickstart:~

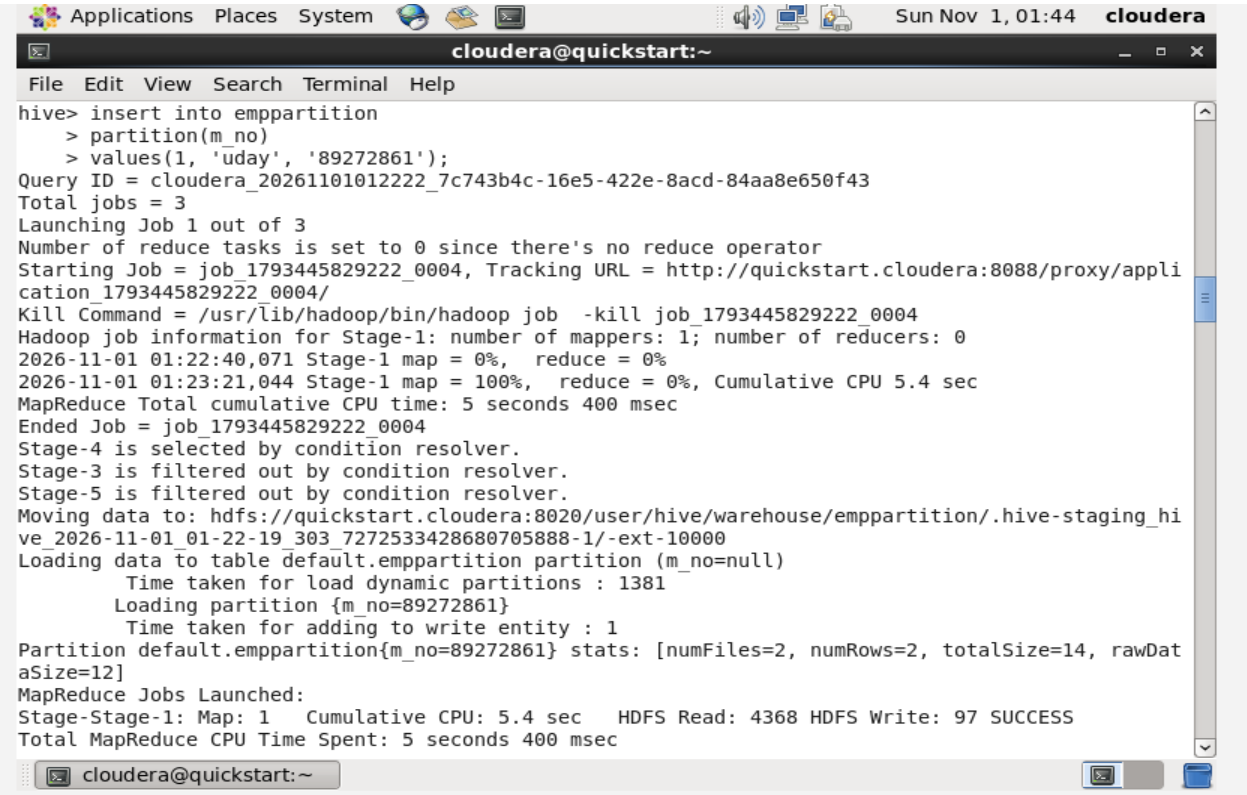

Applications Places System
Sun Nov 1, 01:06 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
hive> create table policy_details_dup
> as select * from policy_details;
Query ID = cloudera_20261101015656_bca94aea-0520-472b-bf89-47c6695a218e
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1793445829222_0002, Tracking URL = http://quickstart.cloudera:8088/proxy/appli
cation_1793445829222_0002/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1793445829222_0002
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-11-01 01:57:07,367 Stage-1 map = 0%, reduce = 0%
2026-11-01 01:57:31,373 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.11 sec
MapReduce Total cumulative CPU time: 2 seconds 110 msec
Ended Job = job_1793445829222_0002
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/licdw.db/.hive-staging_hive_2
026-11-01_01-56-38_763_3141883608188699542-1/-ext-10001
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/licdw.db/policy_details_dup
Table licdw.policy_details_dup stats: [numFiles=1, numRows=4, totalSize=107, rawDataSize=103]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 2.11 sec HDFS Read: 3990 HDFS Write: 188 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 110 msec
OK
Time taken: 56.211 seconds
hive> describe policy_details_dup;
OK
id int
cloudera@quickstart:~

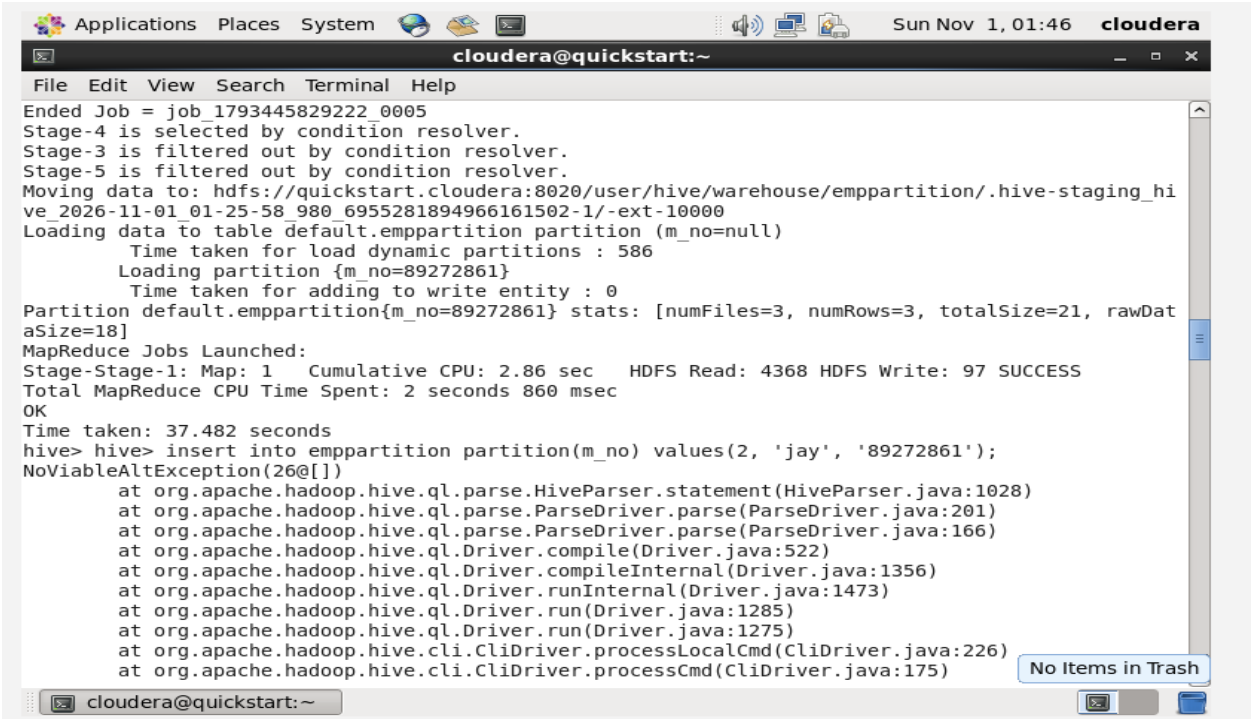

Applications Places System
Sun Nov 1, 01:07 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
id int
name string
type string
age_criteria int
tenure int
maturity string
Time taken: 0.273 seconds, Fetched: 6 row(s)
hive> select * from policy_details_dup;
OK
813 trade health 34 20 30%
814 union motor 18 5 40%
815 backwardhome home 35 50 40%
816 lpg fire 16 10 60%
Time taken: 0.257 seconds, Fetched: 4 row(s)
hive> create table policy_details_like
> like policy_details;
OK
Time taken: 0.277 seconds
hive> describe policy_details_like;
OK
id int
name string
type string
age_criteria int
tenure int
maturity string
Time taken: 0.263 seconds, Fetched: 6 row(s)
hive> select * from policy_details_like;
OK
Click to switch to "Workspace 2"
cloudera@quickstart:~

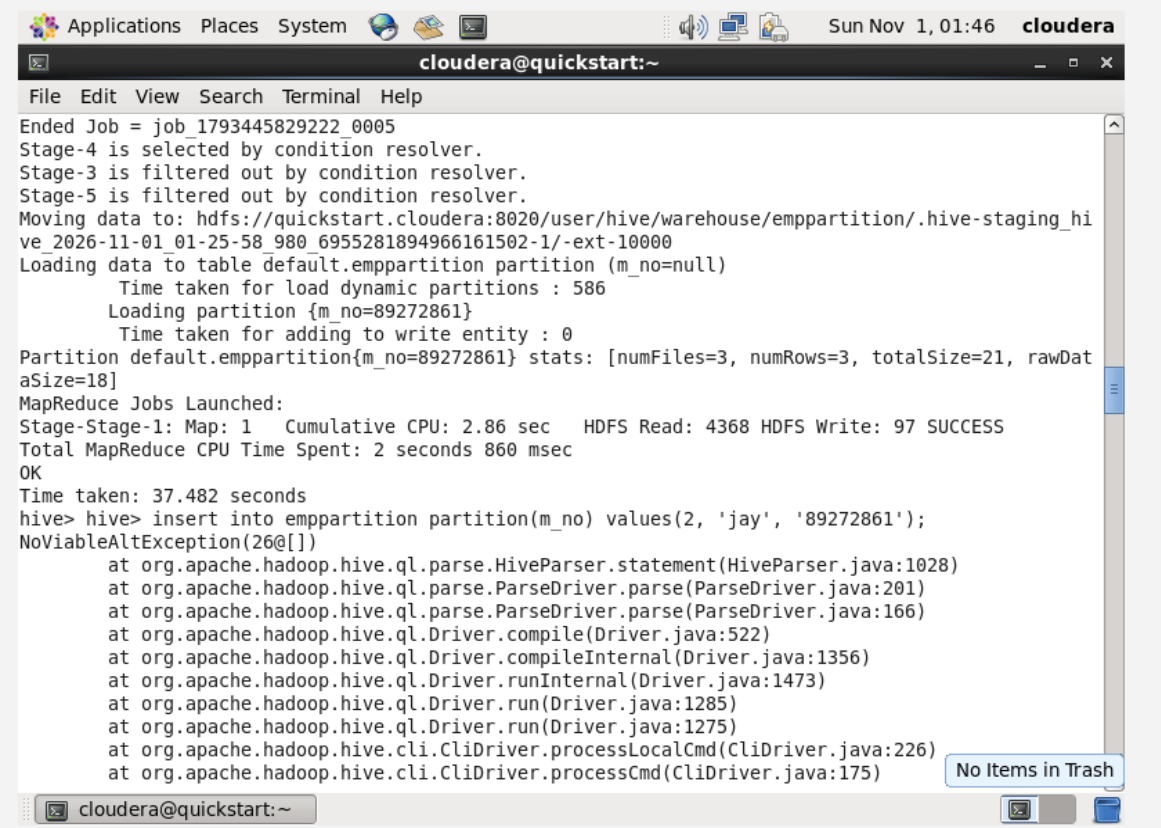
B) Aim: Demonstrating table partitioning, clustering (bucketing in Hive).

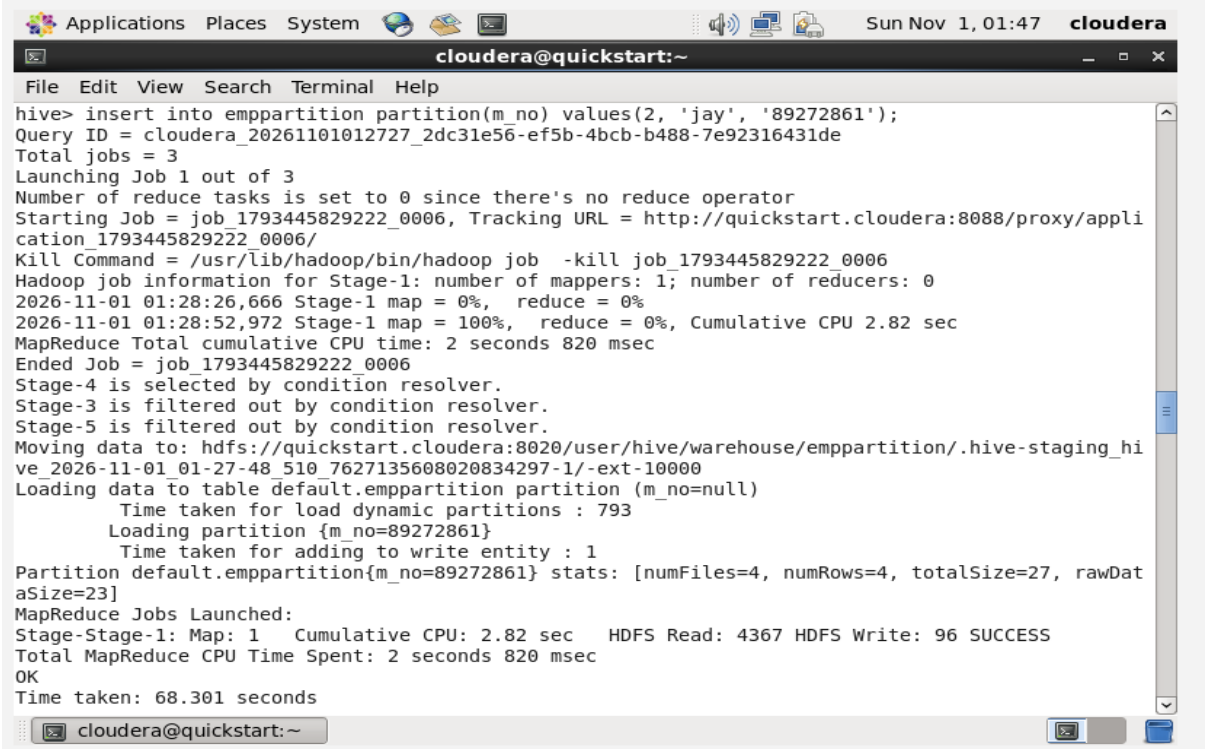


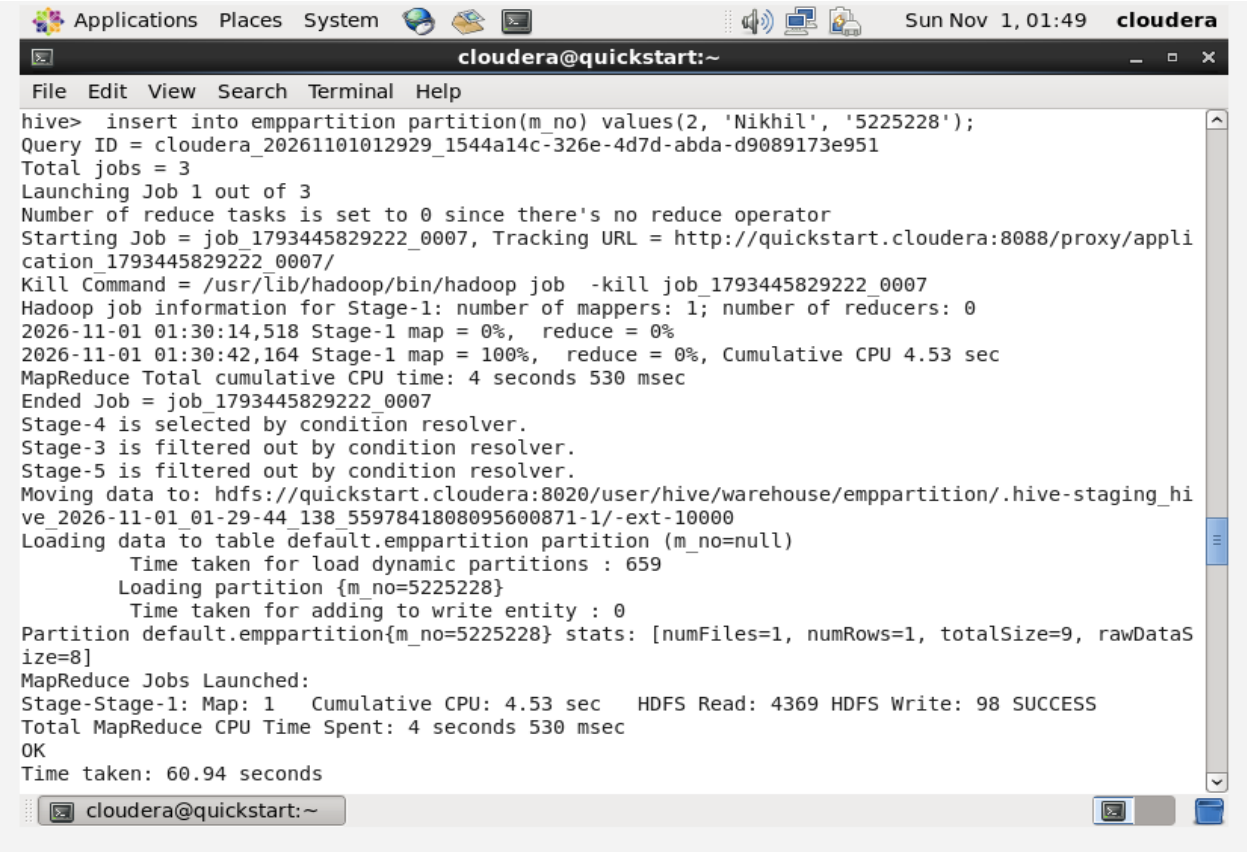


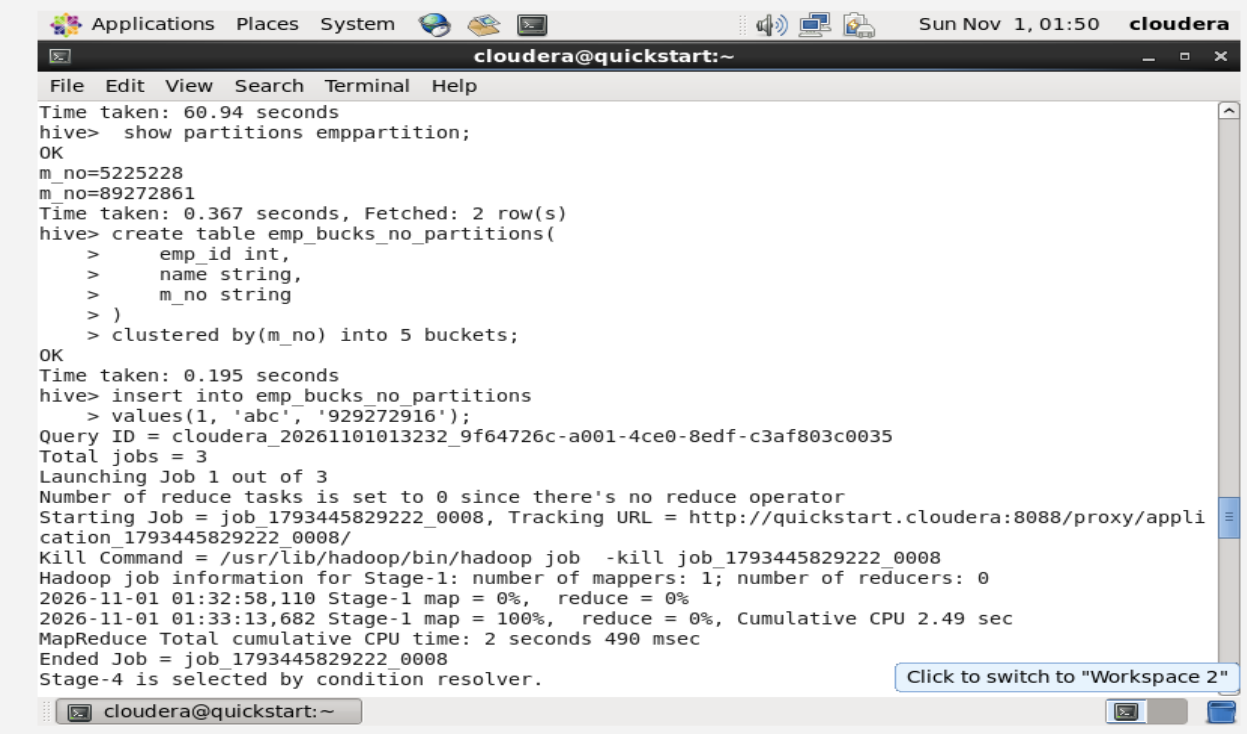

Applications Places System
Sun Nov 1, 01:44 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
hive> insert into emppartition
> partition(m_no)
> values(1, 'uday', '89272861');
Query ID = cloudera_20261101012222_7c743b4c-16e5-422e-8acd-84aa8e650f43
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1793445829222_0004, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1793445829222_0004/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1793445829222_0004
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-11-01 01:22:40,071 Stage-1 map = 0%, reduce = 0%
2026-11-01 01:23:21,044 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 5.4 sec
MapReduce Total cumulative CPU time: 5 seconds 400 msec
Ended Job = job_1793445829222_0004
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/emppartition/.hive-staging_hive_2026-11-01_01-22-19_303_7272533428680705888-1/-ext-10000
Loading data to table default.emppartition partition (m_no=null)
Time taken for load dynamic partitions : 1381
Loading partition {m_no=89272861}
Time taken for adding to write entity : 1
Partition default.emppartition{m_no=89272861} stats: [numFiles=2, numRows=2, totalSize=14, rawDataSize=12]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 5.4 sec HDFS Read: 4368 HDFS Write: 97 SUCCESS
Total MapReduce CPU Time Spent: 5 seconds 400 msec
cloudera@quickstart:~

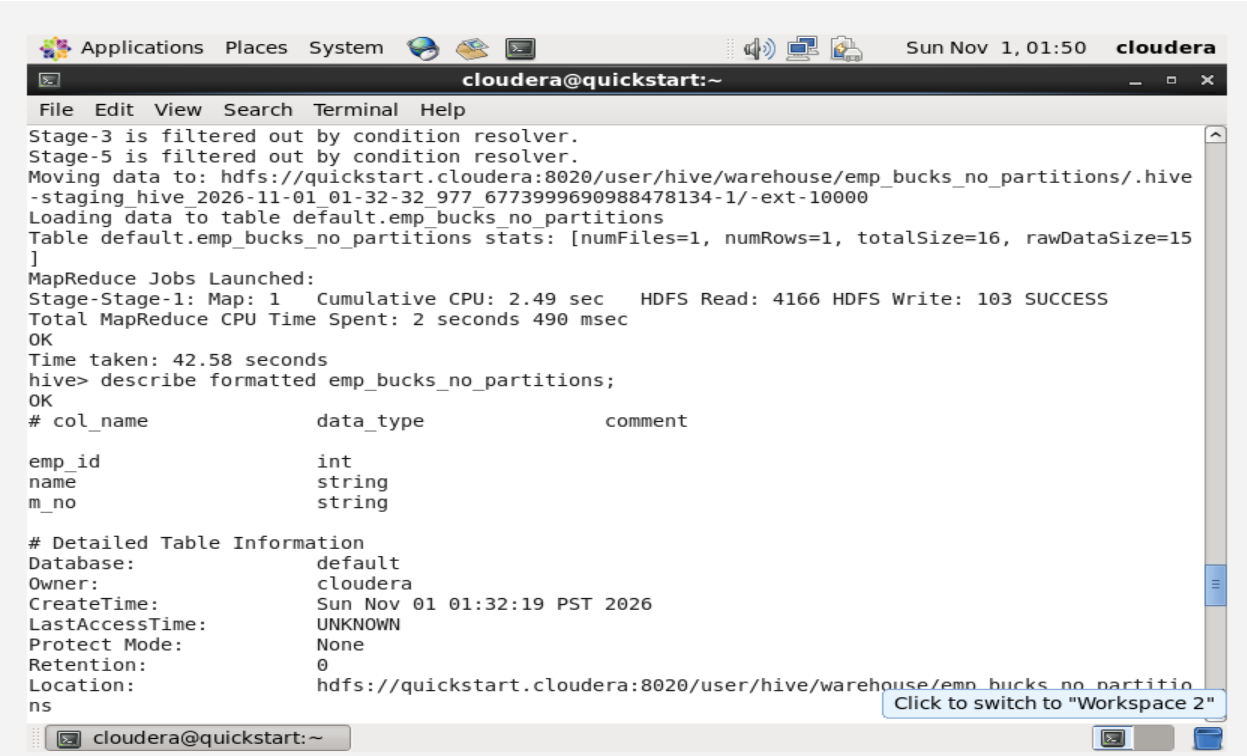

Applications Places System
Sun Nov 1, 01:46 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
Ended Job = job_1793445829222_0005
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/emppartition/.hive-staging_hive_2026-11-01_01-25-58_980_6955281894966161502-1/-ext-10000
Loading data to table default.emppartition partition (m_no=null)
Time taken for load dynamic partitions : 586
Loading partition {m_no=89272861}
Time taken for adding to write entity : 0
Partition default.emppartition{m_no=89272861} stats: [numFiles=3, numRows=3, totalSize=21, rawDataSize=18]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 2.86 sec HDFS Read: 4368 HDFS Write: 97 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 860 msec
OK
Time taken: 37.482 seconds
hive> hive> insert into emppartition partition(m_no) values(2, 'jay', '89272861');
NoViableAltException(26@[])
at org.apache.hadoop.hive.ql.parse.HiveParser.statement(HiveParser.java:1028)
at org.apache.hadoop.hive.ql.parse.ParseDriver.parse(ParseDriver.java:201)
at org.apache.hadoop.hive.ql.parse.ParseDriver.parse(ParseDriver.java:166)
at org.apache.hadoop.hive.ql.Driver.compile(Driver.java:522)
at org.apache.hadoop.hive.ql.Driver.compileInternal(Driver.java:1356)
at org.apache.hadoop.hive.ql.Driver.runInternal(Driver.java:1473)
at org.apache.hadoop.hive.ql.Driver.run(Driver.java:1285)
at org.apache.hadoop.hive.ql.Driver.run(Driver.java:1275)
at org.apache.hadoop.hive.cli.CliDriver.processLocalCmd(CliDriver.java:226)
at org.apache.hadoop.hive.cli.CliDriver.processCmd(CliDriver.java:175)
No Items in Trash
cloudera@quickstart:~

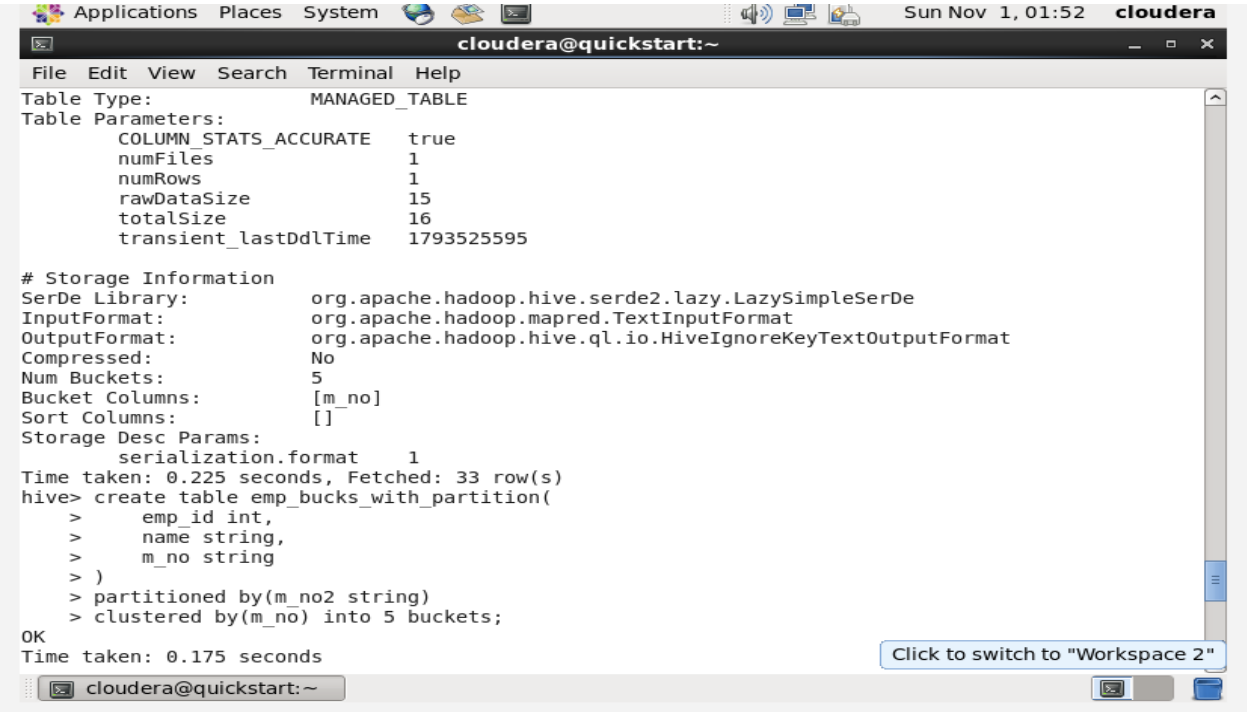

Applications Places System
Sun Nov 1, 01:46 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
Ended Job = job_1793445829222_0005
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/emppartition/.hive-staging_hive_2026-11-01_01-25-58_980_6955281894966161502-1/-ext-10000
Loading data to table default.emppartition partition (m_no=null)
Time taken for load dynamic partitions : 586
Loading partition {m_no=89272861}
Time taken for adding to write entity : 0
Partition default.emppartition{m_no=89272861} stats: [numFiles=3, numRows=3, totalSize=21, rawDataSize=18]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 2.86 sec HDFS Read: 4368 HDFS Write: 97 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 860 msec
OK
Time taken: 37.482 seconds
hive> hive> insert into emppartition partition(m_no) values(2, 'jay', '89272861');
NoViableAltException(26@[])
at org.apache.hadoop.hive.ql.parse.HiveParser.statement(HiveParser.java:1028)
at org.apache.hadoop.hive.ql.parse.ParseDriver.parse(ParseDriver.java:201)
at org.apache.hadoop.hive.ql.parse.ParseDriver.parse(ParseDriver.java:166)
at org.apache.hadoop.hive.ql.Driver.compile(Driver.java:522)
at org.apache.hadoop.hive.ql.Driver.compileInternal(Driver.java:1356)
at org.apache.hadoop.hive.ql.Driver.runInternal(Driver.java:1473)
at org.apache.hadoop.hive.ql.Driver.run(Driver.java:1285)
at org.apache.hadoop.hive.ql.Driver.run(Driver.java:1275)
at org.apache.hadoop.hive.cli.CliDriver.processLocalCmd(CliDriver.java:226)
at org.apache.hadoop.hive.cli.CliDriver.processCmd(CliDriver.java:175)
No Items in Trash
cloudera@quickstart:~

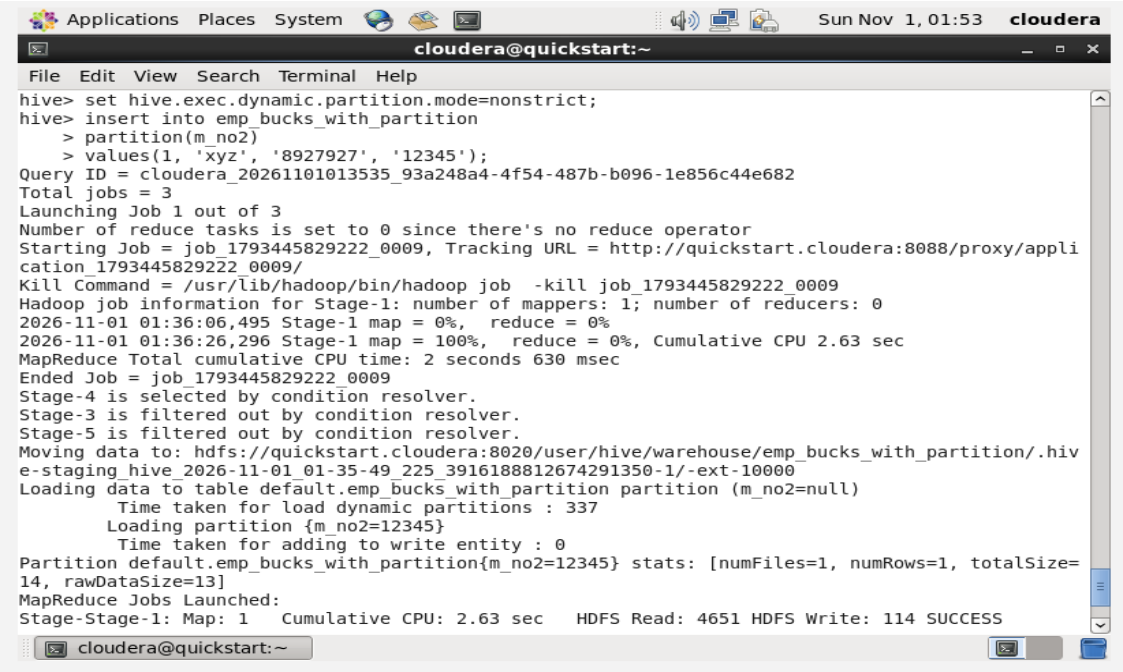

Applications Places System
Sun Nov 1, 01:47 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
hive> insert into emppartition partition(m_no) values(2, 'jay', '89272861');
Query ID = cloudera_20261101012727_2dc31e56-ef5b-4bcb-b488-7e92316431de
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1793445829222_0006, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1793445829222_0006/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1793445829222_0006
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-11-01 01:28:26,666 Stage-1 map = 0%, reduce = 0%
2026-11-01 01:28:52,972 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.82 sec
MapReduce Total cumulative CPU time: 2 seconds 820 msec
Ended Job = job_1793445829222_0006
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/emppartition/.hive-staging_hive_2026-11-01_01-27-48_510_7627135608020834297-1/-ext-10000
Loading data to table default.emppartition partition (m_no=null)
Time taken for load dynamic partitions : 793
Loading partition {m_no=89272861}
Time taken for adding to write entity : 1
Partition default.emppartition{m_no=89272861} stats: [numFiles=4, numRows=4, totalSize=27, rawDataSize=23]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 2.82 sec HDFS Read: 4367 HDFS Write: 96 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 820 msec
OK
Time taken: 68.301 seconds
cloudera@quickstart:~

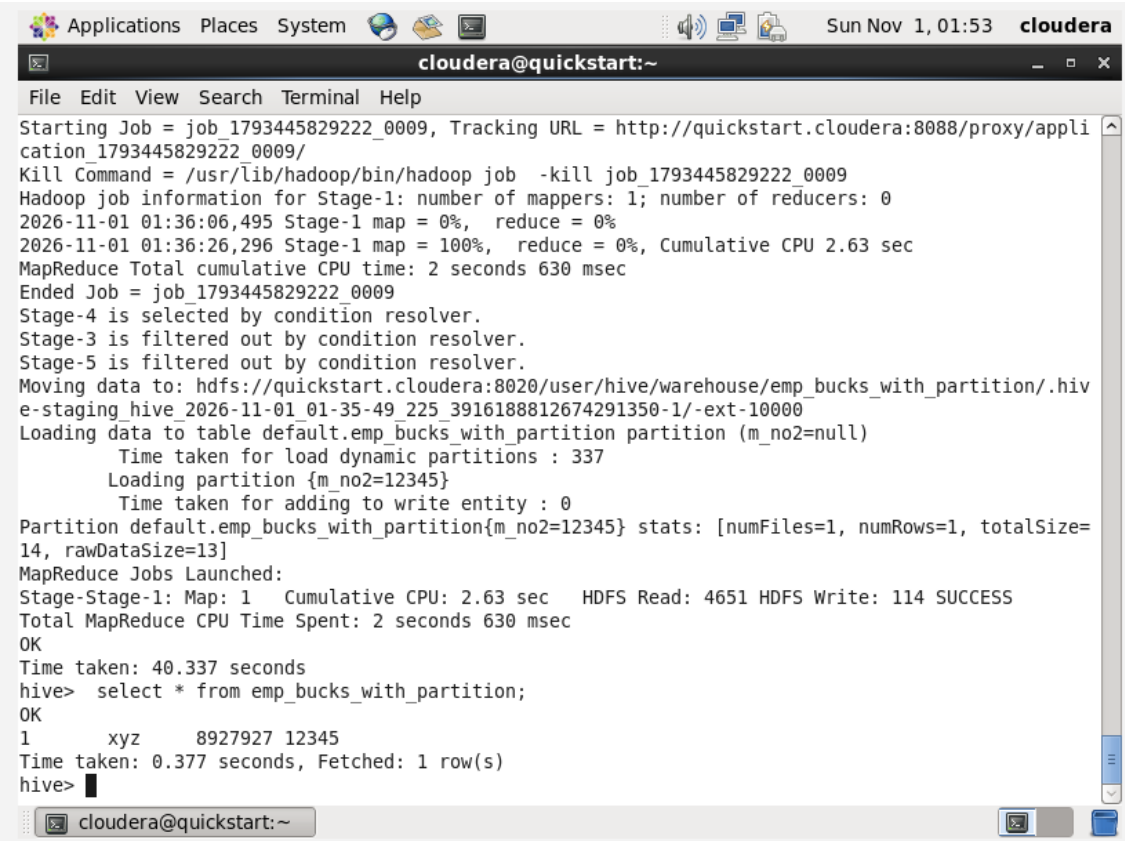

Applications Places System
Sun Nov 1, 01:49 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
hive> insert into emppartition partition(m_no) values(2, 'Nikhil', '5225228');
Query ID = cloudera_20261101012929_1544a14c-326e-4d7d-abda-d9089173e951
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1793445829222_0007, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1793445829222_0007/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1793445829222_0007
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-11-01 01:30:14,518 Stage-1 map = 0%, reduce = 0%
2026-11-01 01:30:42,164 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 4.53 sec
MapReduce Total cumulative CPU time: 4 seconds 530 msec
Ended Job = job_1793445829222_0007
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/emppartition/.hive-staging_hive_2026-11-01_01-29-44_138_5597841808095600871-1/-ext-10000
Loading data to table default.emppartition partition (m_no=null)
Time taken for load dynamic partitions : 659
Loading partition {m_no=5225228}
Time taken for adding to write entity : 0
Partition default.emppartition{m_no=5225228} stats: [numFiles=1, numRows=1, totalSize=9, rawDataSize=8]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 4.53 sec HDFS Read: 4369 HDFS Write: 98 SUCCESS
Total MapReduce CPU Time Spent: 4 seconds 530 msec
OK
Time taken: 60.94 seconds
cloudera@quickstart:~


Applications Places System
Sun Nov 1, 01:50 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
Time taken: 60.94 seconds
hive> show partitions emppartition;
OK
m_no=5225228
m_no=89272861
Time taken: 0.367 seconds, Fetched: 2 row(s)
hive> create table emp_bucks_no_partitions(
> emp_id int,
> name string,
> m_no string
>)
> clustered by(m_no) into 5 buckets;
OK
Time taken: 0.195 seconds
hive> insert into emp_bucks_no_partitions
> values(1, 'abc', '929272916');
Query ID = cloudera_20261101013232_9f64726c-a001-4ce0-8edf-c3af803c0035
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1793445829222_0008, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1793445829222_0008/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1793445829222_0008
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-11-01 01:32:58,110 Stage-1 map = 0%, reduce = 0%
2026-11-01 01:33:13,682 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.49 sec
MapReduce Total cumulative CPU time: 2 seconds 490 msec
Ended Job = job_1793445829222_0008
Stage-4 is selected by condition resolver.
Click to switch to "Workspace 2"
cloudera@quickstart:~


Applications Places System
Sun Nov 1, 01:50 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/emp_bucks_no_partitions/.hive-staging_hive_2026-11-01_01-32-32_977_6773999690988478134-1/-ext-10000
Loading data to table default.emp_bucks_no_partitions
Table default.emp_bucks_no_partitions stats: [numFiles=1, numRows=1, totalSize=16, rawDataSize=15]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 2.49 sec HDFS Read: 4166 HDFS Write: 103 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 490 msec
OK
Time taken: 42.58 seconds
hive> describe formatted emp_bucks_no_partitions;
OK
col_name data_type comment
emp_id int
name string
m_no string
Detailed Table Information
Database: default
Owner: cloudera
CreateTime: Sun Nov 01 01:32:19 PST 2026
LastAccessTime: UNKNOWN
Protect Mode: None
Retention: 0
Location: hdfs://quickstart.cloudera:8020/user/hive/warehouse/emp_bucks_no_partitions
Click to switch to "Workspace 2"
cloudera@quickstart:~


Applications Places System
Sun Nov 1, 01:52 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
Table Type: MANAGED_TABLE
Table Parameters:
COLUMN_STATS_ACCURATE true
numFiles 1
numRows 1
rawDataSize 15
totalSize 16
transient_lastDdlTime 1793525595
Storage Information
SerDe Library: org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe
InputFormat: org.apache.hadoop.mapred.TextInputFormat
OutputFormat: org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat
Compressed: No
Num Buckets: 5
Bucket Columns: [m_no]
Sort Columns: []
Storage Desc Params:
serialization.format 1
Time taken: 0.225 seconds, Fetched: 33 row(s)
hive> create table emp_bucks_with_partition(
> emp_id int,
> name string,
> m_no string
>)
> partitioned by(m_no2 string)
> clustered by(m_no) into 5 buckets;
OK
Time taken: 0.175 seconds
Click to switch to "Workspace 2"
cloudera@quickstart:~


Applications Places System
Sun Nov 1, 01:53 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
hive> set hive.exec.dynamic.partition.mode=nonstrict;
hive> insert into emp_bucks_with_partition
> partition(m_no2)
> values(1, 'xyz', '8927927', '12345');
Query ID = cloudera_20261101013535_93a248a4-4f54-487b-b096-1e856c44e682
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1793445829222_0009, Tracking URL = http://quickstart.cloudera:8088/proxy/appli
cation_1793445829222_0009/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1793445829222_0009
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-11-01 01:36:06,495 Stage-1 map = 0%, reduce = 0%
2026-11-01 01:36:26,296 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.63 sec
MapReduce Total cumulative CPU time: 2 seconds 630 msec
Ended Job = job_1793445829222_0009
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/emp_bucks_with_partition/.hiv
e-staging_hive_2026-11-01_01-35-49_225_3916188812674291350-1/-ext-10000
Loading data to table default.emp_bucks_with_partition partition (m_no2=null)
Time taken for load dynamic partitions : 337
Loading partition {m_no2=12345}
Time taken for adding to write entity : 0
Partition default.emp_bucks_with_partition{m_no2=12345} stats: [numFiles=1, numRows=1, totalSize=
14, rawDataSize=13]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 2.63 sec HDFS Read: 4651 HDFS Write: 114 SUCCESS
cloudera@quickstart:~


Applications Places System
Sun Nov 1, 01:53 cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
Starting Job = job_1793445829222_0009, Tracking URL = http://quickstart.cloudera:8088/proxy/appli
cation_1793445829222_0009/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1793445829222_0009
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-11-01 01:36:06,495 Stage-1 map = 0%, reduce = 0%
2026-11-01 01:36:26,296 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.63 sec
MapReduce Total cumulative CPU time: 2 seconds 630 msec
Ended Job = job_1793445829222_0009
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/emp_bucks_with_partition/.hiv
e-staging_hive_2026-11-01_01-35-49_225_3916188812674291350-1/-ext-10000
Loading data to table default.emp_bucks_with_partition partition (m_no2=null)
Time taken for load dynamic partitions : 337
Loading partition {m_no2=12345}
Time taken for adding to write entity : 0
Partition default.emp_bucks_with_partition{m_no2=12345} stats: [numFiles=1, numRows=1, totalSize=
14, rawDataSize=13]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 2.63 sec HDFS Read: 4651 HDFS Write: 114 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 630 msec
OK
Time taken: 40.337 seconds
hive> select * from emp_bucks_with_partition;
OK
1 xyz 8927927 12345
Time taken: 0.377 seconds, Fetched: 1 row(s)
hive>
cloudera@quickstart:~

Practical No: 07

AIM: To perform Create, Read, Update, and Delete (CRUD) operations and execute advanced queries using MongoDB, including the use of conditional operators, indexing, and sorting on document-based data. (An e-commerce dataset containing products, users, and reviews in JSON format can be considered.)

Date: 26/09/26

Name: Mohd Danish Ansari

Roll No: 13

THEORY:

1. MongoDB is a NoSQL database that stores data in the form of documents using a flexible JSON-like format called BSON.

2. CRUD operations are used to manage documents in MongoDB. Insert is used to add documents, Find to retrieve documents, Update to modify documents and Delete to remove documents.

3. MongoDB provides conditional operators such as $gt, $gte, $in, $and, $or and $not to retrieve documents based on specific conditions.

4. Projection, sorting and aggregation are used to display selected fields, arrange documents in a particular order and perform operations such as grouping, counting and calculating totals.

5. Indexing is used to improve query performance by allowing faster data retrieval. MongoDB also provides commands to delete documents, drop collections and drop databases.

Code :

```
// Show all databases available in MongoDB
```

```
test> show dbs;
admin    40.00 KiB
config   48.00 KiB
local    96.00 KiB
```

// Switch to the "local" and "danish" database

```
test> use('local');
switched to db local
local> use('danish');
switched to db danish
```

// Create a collection named "royal" and Show all collections in the current database

```
sara> db.createCollection("royal");
{ ok: 1 }
sara> show collections;
royal
```

INSERT DOCUMENTS

// Insert multiple royal-family documents into the "royal" collection

```
// insertMany() is used to insert more than one document at once
```

```
sara> db.royal.insertMany([
...   {
...     name: "Arjun",
...     house: "Windmere",
...     age: 34,
...     title: "Duke",
...     gold: 1200,
...     active: true,
...     tags: ["Warrior", "Diploma"]
...   },
...   {
...     name: "Meera",
...     house: "Windmere",
...     age: 29,
...     title: "Countess",
...     gold: 950,
...     active: true,
...     tags: ["Scholar"]
...   },
...   {
...     name: "Rohan",
...     house: "Ashford",
sara>
...     title: "King",
...     gold: 5000,
...     active: true,
...     tags: ["Warrior", "Ruler"]
...   },
...   {
...     name: "Isha",
...     house: "Ashford",
...     age: 22,
...     title: "Princess",
...     gold: 300,
...     active: false,
...     tags: ["Scholar", "Diploma"]
...   },
...   {
...     name: "Kabir",
...     house: "Ravenholt",
...     age: 55,
...     title: "Lord",
...     gold: 2200,
...     active: true,
...     tags: ["Strategist"]
...   },
...   {
...     name: "Tara",
...     house: "Ravenholt",
...     age: 19,
...     title: "Squire",
...     gold: 50,
...     active: false,
...     tags: []
...   }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae34e374203469fe84eeb86'),
    '1': ObjectId('6ae34e374203469fe84eeb87'),
    '2': ObjectId('6ae34e374203469fe84eeb88'),
    '3': ObjectId('6ae34e374203469fe84eeb89'),
    '4': ObjectId('6ae34e374203469fe84eeb8a'),
    '5': ObjectId('6ae34e374203469fe84eeb8b')
  }
```

BASIC READ / FIND OPERATIONS

// Display all documents in the "royal" collection

// pretty() formats the output in a more readable way



// Find and display only one document

// findOne() returns the first matching document

```
sara> db.royal.findOne();
{
  _id: ObjectId('6ae34e374203469fe84eeb86'),
  name: 'Arjun',
  house: 'Windmere',
  age: 34,
  title: 'Duke',
  gold: 1200,
  active: true,
  tags: [ 'Warrior', 'Diploma' ]
}
```

// Find all documents where the house is "Ashford"

```
sara> db.royal.find({
...   house: "Ashford"
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb89'),
    name: 'Isha',
    house: 'Ashford',
    age: 22,
    title: 'Princess',
    gold: 300,
    active: false,
    tags: [ 'Scholar', 'Diploma' ]
  }
]
```

// Find all documents where "Warrior" exists inside the tags array

// MongoDB can search array elements directly

```
sara> db.royal.find({
...   tags: "Warrior"
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  }
]
```

// Find all people whose age is greater than 30

// $gt means "greater than"

```
sara> db.royal.find({
...   age: { $gt: 30 }
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true,
    tags: [ 'Strategist' ]
  }
]
```

// Find all people having gold greater than or equal to 1000

// $gte means "greater than or equal to"

```
sara> db.royal.find({
...   gold: { $gte: 1000 }
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true,
    tags: [ 'Strategist' ]
  }
]
```

// Find people whose title is either "King" or "Duke"

// $in matches any value from the given array

```
sara> db.royal.find({
...   title: { $in: ["King", "Duke"] }
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  }
]
```

// Find people who belong to Windmere AND are active

// Multiple fields in the same query act like AND



// Find people whose age is between 25 and 45

// $and explicitly combines multiple conditions

```
sara> db.royal.find({
...   $and: [
...     { age: { $gte: 25 } },
...     { age: { $lte: 45 } }
...   ]
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb87'),
    name: 'Meera',
    house: 'Windmere',
    age: 29,
    title: 'Countess',
    gold: 950,
    active: true,
    tags: [ 'Scholar' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  }
]
```

// Find people who are from Ravenholt OR have more than 4000 gold

// $or is used when any one of the conditions can be true

```
sara> db.royal.find({
...   $or: [
...     { house: "Ravenholt" },
...     { gold: { $gt: 4000 } }
...   ]
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true,
    tags: [ 'Strategist' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8b'),
    name: 'Tara',
    house: 'Ravenholt',
    age: 19,
    title: 'Squire',
    gold: 50,
    active: false,
    tags: []
  }
]
```

// Find people whose age is NOT less than 25

// $not reverses the specified condition

```
sara> db.royal.find({
...   age: {
...     $not: { $lt: 25 }
...   }
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb87'),
    name: 'Meera',
    house: 'Windmere',
    age: 29,
    title: 'Countess',
    gold: 950,
    active: true,
    tags: [ 'Scholar' ]
sara>
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true,
    tags: [ 'Strategist' ]
  }
]
```

// Find active people who are either from Windmere

// OR have more than 2000 gold

```
sara> db.royal.find({
...   active: true,
...   $or: [
...     { house: "Windmere" },
...     { gold: { $gt: 2000 } }
...   ]
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb87'),
    name: 'Meera',
    house: 'Windmere',
    age: 29,
    title: 'Countess',
    gold: 950,
    active: true,
    tags: [ 'Scholar' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true,
    tags: [ 'Strategist' ]
  }
]
```

// PROJECTION

// Display only the name and gold fields

// 1 means include the field

// _id: 0 means hide the automatically generated _id field

```
sara> db.royal.find(
...   {},
...   {
...     name: 1,
...     gold: 1,
...     _id: 0
...   }
... );
[
  { name: 'Arjun', gold: 1200 },
  { name: 'Meera', gold: 950 },
  { name: 'Rohan', gold: 5000 },
  { name: 'Isha', gold: 300 },
  { name: 'Kabir', gold: 2200 },
  { name: 'Tara', gold: 50 }
]
```

// Display all fields EXCEPT the tags field

// 0 means exclude the field

```
sara> db.royal.find(
...   {},
...   {
...     tags: 0
...   }
... );
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb87'),
    name: 'Meera',
    house: 'Windmere',
    age: 29,
    title: 'Countess',
    gold: 950,
sara>
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb89'),
    name: 'Isha',
    house: 'Ashford',
    age: 22,
sara>
    gold: 300,
    active: false
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8b'),
    name: 'Tara',
    house: 'Ravenholt',
    age: 19,
    title: 'Squire',
    gold: 50,
    active: false
  }
```

// SORTING

// Sort documents by gold in descending order

// -1 means descending order

// 1 means ascending order

```
sara> db.royal.find().sort({
...   gold: -1
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true,
    tags: [ 'Strategist' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb87'),
    name: 'Meera',
    house: 'Windmere',
    age: 29,
    title: 'Countess',
    gold: 950,
    active: true,
    tags: [ 'Scholar' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb89'),
    name: 'Isha',
    house: 'Ashford',
    age: 22,
    title: 'Princess',
    gold: 300,
    active: false,
    tags: [ 'Scholar', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8b'),
    name: 'Tara',
    house: 'Ravenholt',
    age: 19,
    title: 'Squire',
    gold: 50,
    active: false,
    tags: []
  }
]
```

// Sort by age in ascending order

// If two people have the same age, sort their gold in descending order

```
sara> db.royal.find().sort({
...   age: 1,
...   gold: -1
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb8b'),
    name: 'Tara',
    house: 'Ravenholt',
    age: 19,
    title: 'Squire',
    gold: 50,
    active: false,
    tags: []
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb89'),
    name: 'Isha',
    house: 'Ashford',
    age: 22,
    title: 'Princess',
    gold: 300,
    active: false,
    tags: [ 'Scholar', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb87'),
    name: 'Meera',
    house: 'Windmere',
    age: 29,
    title: 'Countess',
    gold: 950,
    active: true,
    tags: [ 'Scholar' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true,
    tags: [ 'Strategist' ]
  }
]
```

// Sort by gold from highest to lowest

// Then return only the first 3 documents

// limit(3) restricts the number of returned documents

```
sara> db.royal.find().sort({ gold: -1 }).limit(3);
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true,
    tags: [ 'Strategist' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  }
]
```

// Sort names alphabetically

// skip(2) skips the first 2 documents

// limit(2) returns the next 2 documents

```
sara> db.royal.find().sort({ name: 1 }).skip(2).limit(2);
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true,
    tags: [ 'Strategist' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb87'),
    name: 'Meera',
    house: 'Windmere',
    age: 29,
    title: 'Countess',
    gold: 950,
    active: true,
    tags: [ 'Scholar' ]
  }
]
```

// ARRAY QUERIES

// Find documents where the "tags" array contains exactly 2 elements

// $size checks the exact number of elements in an array

```
sara> db.royal.find({
...   tags: { $size: 2 }
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb89'),
    name: 'Isha',
    house: 'Ashford',
    age: 22,
    title: 'Princess',
    gold: 300,
sara>
    tags: [ 'Scholar', 'Diploma' ]
  }
]
```

// Find documents where the "tags" array contains BOTH

// "Warrior" AND "Diploma"

// $all matches documents containing all specified array values

```
sara> db.royal.find({
...   tags: { $all: ["Warrior", "Diploma"] }
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  }
]
```

// Find documents where the "tags" array is completely empty

// $size: 0 means the array must contain exactly 0 elements

```
sara> db.royal.find({
...   tags: { $size: 0 }
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb8b'),
    name: 'Tara',
    house: 'Ravenholt',
    age: 19,
    title: 'Squire',
    gold: 50,
    active: false,
    tags: []
  }
]
```

// FIELD EXISTENCE QUERY

// Find documents where the "title" field exists

// $exists: true checks whether the field is present

```
sara> db.royal.find({
...   title: { $exists: true }
... });
[
  {
    _id: ObjectId('6ae34e374203469fe84eeb86'),
    name: 'Arjun',
    house: 'Windmere',
    age: 34,
    title: 'Duke',
    gold: 1200,
    active: true,
    tags: [ 'Warrior', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb87'),
    name: 'Meera',
    house: 'Windmere',
    age: 29,
    title: 'Countess',
    gold: 950,
    active: true,
    tags: [ 'Scholar' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb88'),
    name: 'Rohan',
    house: 'Ashford',
    age: 41,
    title: 'King',
    gold: 5000,
    active: true,
    tags: [ 'Warrior', 'Ruler' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb89'),
    name: 'Isha',
    house: 'Ashford',
    age: 22,
    title: 'Princess',
    gold: 300,
    active: false,
    tags: [ 'Scholar', 'Diploma' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8a'),
    name: 'Kabir',
    house: 'Ravenholt',
    age: 55,
    title: 'Lord',
    gold: 2200,
    active: true,
    tags: [ 'Strategist' ]
  },
  {
    _id: ObjectId('6ae34e374203469fe84eeb8b'),
    name: 'Tara',
    house: 'Ravenholt',
    age: 19,
    title: 'Squire',
    gold: 50,
    active: false,
    tags: []
  }
]
```

// UPDATE OPERATIONS

// Update Tara's gold to 75

// updateOne() updates only the first document matching the condition

// $set changes the value of an existing field or creates it if it doesn't exist

```
sara> db.royal.updateOne(
...   { name: "Tara" },
...   { $set: { gold: 75 } }
... );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

// Make all members of the Ashford house active

// updateMany() updates every document matching the condition

// $set changes the "active" field to true

```
sara> db.royal.updateMany(
...   { house: "Ashford" },
...   { $set: { active: true } }
... );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 1,
  upsertedCount: 0
}
```

// Increase Arjun's gold by 100

// $inc increments a numeric field by the specified amount

// Arjun's gold increases from 1200 to 1300

```
sara> db.royal.updateOne(
...   { name: "Arjun" },
...   { $inc: { gold: 100 } }
... );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

// Add "Squire-in-Training" to Tara's tags

// $addToSet adds a value to an array only if it isn't already present

// Unlike $push, it prevents duplicate values

```
sara> db.royal.updateOne(
...   { name: "Tara" },
...   { $addToSet: { tags: "Squire-in-Training" } }
... );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

// Remove the "title" field from Tara's document

// $unset deletes the specified field from the document

// The value "" is conventionally used with $unset

```
sara> db.royal.updateOne(
...   { name: "Tara" },
...   { $unset: { title: "" } }
... );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

// AGGREGATION

// Group royal members by their house

// $group combines documents that have the same "house" value

// _id: "$house" means each unique house becomes a separate group

// totalGold -> calculates the total gold of all members in each house

// memberCount -> counts the number of members in each house

// avgAge   -> calculates the average age of members in each house

```
sara> db.royal.aggregate([
...   {
...     $group: {
...       _id: "$house",
...       totalGold: { $sum: "$gold" },
...       memberCount: { $sum: 1 },
...       avgAge: { $avg: "$age" }
...     }
...   }
... ]);
[
  { _id: 'Ashford', totalGold: 5300, memberCount: 2, avgAge: 31.5 },
  { _id: 'Windmere', totalGold: 2250, memberCount: 2, avgAge: 31.5 },
  { _id: 'Ravenholt', totalGold: 2275, memberCount: 2, avgAge: 37 }
]
```

// AGGREGATION PIPELINE - MATCH + GROUP + SORT

// Find only active members

// Group them by house

// Calculate the total gold for each house

// Finally, sort the houses by total gold from highest to lowest

// $match -> filters documents

// $group -> groups documents by house

// $sum   -> adds the gold values

// $sort   -> sorts the aggregation results.

```
sara> db.royal.aggregate([
...   {
...     $match: {
...       active: true
...     }
...   },
...   {
...     $group: {
...       _id: "$house",
...       totalGold: { $sum: "$gold" }
...     }
...   },
...   {
...     $sort: {
...       totalGold: -1
...     }
...   }
sara>
[
  { _id: 'Ashford', totalGold: 5300 },
  { _id: 'Windmere', totalGold: 2250 },
  { _id: 'Ravenholt', totalGold: 2200 }
sara>
```

// AGGREGATION - PROJECT + DIVIDE

// Display the name and house of each member

// Convert gold into thousands by dividing the gold value by 1000

// $project -> selects fields and can create calculated fields

// $divide -> divides one value by another

// _id: 0 hides the automatically generated _id field

```
sara> db.royal.aggregate([
...   {
...     $project: {
...       name: 1,
...       house: 1,
...       goldInThousands: {
...         $divide: ["$gold", 1000]
...       },
...       _id: 0
...     }
...   }
sara>
[
  { name: 'Arjun', house: 'Windmere', goldInThousands: 1.3 },
  { name: 'Meera', house: 'Windmere', goldInThousands: 0.95 },
  { name: 'Rohan', house: 'Ashford', goldInThousands: 5 },
  { name: 'Isha', house: 'Ashford', goldInThousands: 0.3 },
  { name: 'Kabir', house: 'Ravenholt', goldInThousands: 2.2 },
  { name: 'Tara', house: 'Ravenholt', goldInThousands: 0.075 }
sara>
```

// AGGREGATION - UNWIND + GROUP + SORT

// Break the "tags" array into separate documents

// Count how many times each tag appears

// Sort the tags from most common to least common

// $unwind -> creates a separate document for each array element

// $group -> groups documents by tag

// $sum: 1 -> counts the number of documents

// $sort -> sorts the count in descending order

```
sara> db.royal.aggregate([
...   {
...     $unwind: "$tags"
...   },
...   {
...     $group: {
...       _id: "$tags",
...       count: {
...         $sum: 1
...       }
...     }
...   },
...   {
...     $sort: {
...       count: -1
...     }
...   }
sara>
[
  { _id: 'Scholar', count: 2 },
  { _id: 'Diploma', count: 2 },
  { _id: 'Warrior', count: 2 },
  { _id: 'Strategist', count: 1 },
  { _id: 'Squire-in-Training', count: 1 },
  { _id: 'Ruler', count: 1 }
sara>
```

// INDEX OPERATIONS

// Display all indexes currently created on the "royal" collection

// MongoDB automatically creates an index on the "_id" field

```
sara> db.royal.getIndexes();
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
sara>
```

// Create a single-field index on the "house" field

// 1 means ascending order

// This index can improve queries that search or sort using "house"

```
sara> db.royal.createIndex({
...   house: 1
... });
house_1
sara>
```

// Create a compound index on "house" and "gold"

// house: 1 -> ascending order

// gold: -1 -> descending order

```
sara> db.royal.createIndex({
...   house: 1,
...   gold: -1
... });
house_1_gold_-1
sara>
```

// DELETE OPERATIONS

// Delete one document where the name is "Tara"

// deleteOne() removes only the first document matching the condition

```
sara> db.royal.deleteOne({
...   name: "Tara"
... });
{ acknowledged: true, deletedCount: 0 }
sara>
```

// Delete all documents where active is false

// deleteMany() removes every document matching the condition

```
sara> db.royal.deleteMany({
...   active: false
... })
{ acknowledged: true, deletedCount: 0 }
sara>
```

// DROP COLLECTION

// Delete the entire "royal" collection

// This removes the collection AND all documents inside it

```
sara> db.royal.drop();
true
sara>
```

// DROP DATABASE

// Delete the entire current database

```
sara> db.dropDatabase();
{ ok: 1, dropped: 'sara' }
sara>
```

Practical No: 08

AIM: To implement distributed data storage using Cassandra by creating key spaces and tables and performing data operations, and to model and analyze relationships using a graph database through Neo4j by creating nodes, relationships, and executing Cypher queries. (Time-series or transaction datasets for Cassandra and social network datasets for Neo4j can be considered.)

Date: 30/09/26

Name: Mohd Danish Ansari

Roll No: 13

Theory:

1. Introduction to Neo4j: Neo4j is a NoSQL graph database used to store and manage highly connected data. It represents data as nodes and relationships instead of traditional rows and columns.

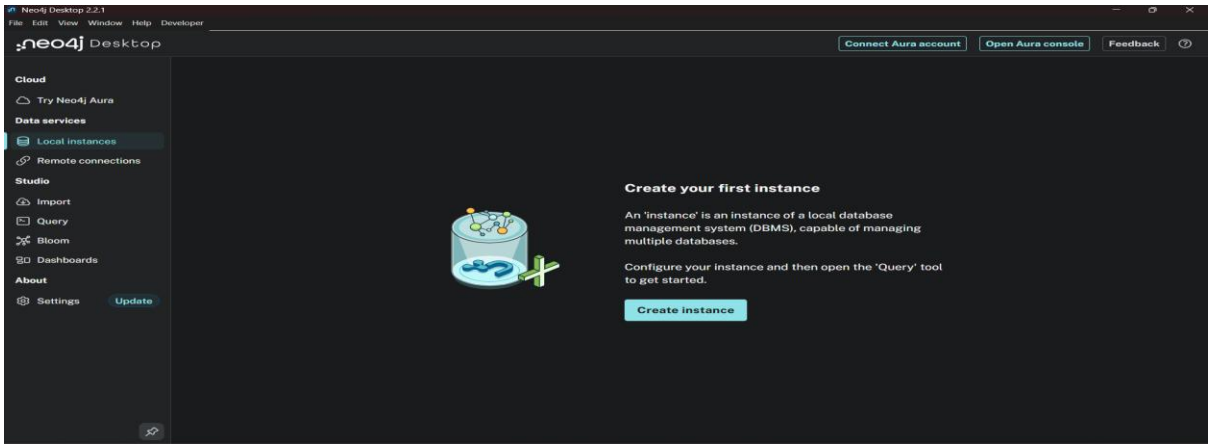
2. Nodes and Properties: Nodes represent real-world entities such as people, students, or products. Each node can have labels and properties that describe its characteristics, such as name, age, or department.

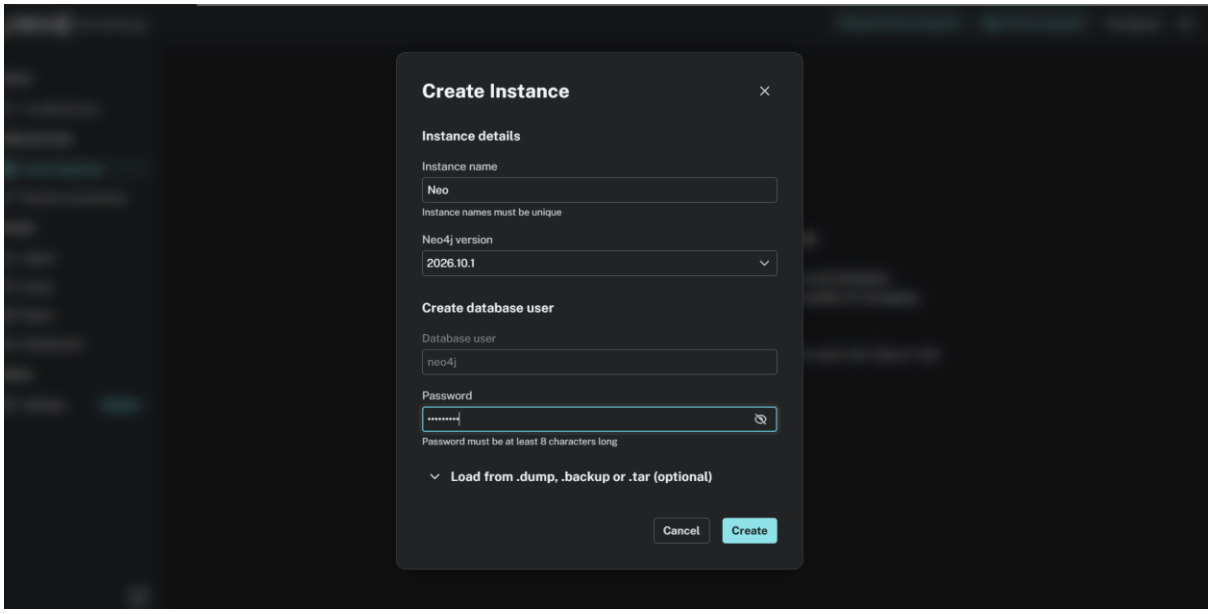
3. Relationships in Neo4j: Relationships connect nodes and represent how entities are related. They have a direction, a type, and can also contain properties. For example, a STUDIES relationship can connect a student node to a course node.

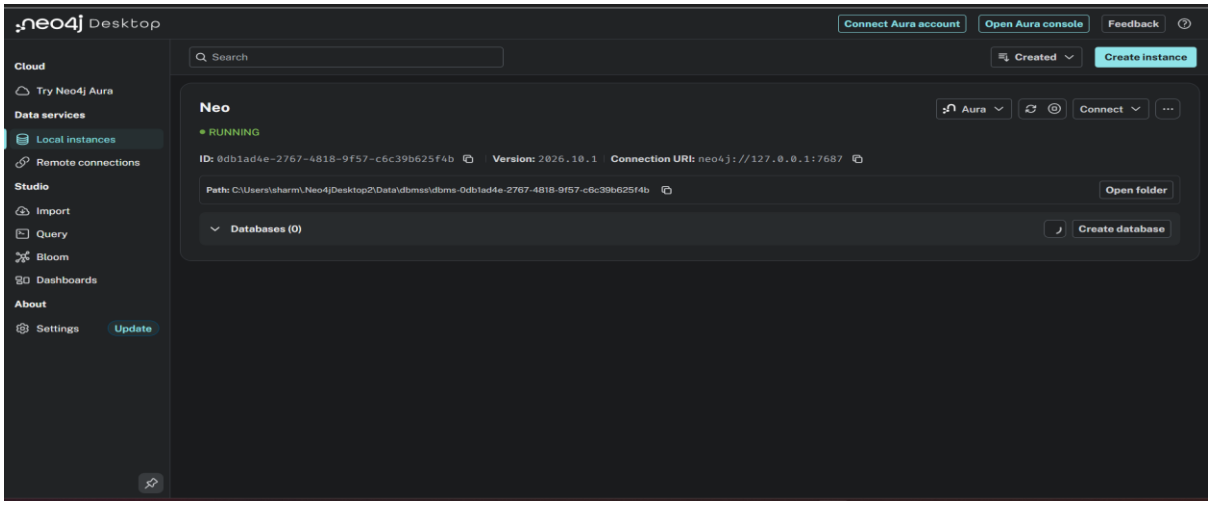
4. Cypher Query Language: Cypher is Neo4j's query language used to create, retrieve, update, and delete nodes and relationships. Commands such as CREATE, MATCH, SET, and DELETE help perform these operations.

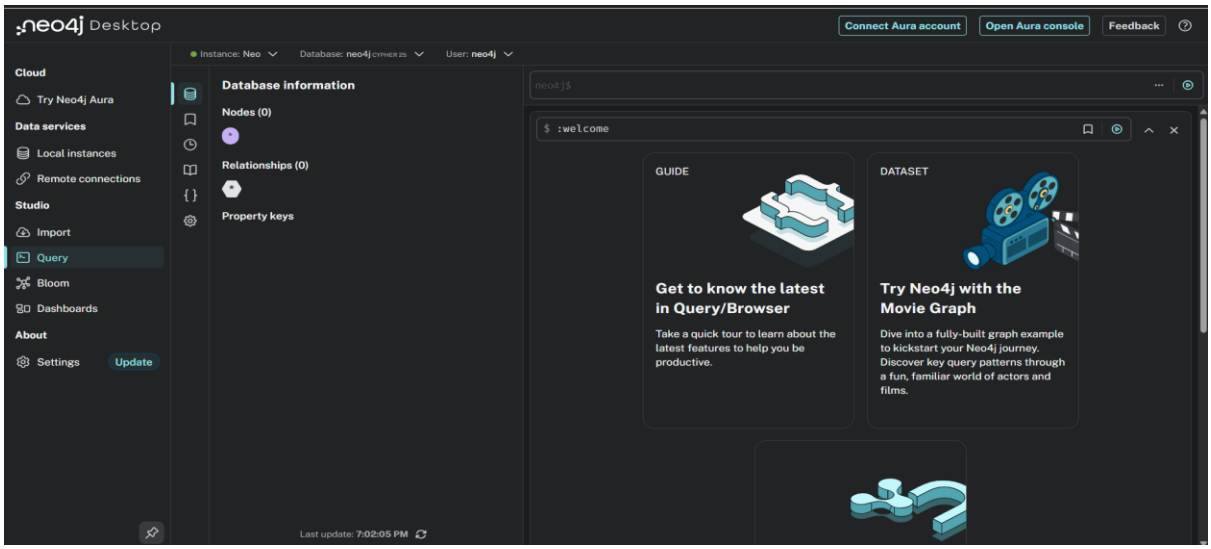
5. Graph Analysis and Applications: Neo4j allows users to explore connections and identify patterns in data using Cypher queries. It is useful in applications such as social networks, recommendation systems, fraud detection, and network analysis.

Code:





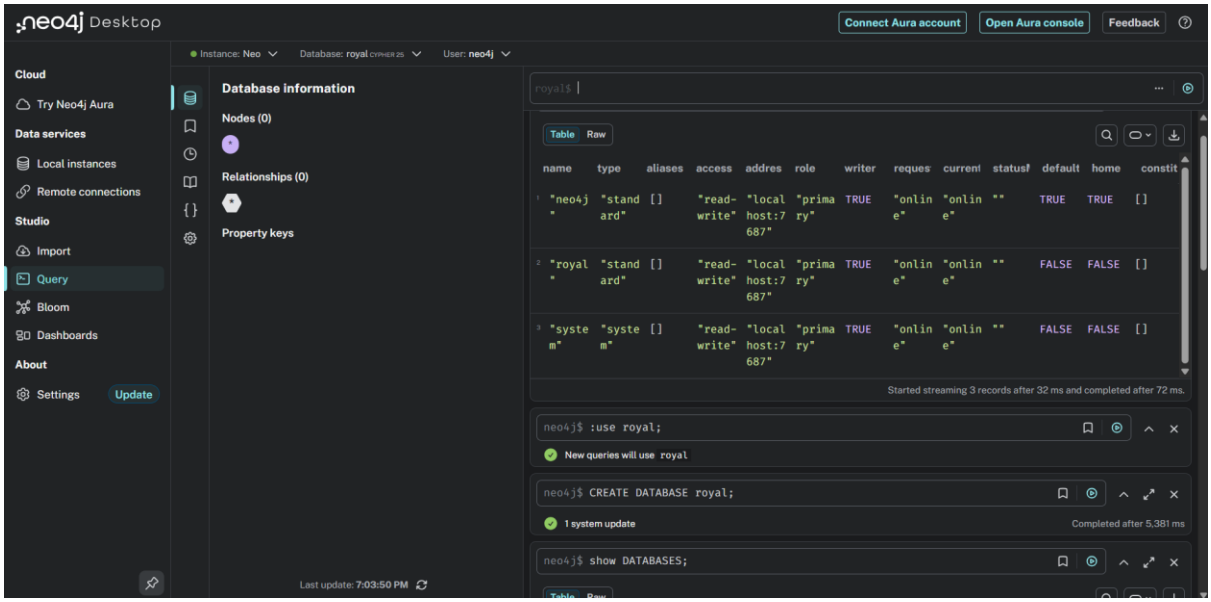




show DATABASES;

CREATE DATABASE royal;

:use royal;

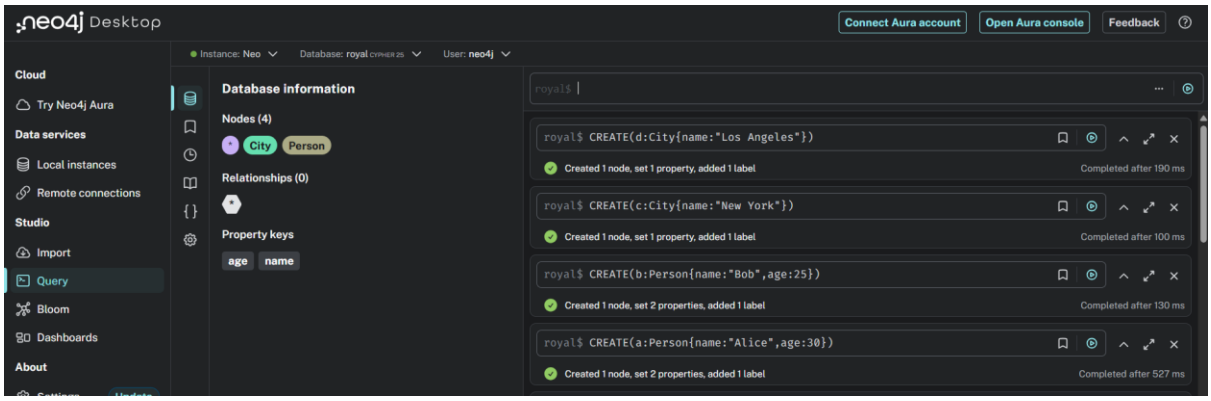


CREATE(a:Person{name:"Alice",age:30})

CREATE(b:Person{name:"Bob",age:25})

CREATE(c:City{name:"New York"})

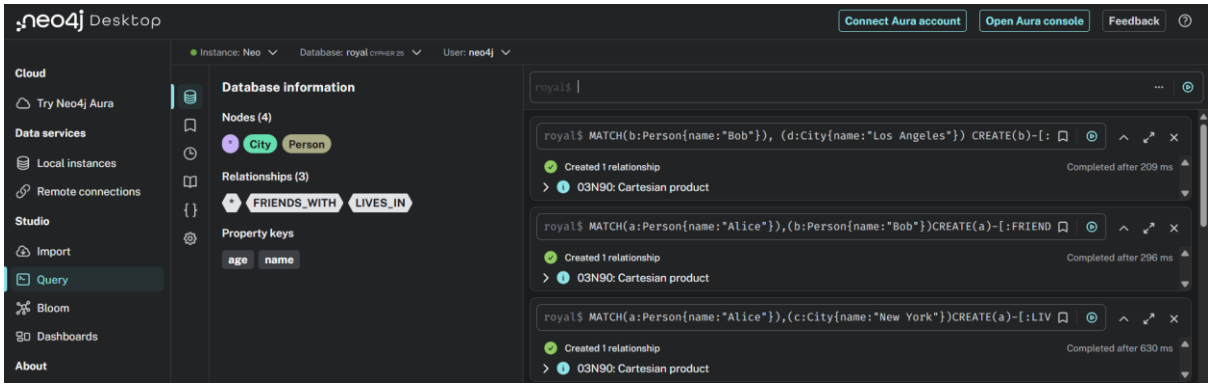
CREATE(d:City{name:"Los Angeles"})



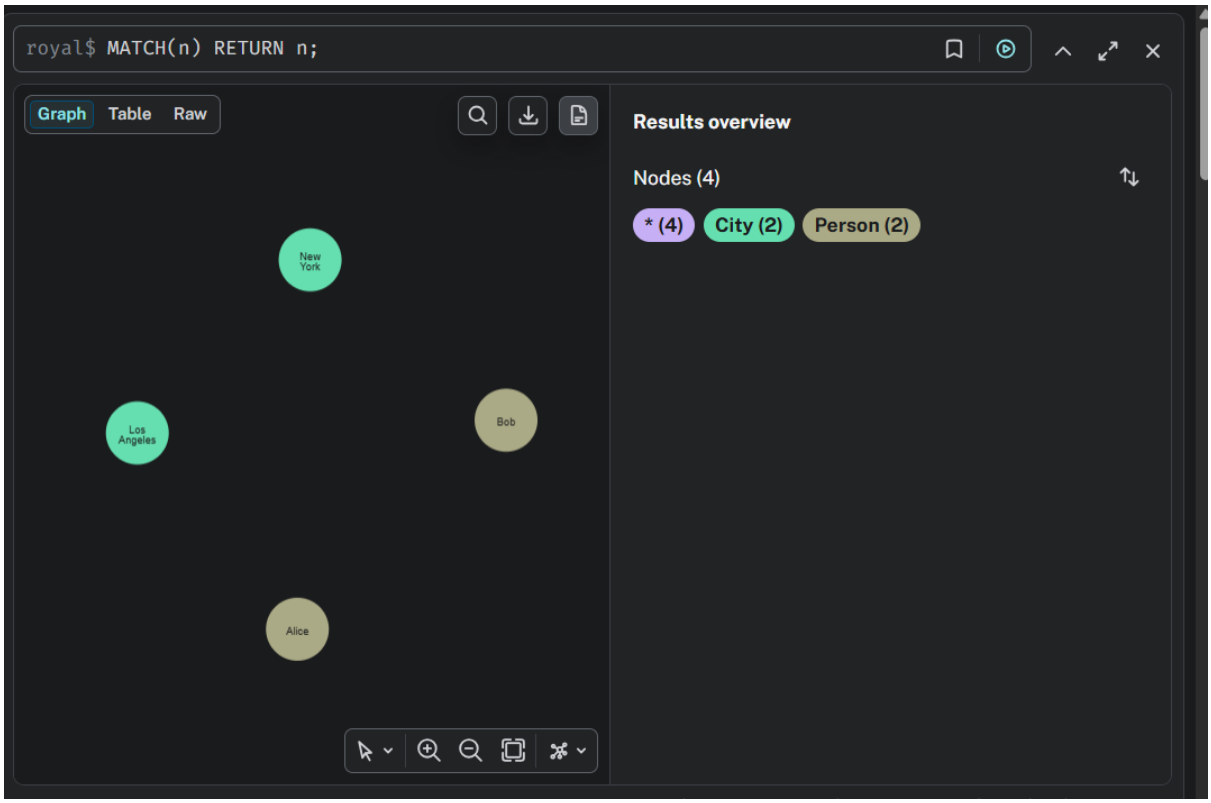
MATCH(a:Person{name:"Alice"}),(c:City{name:"New York"})CREATE(a)-[:LIVES_IN]->(c);

MATCH(a:Person{name:"Alice"}),(b:Person{name:"Bob"})CREATE(a)-[:FRIENDS_WITH]->(b);

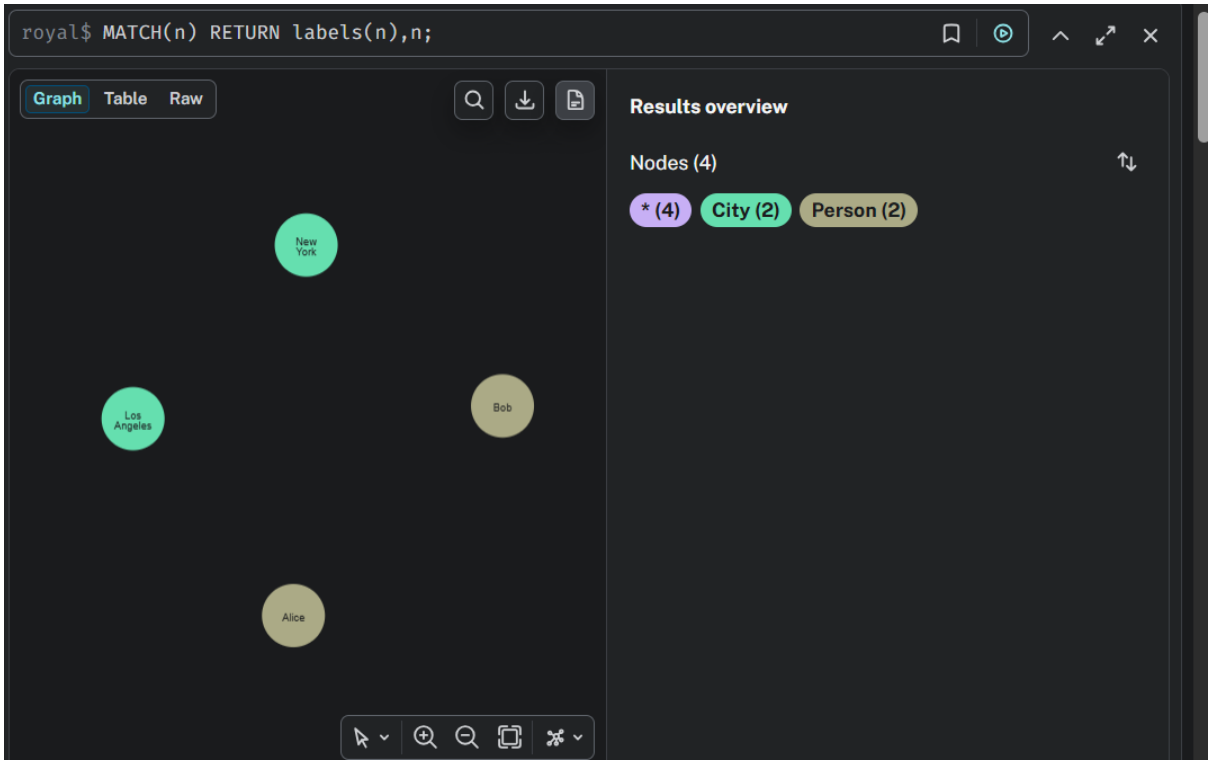
MATCH(b:Person{name:"Bob"}), (d:City{name:"Los Angeles"}) CREATE(b)-[:LIVES_IN]->(d);



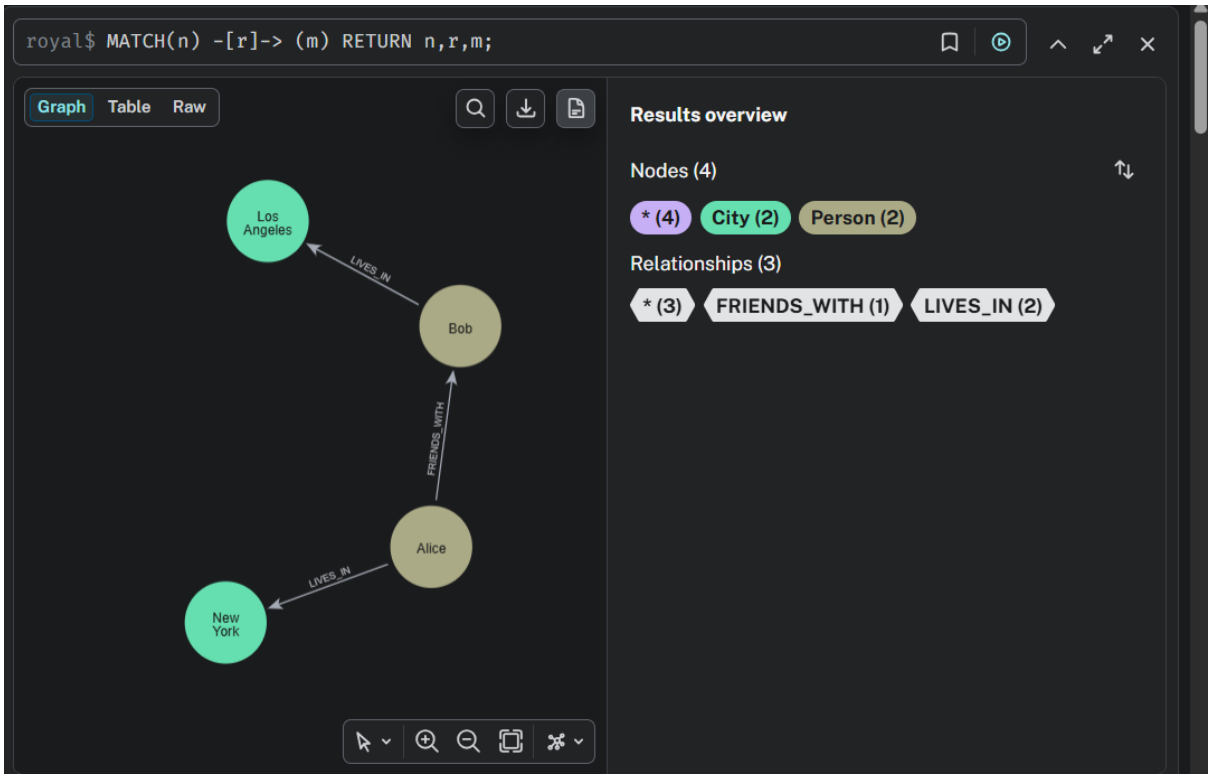
MATCH(n) RETURN n;



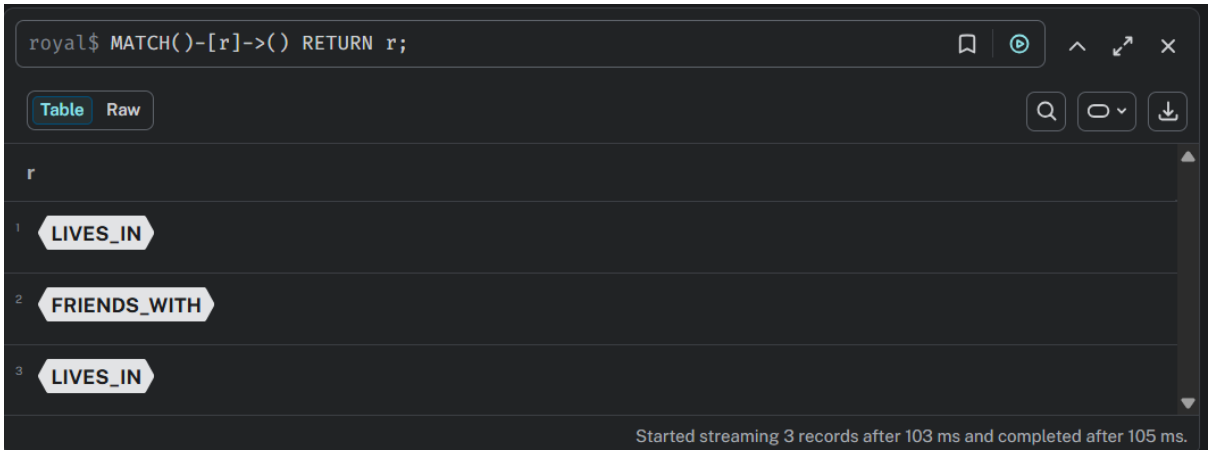
MATCH(n) RETURN labels(n),n;



MATCH(n) -[r]-> (m) RETURN n,r,m;



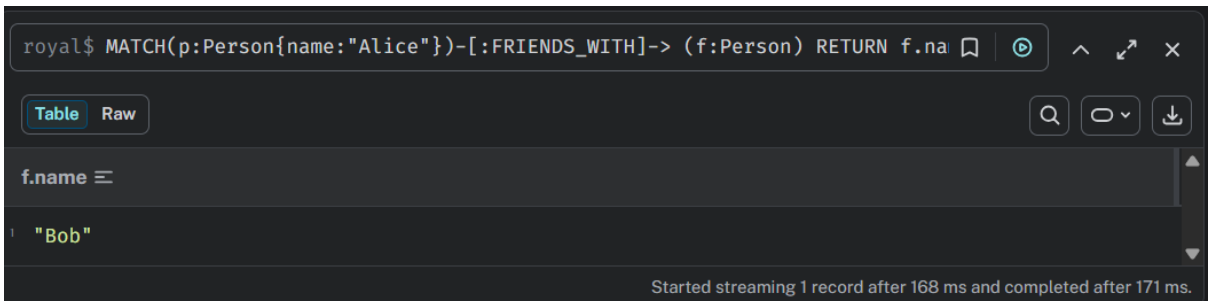
MATCH()-[r]->() RETURN r;



MATCH(p:Person)-[:LIVES_IN]-> (c:City{name:"New York"}) RETURN p.name;

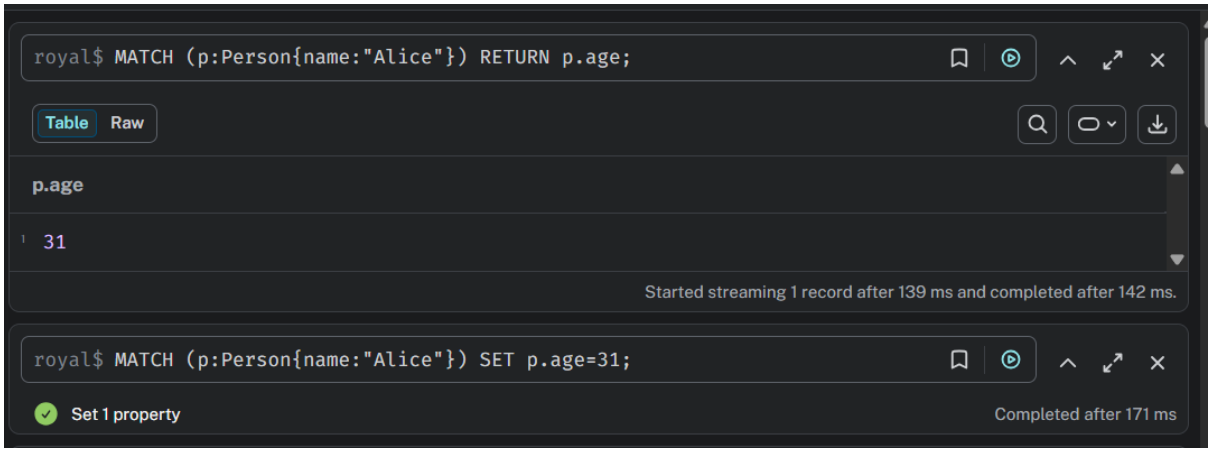


MATCH(p:Person{name:"Alice"})-[:FRIENDS_WITH]-> (f:Person) RETURN f.name;



MATCH (p:Person{name:"Alice"}) SET p.age=31;

MATCH (p:Person{name:"Alice"}) RETURN p.age;



MATCH(a:Person{name:"Alice"})-[r:FRIENDS_WITH]-> (b:Person{name:"Bob"}) DELETE r;

MATCH(n:Person{name:"Alice"}) -[r]-() DELETE r;

MATCH(n:Person{name:"Alice"}) DELETE n;

